

Cell 1 — setup + load the validated tract-year panel for the new cluster-time workflow

```
In [47]: # Cell 1 — setup + load the validated tract-year panel for the new cluster-t

from pathlib import Path
import numpy as np
import pandas as pd

pd.set_option("display.max_columns", 200)
pd.set_option("display.width", 200)

# -----
# Paths / core settings
# -----
DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
OUT_DIR = DATA_DIR / "processed"

LOW_IGS_THRESHOLD = 45
ANALYSIS_YEARS = list(range(2017, 2025)) # ACS-aligned years only
PRE_COVID_YEARS = [2017, 2018, 2019]
COVID_YEARS = [2020, 2021]
RECOVERY_YEARS = [2022, 2023, 2024]

ECON_VARS = [
    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_16p",
]

CLUSTER_CORE_VARS = [
    "igs_total",
    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_16p",
    "pop_total",
]

DISPLAY_COLS = [
    "display_state",
    "display_county",
    "display_name",
]

REQUIRED_COLS = [
    "geoid",
    "year",
    "igs_total",
```

```

    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_16p",
    "pop_total",
]

PANEL_CANDIDATES = [
    OUT_DIR / "eda_panel_clean.parquet",
    OUT_DIR / "igs_x_acs_full.parquet",
]

def normalize_geoid_series(series: pd.Series) -> pd.Series:
    s = series.astype("string").str.strip()
    numeric = pd.to_numeric(s, errors="coerce")
    numeric_mask = numeric.notna()
    s = s.copy()
    s.loc[numeric_mask] = numeric.loc[numeric_mask].astype("Int64").astype("string")
    s = s.str.replace(r"\.0$", "", regex=True)
    s = s.str.replace(r"\D", "", regex=True)
    s = s.str.zfill(11)
    return s.where(s.str.fullmatch(r"\d{11}"), pd.NA)

def add_display_fields(df: pd.DataFrame) -> pd.DataFrame:
    out = df.copy()

    if "display_state" not in out.columns:
        if "State" in out.columns or "state" in out.columns:
            out["display_state"] = out.get("State", pd.Series(index=out.index, data=""))
            out.get("state", pd.Series(index=out.index, data=""))
        else:
            out["display_state"] = pd.NA

    if "display_county" not in out.columns:
        if "County" in out.columns or "county" in out.columns:
            out["display_county"] = out.get("County", pd.Series(index=out.index, data=""))
            out.get("county", pd.Series(index=out.index, data=""))
        else:
            out["display_county"] = pd.NA

    if "display_name" not in out.columns:
        if "NAME" in out.columns:
            out["display_name"] = out["NAME"].astype("string")
        elif "Census Tract FIPS code" in out.columns:
            out["display_name"] = "Census Tract " + out["Census Tract FIPS code"].astype("string")
        else:
            out["display_name"] = out["geoid"].astype("string")

    return out

def assign_period(year: int) -> str:
    if year in PRE_COVID_YEARS:
        return "pre covid"
    elif year in COVID_YEARS:
        return "covid"
    else:
        return "post covid"

```

```

if year in COVID_YEARS:
    return "covid"
if year in RECOVERY_YEARS:
    return "recovery"
return "outside_scope"

def load_analysis_panel() -> tuple[pd.DataFrame, str]:
    # Prefer in-memory validated objects if they already exist in the notebook
    if "eda_df" in globals():
        df = eda_df.copy()
        source = "existing global: eda_df"
    elif "model_df_full" in globals():
        df = model_df_full.copy()
        source = "existing global: model_df_full"
    else:
        df = None
        source = None
        for p in PANEL_CANDIDATES:
            if p.exists():
                df = pd.read_parquet(p)
                source = f"file: {p}"
                break
        if df is None:
            searched = ", ".join(str(p) for p in PANEL_CANDIDATES)
            raise FileNotFoundError(
                "Could not find a processed merged panel. "
                f"Expected one of: {searched}"
            )

    # Standardize core keys
    df = df.copy()
    df["geoid"] = normalize_geoid_series(df["geoid"])
    df["year"] = pd.to_numeric(df["year"], errors="coerce").astype("Int64")

    # Restrict to ACS-aligned period only
    df = df[df["year"].isin(ANALYSIS_YEARS)].copy()
    df["year"] = df["year"].astype(int)

    # Add consistent display fields
    df = add_display_fields(df)

    # Remove duplicate tract-year rows if any slipped in
    df = (
        df.sort_values(["geoid", "year"])
        .drop_duplicates(["geoid", "year"])
        .reset_index(drop=True)
    )

    # Validate required columns
    missing = [c for c in REQUIRED_COLS if c not in df.columns]
    if missing:
        raise ValueError(f"Missing required columns for new workflow: {missing}")

    # Add time-period labels for the new method
    df["analysis_period"] = df["year"].map(assign_period)

```

```

    return df, source

panel_df, PANEL_SOURCE = load_analysis_panel()

analysis_cols = (
    ["geoid", "year", "analysis_period"] +
    DISPLAY_COLS +
    CLUSTER_CORE_VARS
)

analysis_panel = panel_df[analysis_cols].copy()

print("Loaded panel from:", PANEL_SOURCE)
print("Analysis panel shape:", analysis_panel.shape)
print("Unique tracts:", analysis_panel["geoid"].nunique())
print("Years:", sorted(analysis_panel["year"].unique().tolist()))
print("Periods:", analysis_panel["analysis_period"].value_counts(dropna=False))
print()
print("Missingness in core vars:")
print(analysis_panel[CLUSTER_CORE_VARS].isna().mean().sort_values(ascending=False))
print()
print(analysis_panel.head())

```

```

Loaded panel from: file: data\processed\eda_panel_clean.parquet
Analysis panel shape: (680256, 13)
Unique tracts: 85032
Years: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]
Periods: {'pre_covid': 255096, 'recovery': 255096, 'covid': 170064}

```

Missingness in core vars:

```

median_household_income    0.116411
poverty_rate               0.111802
unemp_rate                 0.111357
lfpr_l6p                   0.109857
pop_total                  0.104783
igs_total                  0.008863
igs_economy                0.001313
dtype: float64

```

	geoid	year	analysis_period	display_state	display_county	display_name	igs_total	igs_economy	poverty_rate	unemp_rate	median_household_income	lfpr_l6p
0	01001020100	2017	pre_covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	67826.0	0.622387	47.0	34.0	0.106775	0.045504
1	01001020100	2018	pre_covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	58625.0	0.618438	52.0	48.0	0.113365	0.036232
2	01001020100	2019	pre_covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	60208.0	0.606061	46.0	40.0	0.166583	0.028571
3	01001020100	2020	covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	60388.0	0.546855	46.0	48.0	0.136528	0.021127
4	01001020100	2021	covid	Alabama	Autauga County	Census Tract 201, Autauga County, Alabama	57399.0	0.534035	38.0	32.0	0.153546	0.024967

```

pop_total
0    1845.0
1    1923.0
2    1993.0
3    1941.0
4    1791.0

```

Cell 2 — reuse or rebuild tract -> cluster mapping from the validated adjacency work

```

In [48]: # Cell 2 – broadened rebuild of tract -> cluster mapping
# Goal:
# 1) keep the same vulnerability logic
# 2) broaden the actual candidate universe upstream
# 3) prevent the seed-county stage from collapsing into only one geography
# 4) keep adjacency clustering as the foundation

from pathlib import Path
from collections import defaultdict, deque
import zipfile

```

```

import requests
import geopandas as gpd
import numpy as np
import pandas as pd

# -----
# Settings
# -----
MIN_CLUSTER_TRACTS = 3
MIN_CLUSTER_POP = 5000

# Larger national pool used only to identify where strong signals exist
COUNTY_SIGNAL_POOL_N = 1500

# Diversified county selection
TARGET_SEED_COUNTIES_N = 40
PER_STATE_BASE_COUNTIES = 1
PER_STATE_MAX_COUNTIES = 3
MIN_SIGNAL_TRACTS_PER_COUNTY = 2

# County expansion after seed selection
BROADER_MAX_PER_COUNTY = 35

# Add one-hop tract connectors so communities do not fragment
CONNECTOR_HOPS = 1

TRACT_BASE_URL = "https://www2.census.gov/geo/tiger/TIGER2024/TRACT/"
TRACT_ROOT_DIR = RAW_DIR / "tl_2024_state_tracts"
TRACT_ROOT_DIR.mkdir(parents=True, exist_ok=True)

# -----
# Helpers
# -----
def bad_when_high(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return s.rank(pct=True, method="average")

def bad_when_low(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return (-s).rank(pct=True, method="average")

def load_state_tract_geometry(state_fips_list):
    tract_gdf_list = []

    for st in state_fips_list:
        zip_name = f"tl_2024_{st}_tract.zip"
        shp_name = f"tl_2024_{st}_tract.shp"
        zip_path = TRACT_ROOT_DIR / zip_name
        extract_dir = TRACT_ROOT_DIR / f"tl_2024_{st}_tract"
        shp_path = extract_dir / shp_name

        if not shp_path.exists():
            extract_dir.mkdir(parents=True, exist_ok=True)

            if not zip_path.exists():
                url = f"{TRACT_BASE_URL}{zip_name}"

```

```

        print(f"Downloading {url}")
        r = requests.get(url, timeout=120)
        r.raise_for_status()
        with open(zip_path, "wb") as f:
            f.write(r.content)

        print(f"Extracting {zip_name}")
        with zipfile.ZipFile(zip_path, "r") as zf:
            zf.extractall(extract_dir)

    if not shp_path.exists():
        raise FileNotFoundError(f"Missing shapefile after extraction: {s

    gdf_state = gpd.read_file(shp_path)
    tract_gdf_list.append(gdf_state)

tract_gdf = pd.concat(tract_gdf_list, ignore_index=True)
tract_gdf = gpd.GeoDataFrame(
    tract_gdf,
    geometry="geometry",
    crs=tract_gdf_list[0].crs if tract_gdf_list else None
)

tract_gdf["geoid"] = normalize_geoid_series(tract_gdf["GEOID"])
tract_gdf = tract_gdf.dropna(subset=["geoid", "geometry"]).copy()
tract_gdf = tract_gdf[tract_gdf.geometry.notna()].copy()
tract_gdf = tract_gdf[~tract_gdf.geometry.is_empty].copy()
tract_gdf = tract_gdf.drop_duplicates(subset=["geoid"]).copy()

return tract_gdf

def build_adjacency_edges_fast(gdf: gpd.GeoDataFrame) -> pd.DataFrame:
    """
    Build tract-touching edges within each county.
    """
    edge_frames = []

    for (state_name, county_name), grp in gdf.groupby(
        ["display_state", "display_county"],
        dropna=False
    ):
        grp = grp[["geoid", "display_state", "display_county", "geometry"]].copy()
        grp = grp.dropna(subset=["geoid", "geometry"]).copy()

        if len(grp) < 2:
            continue

        left = grp[["geoid", "geometry"]].copy()
        right = grp[["geoid", "geometry"]].copy()

        joined = gpd.sjoin(left, right, how="inner", predicate="touches")
        if joined.empty:
            continue

        joined = joined.reset_index(drop=True)
        joined = joined.rename(columns={"geoid_left": "geoid_1", "geoid_right": "geoid_2", "display_state": "state", "display_county": "county"})

```

```

        joined = joined[joined["geoid_1"] != joined["geoid_2"]].copy()
        if joined.empty:
            continue

        joined["a"] = joined[["geoid_1", "geoid_2"]].min(axis=1)
        joined["b"] = joined[["geoid_1", "geoid_2"]].max(axis=1)
        joined = joined.drop_duplicates(subset=["a", "b"]).copy()

        out = joined[["a", "b"]].rename(columns={"a": "geoid_1", "b": "geoid_2"})
        out["display_state"] = state_name
        out["display_county"] = county_name
        edge_frames.append(out)

    if not edge_frames:
        return pd.DataFrame(columns=["display_state", "display_county", "geoid_1", "geoid_2"])

    return pd.concat(edge_frames, ignore_index=True)

def connected_components_from_edges(nodes, edges_df):
    neighbors = defaultdict(set)

    for n in nodes:
        neighbors[n] = set()

    if edges_df is not None and len(edges_df) > 0:
        for _, row in edges_df.iterrows():
            a = str(row["geoid_1"])
            b = str(row["geoid_2"])
            neighbors[a].add(b)
            neighbors[b].add(a)

    visited = set()
    node_to_cluster_num = {}
    cluster_num = 0

    for n in nodes:
        if n in visited:
            continue

        cluster_num += 1
        q = deque([n])
        visited.add(n)

        while q:
            cur = q.popleft()
            node_to_cluster_num[cur] = cluster_num

            for nxt in neighbors[cur]:
                if nxt not in visited:
                    visited.add(nxt)
                    q.append(nxt)

    return node_to_cluster_num

def diversified_seed_counties(
    county_signal_df: pd.DataFrame,

```



```

target_n: int,
per_state_base: int,
per_state_max: int,
) -> pd.DataFrame:
    """
    Step 1: take a base number of strong counties per state/territory.
    Step 2: backfill nationally until target_n, while respecting state caps.
    """
    county_signal_df = county_signal_df.copy()

    county_signal_df = county_signal_df.sort_values(
        ["display_state", "county_seed_score", "n_signal_tracts", "mean_scope_score"],
        ascending=[True, False, False, False, True]
    ).reset_index(drop=True)

    county_signal_df["state_rank"] = (
        county_signal_df.groupby("display_state").cumcount() + 1
    )

    base = county_signal_df.loc[county_signal_df["state_rank"] <= per_state_base]

    selected = base[
        ["display_state", "display_county"]
    ].drop_duplicates()

    selected_state_counts = (
        selected.groupby("display_state").size().to_dict()
        if len(selected) > 0 else {}
    )

    remaining = county_signal_df.merge(
        selected,
        on=["display_state", "display_county"],
        how="left",
        indicator=True
    )
    remaining = remaining.loc[remaining["_merge"] == "left_only"].drop("_merge", axis=1)
    remaining = remaining.sort_values(
        ["county_seed_score", "n_signal_tracts", "mean_scope_score", "mean_iqr_score"],
        ascending=[False, False, False, True]
    ).reset_index(drop=True)

    chosen_rows = [selected.copy()] if len(selected) > 0 else []

    for _, row in remaining.iterrows():
        state_name = row["display_state"]

        current_ct = selected_state_counts.get(state_name, 0)
        if current_ct >= per_state_max:
            continue

        candidate = pd.DataFrame([
            {
                "display_state": row["display_state"],
                "display_county": row["display_county"]
            }
        ])

```

```

        chosen_rows.append(candidate)
        selected_state_counts[state_name] = current_ct + 1

        total_selected = sum(selected_state_counts.values())
        if total_selected >= target_n:
            break

    if len(chosen_rows) == 0:
        return pd.DataFrame(columns=["display_state", "display_county"])

    out = (
        pd.concat(chosen_rows, ignore_index=True)
        .drop_duplicates(subset=["display_state", "display_county"])
        .reset_index(drop=True)
    )

    return out

# -----
# A. Build recovery-period tract scoring base
# -----
tract_recovery = (
    analysis_panel.loc[analysis_panel["analysis_period"] == "recovery"]
    .groupby(
        ["geoid", "display_state", "display_county", "display_name"],
        as_index=False
    )
    .agg({
        "igs_total": "mean",
        "igs_economy": "mean",
        "poverty_rate": "mean",
        "unemp_rate": "mean",
        "median_household_income": "mean",
        "lfpr_16p": "mean",
        "pop_total": "mean",
    })
    .copy()
)

tract_recovery["score_low_igs"] = (
    bad_when_low(tract_recovery["igs_total"]) +
    bad_when_low(tract_recovery["igs_economy"])
) / 2

tract_recovery["score_econ_vulnerability"] = (
    bad_when_low(tract_recovery["igs_economy"]) +
    bad_when_high(tract_recovery["poverty_rate"]) +
    bad_when_high(tract_recovery["unemp_rate"]) +
    bad_when_low(tract_recovery["median_household_income"]) +
    bad_when_low(tract_recovery["lfpr_16p"])
) / 5

tract_recovery["scope_score"] = 100 * (
    0.50 * tract_recovery["score_low_igs"] +
    0.50 * tract_recovery["score_econ_vulnerability"]
)

```

```

tract_recovery = tract_recovery.sort_values(
    ["scope_score", "igs_total"],
    ascending=[False, True]
).reset_index(drop=True)

# -----
# B. Use a much larger signal pool to identify strong counties
# -----
signal_pool = tract_recovery.head(COUNTY_SIGNAL_POOL_N).copy()

county_signal = (
    signal_pool
    .groupby(["display_state", "display_county"], as_index=False)
    .agg(
        n_signal_tracts=("geoid", "nunique"),
        mean_scope_score=("scope_score", "mean"),
        median_scope_score=("scope_score", "median"),
        mean_igs_total=("igs_total", "mean"),
        min_igs_total=("igs_total", "min"),
        total_signal_pop=("pop_total", "sum"),
    )
    .copy()
)

county_signal = county_signal.loc[
    county_signal["n_signal_tracts"] >= MIN_SIGNAL_TRACTS_PER_COUNTY
].copy()

county_signal["county_seed_score"] = (
    0.40 * bad_when_high(county_signal["n_signal_tracts"]) +
    0.35 * bad_when_high(county_signal["mean_scope_score"]) +
    0.15 * bad_when_low(county_signal["mean_igs_total"]) +
    0.10 * bad_when_low(county_signal["min_igs_total"])
)

county_signal = county_signal.sort_values(
    ["county_seed_score", "n_signal_tracts", "mean_scope_score", "mean_igs_t
    ascending=[False, False, False, True]
).reset_index(drop=True)

seed_counties = diversified_seed_counties(
    county_signal_df=county_signal,
    target_n=TARGET_SEED_COUNTIES_N,
    per_state_base=PER_STATE_BASE_COUNTIES,
    per_state_max=PER_STATE_MAX_COUNTIES,
)

seed_counties = (
    seed_counties
    .merge(
        county_signal[
            ["display_state", "display_county", "n_signal_tracts", "mean_sco
        ],
        on=["display_state", "display_county"],
        how="left",

```

```

        validate="1:1"
    )
    .sort_values(["county_seed_score", "n_signal_tracts", "mean_scope_score"]
    .reset_index(drop=True)
)

# -----
# C. Expand within selected counties
# -----
broader_county_pool = (
    tract_recovery
    .merge(
        seed_counties[["display_state", "display_county"]],
        on=["display_state", "display_county"],
        how="inner"
    )
    .copy()
)

broader_county_pool["county_rank_by_score"] = (
    broader_county_pool
    .groupby(["display_state", "display_county"])["scope_score"]
    .rank(method="first", ascending=False)
)

core_scope_df = (
    broader_county_pool
    .loc[broader_county_pool["county_rank_by_score"] <= BROADER_MAX_PER_COUNTY]
    .drop(columns=["county_rank_by_score"])
    .drop_duplicates(subset=["geoid"])
    .sort_values(["display_state", "display_county", "scope_score"], ascending=True)
    .reset_index(drop=True)
)

# -----
# D. Load geometry only for selected states
# -----
needed_state_fips = sorted(
    core_scope_df["geoid"].astype(str).str[:2].dropna().unique().tolist()
)

tract_gdf = load_state_tract_geometry(needed_state_fips)

tract_geo_scored = tract_gdf.merge(
    tract_recovery,
    on="geoid",
    how="inner",
    validate="1:1"
)

seed_county_gdf = (
    tract_geo_scored
    .merge(
        seed_counties[["display_state", "display_county"]],
        on=["display_state", "display_county"],
        how="inner"
    )
)

```

```

    )
    .copy()
)

seed_county_gdf = seed_county_gdf[seed_county_gdf.geometry.notna()].copy()
seed_county_gdf = seed_county_gdf[~seed_county_gdf.geometry.is_empty].copy()

# -----
# E. Add one-hop geographic connector tracts
# -----
full_seed_edges = build_adjacency_edges_fast(seed_county_gdf)

core_nodes = set(core_scope_df["geoid"].astype(str).tolist())
expanded_nodes = set(core_nodes)

for _ in range(CONNECTOR_HOPS):
    if full_seed_edges.empty:
        break

    touching_core = full_seed_edges[
        full_seed_edges["geoid_1"].astype(str).isin(expanded_nodes) |
        full_seed_edges["geoid_2"].astype(str).isin(expanded_nodes)
    ].copy()

    neighbor_nodes = set(touching_core["geoid_1"].astype(str).tolist() | se
        touching_core["geoid_2"].astype(str).tolist()
    )

    expanded_nodes = expanded_nodes | neighbor_nodes

CLUSTER_SCOPE_DF = (
    seed_county_gdf.loc[seed_county_gdf["geoid"].astype(str).isin(expanded_r
    .drop_duplicates(subset=["geoid"])]
    .sort_values(["display_state", "display_county", "scope_score"], ascendi
    .reset_index(drop=True)
)

adjacency_gdf = CLUSTER_SCOPE_DF.copy()

adjacency_edges = full_seed_edges.loc[
    full_seed_edges["geoid_1"].astype(str).isin(set(adjacency_gdf["geoid"].a
    full_seed_edges["geoid_2"].astype(str).isin(set(adjacency_gdf["geoid"].a
)].copy()

# -----
# F. Connected components on broadened scope
# -----
all_candidate_nodes = adjacency_gdf["geoid"].astype(str).unique().tolist()
node_to_cluster_num = connected_components_from_edges(all_candidate_nodes, a

adjacency_gdf["cluster_id_num"] = adjacency_gdf["geoid"].astype(str).map(noc
adjacency_gdf["cluster_id"] = (
    adjacency_gdf["display_state"].astype(str)
    + " | "
    + adjacency_gdf["display_county"].astype(str)
    + " | cluster_"

```

```

    + adjacency_gdf["cluster_id_num"].astype(str)
)

tract_cluster_map = (
    adjacency_gdf[
        ["geoid", "cluster_id", "display_state", "display_county", "display_
    ]
    .drop_duplicates(subset=["geoid"])
    .copy()
)

tract_cluster_map["n_cluster_tracts"] = (
    tract_cluster_map.groupby("cluster_id")["geoid"].transform("nunique")
)

# -----
# G. Save outputs
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)

tract_cluster_map.to_parquet(OUT_DIR / "tract_cluster_map.parquet", index=False)
CLUSTER_SCOPE_DF.drop(columns="geometry").to_parquet(OUT_DIR / "cluster_scope.parquet")
adjacency_gdf.drop(columns="geometry").to_parquet(OUT_DIR / "adjacency_cluster_scope.parquet")
seed_counties.to_parquet(OUT_DIR / "seed_counties.parquet", index=False)
county_signal.to_parquet(OUT_DIR / "county_signal.parquet", index=False)

# -----
# H. Diagnostics
# -----
print("Recovery tract universe:", tract_recovery.shape)
print("Signal pool shape:", signal_pool.shape)
print("County signal shape:", county_signal.shape)
print("Seed counties selected:", seed_counties.shape[0])
print("Needed state FIPS:", needed_state_fips)
print()

print("Seed counties by state:")
print(seed_counties["display_state"].value_counts(dropna=False).sort_values(ascending=False))
print()

print("County-expanded core scope:", core_scope_df.shape)
print("Seed-county tract universe:", seed_county_gdf.shape)
print("Connector hops:", CONNECTOR_HOPS)
print("Final CLUSTER_SCOPE_DF shape:", CLUSTER_SCOPE_DF.shape)
print("Connector tracts added:", len(set(CLUSTER_SCOPE_DF["geoid"]) - set(cc_tract_gdf["geoid"])))
print()

print("Adjacency merge shape:", adjacency_gdf.shape)
print("Adjacency edges shape:", adjacency_edges.shape)
print()

print("tract_cluster_map shape:", tract_cluster_map.shape)
print("Unique tracts mapped:", tract_cluster_map["geoid"].nunique())
print("Unique clusters:", tract_cluster_map["cluster_id"].nunique())
print("Unique states in cluster universe:", tract_cluster_map["display_state"].nunique())
print("Unique counties in cluster universe:", tract_cluster_map["display_county"].nunique())

```

```
print()

print("Top 25 seed counties:")
print(
    seed_counties[
        ["display_state", "display_county", "n_signal_tracts", "mean_scope_s
    ].head(25)
)

print()
print("Top 25 clusters by size:")
print(
    tract_cluster_map.groupby(["cluster_id", "display_state", "display_count
    .agg(n_cluster_tracts=("geoid", "nunique"))
    .sort_values(["n_cluster_tracts", "cluster_id"], ascending=[False, True]
    .head(25)
)
```

Recovery tract universe: (84153, 14)
Signal pool shape: (1500, 14)
County signal shape: (190, 9)
Seed counties selected: 40
Needed state FIPS: ['01', '04', '05', '06', '11', '12', '13', '17', '18', '20', '21', '22', '24', '25', '26', '28', '29', '32', '34', '35', '36', '37', '39', '40', '42', '45', '47', '48', '51', '54', '55', '72']

Seed counties by state:

display_state	
Puerto Rico	3
Arizona	3
New Mexico	2
Ohio	2
Michigan	2
New York	2
Georgia	1
Indiana	1
Pennsylvania	1
Illinois	1
Tennessee	1
Wisconsin	1
California	1
Maryland	1
Louisiana	1
Alabama	1
Missouri	1
Kentucky	1
West Virginia	1
Mississippi	1
New Jersey	1
District of Columbia	1
South Carolina	1
Florida	1
North Carolina	1
Oklahoma	1
Arkansas	1
Texas	1
Nevada	1
Virginia	1
Massachusetts	1
Kansas	1

Name: count, dtype: int64

County-expanded core scope: (1227, 14)
Seed-county tract universe: (11039, 28)
Connector hops: 1
Final CLUSTER_SCOPE_DF shape: (2942, 28)
Connector tracts added: 1715

Adjacency merge shape: (2942, 30)
Adjacency edges shape: (7518, 4)

tract_cluster_map shape: (2942, 6)
Unique tracts mapped: 2942
Unique clusters: 86

Unique states in cluster universe: 32
Unique counties in cluster universe: 40

Top 25 seed counties:

display_state	display_county	n_signal_tracts	mean_scope_score	m
ean_igs_total	county_seed_score			
0 Puerto Rico	Utuaado Municipio	8	98.872748	
24.520833	0.892895			
1 Puerto Rico	Mayagüez Municipio	22	97.791283	
28.303030	0.887895			
2 Puerto Rico	Río Grande Municipio	8	97.771098	
19.829167	0.875000			
3 Arizona	Navajo County	7	98.028190	
22.428571	0.868684			
4 Arizona	Apache County	9	96.936950	
23.703704	0.830526			
5 Alabama	Mobile County	14	96.645521	
24.611905	0.822105			
6 Indiana	Lake County	14	96.568595	
26.057143	0.803947			
7 Ohio	Mahoning County	8	96.964295	
28.541667	0.762895			
8 Georgia	Bibb County	7	97.479408	
28.452381	0.744737			
9 New Mexico	McKinley County	9	96.417365	
27.129630	0.744211			
10 Pennsylvania	Allegheny County	11	96.098523	
27.939394	0.730263			
11 Michigan	Wayne County	74	95.904149	
29.763964	0.724211			
12 Illinois	Cook County	58	95.719054	
27.893678	0.716842			
13 Michigan	Genesee County	16	96.172425	
29.562500	0.716053			
14 Tennessee	Shelby County	26	95.753324	
27.750000	0.714079			
15 Ohio	Cuyahoga County	38	95.771998	
30.440351	0.679737			
16 New Mexico	San Juan County	5	96.112597	
23.033333	0.670789			
17 Wisconsin	Milwaukee County	22	95.326373	
28.757576	0.635263			
18 California	Los Angeles County	7	96.143938	
28.828571	0.622368			
19 Maryland	Baltimore city	17	95.596453	
30.107843	0.621053			
20 Louisiana	Orleans Parish	14	95.693053	
29.959524	0.614737			
21 Missouri	St. Louis city	8	95.473045	
27.629167	0.612632			
22 Arizona	Coconino County	2	97.468963	
14.666667	0.598947			
23 New York	Kings County	21	95.887047	
32.341270	0.594737			
24 New York	Monroe County	16	95.684108	
31.525000	0.592895			

Top 25 clusters by size:

display_county	n_cluster_tracts	cluster_id	display_state
37 Michigan Wayne County	152	cluster_38	Michigan
Wayne County			
13 California Los Angeles County	148	cluster_8	California
Los Angeles County			
32 Louisiana Orleans Parish	107	cluster_33	Louisiana
Orleans Parish			
62 Ohio Cuyahoga County	107	cluster_63	Ohio
Cuyahoga County			
33 Maryland Baltimore city	105	cluster_34	Maryland
Baltimore city			
85 Wisconsin Milwaukee County	98	cluster_86	Wisconsin
Milwaukee County			
76 Tennessee Shelby County	96	cluster_77	Tennessee
Shelby County			
42 Nevada Clark County	95	cluster_43	Nevada
Clark County			
23 Illinois Cook County	85	cluster_24	Illinois
Cook County			
0 Alabama Mobile County	81	cluster_1	Alabama
Mobile County			
66 Oklahoma Tulsa County	77	cluster_67	Oklahoma
Tulsa County			
60 New York Monroe County	75	cluster_61	New York
Monroe County			
75 South Carolina Richland County	73	cluster_76	South Carolina
Richland County			
77 Texas Harris County	72	cluster_78	Texas
Harris County			
36 Michigan Genesee County	71	cluster_37	Michigan
Genesee County			
29 Indiana Lake County	69	cluster_30	Indiana
Lake County			
83 Virginia Richmond city	67	cluster_84	Virginia
Richmond city			
15 District of Columbia District of Columbia	65	...	District of Columbia
District of Columbia			
34 Massachusetts Hampden County	64	cluster_35	Massachusetts
Hampden County			
68 Pennsylvania Allegheny County	62	cluster_69	Pennsylvania
Allegheny County			
52 New York Kings County	58	cluster_53	New York
Kings County			
30 Kansas Wyandotte County	54	cluster_31	Kansas
Wyandotte County			
17 Florida Palm Beach County	52	cluster_18	Florida
Palm Beach County			
64 Ohio Mahoning County	52	cluster_65	Ohio
Mahoning County			
46 New Jersey Camden County	50	cluster_47	New Jersey
Camden County			

Cell 3 — join tract panel to clusters and build a cluster-year panel

```
In [49]: # Cell 3 – join tract panel to clusters and build a cluster-year panel

def safe_weighted_mean(values: pd.Series, weights: pd.Series):
    values = pd.to_numeric(values, errors="coerce")
    weights = pd.to_numeric(weights, errors="coerce")

    mask = values.notna() & weights.notna() & (weights > 0)
    if mask.sum() == 0:
        valid = values.notna()
        if valid.sum() == 0:
            return np.nan
        return values.loc[valid].mean()

    return np.average(values.loc[mask], weights=weights.loc[mask])

clustered_panel = (
    analysis_panel
    .merge(
        tract_cluster_map[
            ["geoid", "cluster_id", "display_state", "display_county", "n_cl
        ],
        on="geoid",
        how="inner",
        validate="many_to_one",
        suffixes=("", "_cluster")
    )
    .copy()
)

# Prefer cluster metadata from the tract_cluster_map
clustered_panel["cluster_display_state"] = clustered_panel["display_state_cl
clustered_panel["cluster_display_county"] = clustered_panel["display_county_

# Keep only true cluster aggregation vars here
cluster_core_vars = [
    "igs_total",
    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_l6p",
]

cluster_year_rows = []

for (cluster_id, year, period), grp in clustered_panel.groupby(
    ["cluster_id", "year", "analysis_period"],
    dropna=False
):
    igs_vals = pd.to_numeric(grp["igs_total"], errors="coerce")
    valid_igs = igs_vals.notna()
```

```

row = {
    "cluster_id": cluster_id,
    "year": int(year),
    "analysis_period": period,
    "display_state": grp["cluster_display_state"].dropna().iloc[0] if gr
    "display_county": grp["cluster_display_county"].dropna().iloc[0] if
    "n_cluster_tracts": grp["n_cluster_tracts"].dropna().iloc[0] if grp[
    "n_observed_tracts_year": grp["geoid"].nunique(),
    "cluster_pop_total": pd.to_numeric(grp["pop_total"], errors="coerce"
    "low_igs_tract_share": (
        igs_vals.loc[valid_igs].lt(LOW_IGS_THRESHOLD).mean()
        if valid_igs.any() else np.nan
    ),
}

weights = pd.to_numeric(grp["pop_total"], errors="coerce")

for col in cluster_core_vars:
    row[col] = safe_weighted_mean(grp[col], weights)

cluster_year_rows.append(row)

cluster_year_panel = (
    pd.DataFrame(cluster_year_rows)
    .sort_values(["cluster_id", "year"])
    .reset_index(drop=True)
)

print("clustered_panel shape:", clustered_panel.shape)
print("cluster_year_panel shape:", cluster_year_panel.shape)
print("Unique clusters in panel:", cluster_year_panel["cluster_id"].nunique()
print("Years in cluster panel:", sorted(cluster_year_panel["year"].unique()).
print()
print(cluster_year_panel.head(15))

```

clustered_panel shape: (23536, 19)
cluster_year_panel shape: (688, 15)
Unique clusters in panel: 86
Years in cluster panel: [2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024]

	cluster_id	year	analysis_period	display_state
display_county	n_cluster_tracts	n_observed_tracts_year	cluster_pop_total	
low_igs_tract_share	igs_total	igs_economy \		
0	Alabama Mobile County cluster_1	2017	pre_covid	Alabama
Mobile County	81		81	174074.0
0.777778	38.670840	40.815976		
1	Alabama Mobile County cluster_1	2018	pre_covid	Alabama
Mobile County	81		81	171909.0
0.753086	37.426682	39.276507		
2	Alabama Mobile County cluster_1	2019	pre_covid	Alabama
Mobile County	81		81	171726.0
0.814815	38.016493	40.396335		
3	Alabama Mobile County cluster_1	2020	covid	Alabama
Mobile County	81		81	215309.0
0.800000	40.528150	43.578496		
4	Alabama Mobile County cluster_1	2021	covid	Alabama
Mobile County	81		81	214402.0
0.787500	40.881874	42.742312		
5	Alabama Mobile County cluster_1	2022	recovery	Alabama
Mobile County	81		81	215200.0
0.800000	38.472657	40.568162		
6	Alabama Mobile County cluster_1	2023	recovery	Alabama
Mobile County	81		81	212436.0
0.737500	40.966475	39.426842		
7	Alabama Mobile County cluster_1	2024	recovery	Alabama
Mobile County	81		81	210593.0
0.775000	40.376465	41.524483		
8	Alabama Mobile County cluster_2	2017	pre_covid	Alabama
Mobile County	6		6	21773.0
0.833333	44.995361	44.875378		
9	Alabama Mobile County cluster_2	2018	pre_covid	Alabama
Mobile County	6		6	22780.0
0.833333	44.957028	47.489776		
10	Alabama Mobile County cluster_2	2019	pre_covid	Alabama
Mobile County	6		6	21366.0
0.666667	45.581859	48.723205		
11	Alabama Mobile County cluster_2	2020	covid	Alabama
Mobile County	6		6	21653.0
0.500000	45.797626	49.527225		
12	Alabama Mobile County cluster_2	2021	covid	Alabama
Mobile County	6		6	21498.0
0.500000	48.269281	50.113555		
13	Alabama Mobile County cluster_2	2022	recovery	Alabama
Mobile County	6		6	21350.0
0.666667	42.940422	46.413653		
14	Alabama Mobile County cluster_2	2023	recovery	Alabama
Mobile County	6		6	20875.0
0.333333	46.593054	49.577102		
	poverty_rate	unemp_rate	median_household_income	lfpr_16p
0	0.271643	0.102214	37303.201328	0.522255

1	0.266367	0.088608	38388.685427	0.513352
2	0.247494	0.080236	39726.689657	0.527097
3	0.221165	0.074494	45369.758710	0.534292
4	0.215452	0.072882	47611.121599	0.531995
5	0.220681	0.075142	51866.762718	0.531486
6	0.202819	0.065167	55668.732036	0.536394
7	0.202395	0.071801	56418.745776	0.540548
8	0.191061	0.063831	43784.371331	0.621807
9	0.175418	0.055939	44464.372739	0.593021
10	0.182298	0.056576	45716.814518	0.580650
11	0.160392	0.051975	47756.066411	0.564805
12	0.154294	0.050258	48741.349707	0.567284
13	0.178421	0.051287	53507.745340	0.577691
14	0.180455	0.052773	52358.897820	0.595316

Cell 4 — build period-level summaries and transparent trajectory features

In [50]: *# Cell 4 – build period-level summaries and transparent trajectory features*

```

PERIOD_ORDER = ["pre_covid", "covid", "recovery"]

# -----
# Period-level aggregation
# -----
period_metric_cols = [
    "cluster_pop_total",
    "low_igs_tract_share",
    "igs_total",
    "igs_economy",
    "poverty_rate",
    "unemp_rate",
    "median_household_income",
    "lfpr_l6p",
]

agg_dict = {
    "display_state": "first",
    "display_county": "first",
    "n_cluster_tracts": "max",
    "cluster_pop_total": "mean",
    "low_igs_tract_share": "mean",
    "igs_total": "mean",
    "igs_economy": "mean",
    "poverty_rate": "mean",
    "unemp_rate": "mean",
    "median_household_income": "mean",
    "lfpr_l6p": "mean",
}

# optional diagnostic if Cell 3 includes this column
if "n_observed_tracts_year" in cluster_year_panel.columns:
    agg_dict["n_observed_tracts_year"] = "min"

cluster_period_panel = (

```

```

        cluster_year_panel
        .groupby(["cluster_id", "analysis_period"], as_index=False)
        .agg(agg_dict)
        .copy()
    )

    cluster_period_panel["analysis_period"] = pd.Categorical(
        cluster_period_panel["analysis_period"],
        categories=PERIOD_ORDER,
        ordered=True
    )

    cluster_period_panel = cluster_period_panel.sort_values(
        ["cluster_id", "analysis_period"]
    ).reset_index(drop=True)

    # -----
    # Stable cluster metadata
    # -----
    latest_cluster_meta = (
        cluster_year_panel
        .sort_values(["cluster_id", "year"])
        .groupby("cluster_id", as_index=False)
        .tail(1)[["cluster_id", "display_state", "display_county", "n_cluster_tr
        .rename(columns={"cluster_pop_total": "latest_cluster_pop_total"})
        .copy()
    )

    # -----
    # Wide period summary
    # Only pivot true period-varying metrics
    # -----
    cluster_period_wide = (
        cluster_period_panel
        .pivot(index="cluster_id", columns="analysis_period", values=period_metr
    )

    expected_cols = pd.MultiIndex.from_product(
        [period_metric_cols, PERIOD_ORDER],
        names=["metric", "analysis_period"]
    )
    cluster_period_wide = cluster_period_wide.reindex(columns=expected_cols)

    cluster_period_wide.columns = [
        f"{metric}_{period}" for metric, period in cluster_period_wide.columns
    ]
    cluster_period_wide = cluster_period_wide.reset_index()

    cluster_period_summary = (
        latest_cluster_meta
        .merge(cluster_period_wide, on="cluster_id", how="left", validate="1:1")
        .copy()
    )

    # -----
    # Period completeness diagnostics

```

```

# -----
cluster_period_summary["has_pre_covid"] = cluster_period_summary["igs_total_pre_covid"]
cluster_period_summary["has_covid"] = cluster_period_summary["igs_total_covid"]
cluster_period_summary["has_recovery"] = cluster_period_summary["igs_total_recovery"]

cluster_period_summary["complete_periods_flag"] = (
    cluster_period_summary["has_pre_covid"] &
    cluster_period_summary["has_covid"] &
    cluster_period_summary["has_recovery"]
)

# -----
# Trajectory deltas
# Positive shock metrics = worse COVID hit
# Positive rebound metrics = stronger recovery
# -----

# Lower is worse: igs_total, igs_economy
cluster_period_summary["igs_total_drop_pre_to_covid"] = (
    cluster_period_summary["igs_total_pre_covid"] - cluster_period_summary["igs_total_covid"]
)
cluster_period_summary["igs_economy_drop_pre_to_covid"] = (
    cluster_period_summary["igs_economy_pre_covid"] - cluster_period_summary["igs_economy_covid"]
)
cluster_period_summary["igs_total_rebound_covid_to_recovery"] = (
    cluster_period_summary["igs_total_recovery"] - cluster_period_summary["igs_total_covid"]
)
cluster_period_summary["igs_economy_rebound_covid_to_recovery"] = (
    cluster_period_summary["igs_economy_recovery"] - cluster_period_summary["igs_economy_covid"]
)

# Higher is worse: poverty_rate, unemp_rate
cluster_period_summary["poverty_increase_pre_to_covid"] = (
    cluster_period_summary["poverty_rate_covid"] - cluster_period_summary["poverty_rate_pre_covid"]
)
cluster_period_summary["unemp_increase_pre_to_covid"] = (
    cluster_period_summary["unemp_rate_covid"] - cluster_period_summary["unemp_rate_pre_covid"]
)
cluster_period_summary["poverty_improve_covid_to_recovery"] = (
    cluster_period_summary["poverty_rate_covid"] - cluster_period_summary["poverty_rate_recovery"]
)
cluster_period_summary["unemp_improve_covid_to_recovery"] = (
    cluster_period_summary["unemp_rate_covid"] - cluster_period_summary["unemp_rate_recovery"]
)

# Higher is better: income, labor-force participation
cluster_period_summary["income_drop_pre_to_covid"] = (
    cluster_period_summary["median_household_income_pre_covid"] - cluster_period_summary["median_household_income_covid"]
)
cluster_period_summary["lfpr_drop_pre_to_covid"] = (
    cluster_period_summary["lfpr_16p_pre_covid"] - cluster_period_summary["lfpr_16p_covid"]
)
cluster_period_summary["income_rebound_covid_to_recovery"] = (
    cluster_period_summary["median_household_income_recovery"] - cluster_period_summary["median_household_income_covid"]
)
cluster_period_summary["lfpr_rebound_covid_to_recovery"] = (
    cluster_period_summary["lfpr_16p_recovery"] - cluster_period_summary["lfpr_16p_covid"]
)

```



```

        cluster_period_summary["lfr_16p_recovery"] - cluster_period_summary["lfr_16p_recovery"]
    )

# -----
# Structural filters
# -----
cluster_period_summary["serious_cluster_flag"] = (
    cluster_period_summary["complete_periods_flag"] &
    (cluster_period_summary["n_cluster_tracts"] >= MIN_CLUSTER_TRACTS) &
    (cluster_period_summary["latest_cluster_pop_total"] >= MIN_CLUSTER_POP)
)

cluster_period_summary["low_igs_recovery_flag"] = (
    cluster_period_summary["igs_total_recovery"] < LOW_IGS_THRESHOLD
)

cluster_period_summary["low_igs_economy_recovery_flag"] = (
    cluster_period_summary["igs_economy_recovery"] < LOW_IGS_THRESHOLD
)

# -----
# Diagnostics
# -----
print("cluster_period_panel shape:", cluster_period_panel.shape)
print("cluster_period_summary shape:", cluster_period_summary.shape)
print()
print("Complete-period cluster count:", int(cluster_period_summary["complete_periods_flag"].sum()))
print("Serious cluster count:", int(cluster_period_summary["serious_cluster_flag"].sum()))
print("Recovery low-IGS cluster count:", int(cluster_period_summary["low_igs_recovery_flag"].sum()))
print()
print(
    cluster_period_summary[
        [
            "cluster_id",
            "display_state",
            "display_county",
            "n_cluster_tracts",
            "latest_cluster_pop_total",
            "igs_total_pre_covid",
            "igs_total_covid",
            "igs_total_recovery",
            "poverty_rate_pre_covid",
            "poverty_rate_covid",
            "poverty_rate_recovery",
            "complete_periods_flag",
            "serious_cluster_flag",
            "low_igs_recovery_flag",
        ]
    ].head(15)
)

```


	poverty_rate_pre_covid	poverty_rate_covid	poverty_rate_recovery	complete_periods_flag	serious_cluster_flag	low_igs_recovery_flag
0	0.261835	0.218308	0.208632	True	True	True
1	0.182926	0.157343	0.176671	True	True	True
2	0.367412	0.341306	0.313574	True	True	True
3	0.280143	0.214152	0.206512	True	True	False
4	0.320524	0.252731	0.247988	True	True	True
5	0.286492	0.287834	0.246815	True	True	True
6	0.301040	0.254905	0.225949	True	True	True
7	0.261123	0.273549	0.257480	True	True	True
8	0.318769	0.260076	0.236495	True	True	True
9	0.280843	0.213290	0.187341	True	True	True
10	0.261532	0.217998	0.209009	True	True	True
11	0.141211	0.160974	0.183929	True	True	True
12	0.721659	0.644663	0.613950	True	True	True
13	0.308654	0.267020	0.251133	True	True	True
14	0.203908	0.147500	0.150510	True	True	True

Cell 5 — practical shortlist logic for the new method

```
In [51]: # Cell 5 – practical shortlist logic for the new method

def bad_when_high(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return s.rank(pct=True, method="average")

def bad_when_low(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return (-s).rank(pct=True, method="average")

cluster_scored = cluster_period_summary.copy()

# -----
# Scoring universe
# Score only clusters that are structurally usable
# Then apply final low-IGS shortlist filter afterward
# -----
cluster_scored["eligible_for_scoring"] = (
    cluster_scored["complete_periods_flag"].fillna(False) &
    cluster_scored["serious_cluster_flag"].fillna(False)
```

```

)

score_mask = cluster_scored["eligible_for_scoring"]
score_df = cluster_scored.loc[score_mask].copy()

# -----
# Shock variables:
# Positive values = adverse COVID shock
# Negative values are clipped to 0 so they do not behave like "anti-shock"
# -----
shock_cols = [
    "igs_total_drop_pre_to_covid",
    "igs_economy_drop_pre_to_covid",
    "poverty_increase_pre_to_covid",
    "unemp_increase_pre_to_covid",
    "income_drop_pre_to_covid",
    "lfpr_drop_pre_to_covid",
]

for col in shock_cols:
    score_df[f"{col}_shock_only"] = pd.to_numeric(score_df[col], errors="coerce")

# -----
# Weak-recovery shortfall variables:
# Higher = weaker recovery
# -----
score_df["weakrec_igs_total"] = -pd.to_numeric(score_df["igs_total_rebound_covid_to_recovery"])
score_df["weakrec_igs_economy"] = -pd.to_numeric(score_df["igs_economy_rebound_covid_to_recovery"])
score_df["weakrec_poverty"] = -pd.to_numeric(score_df["poverty_improve_covid_to_recovery"])
score_df["weakrec_unemp"] = -pd.to_numeric(score_df["unemp_improve_covid_to_recovery"])
score_df["weakrec_income"] = -pd.to_numeric(score_df["income_rebound_covid_to_recovery"])
score_df["weakrec_lfpr"] = -pd.to_numeric(score_df["lfpr_rebound_covid_to_recovery"])

# -----
# 1) Still low on IGS in recovery
# Use overall IGS + share of tracts below the IGS threshold
# This avoids double-counting igs_economy here
# -----
score_df["score_low_igs_recovery"] = (
    bad_when_low(score_df["igs_total_recovery"]) +
    bad_when_high(score_df["low_igs_tract_share_recovery"])
) / 2

# -----
# 2) Economically vulnerable in recovery
# Keep igs_economy only here, as the economy anchor + ACS support
# -----
score_df["score_econ_vulnerability_recovery"] = (
    bad_when_low(score_df["igs_economy_recovery"]) +
    bad_when_high(score_df["poverty_rate_recovery"]) +
    bad_when_high(score_df["unemp_rate_recovery"]) +
    bad_when_low(score_df["median_household_income_recovery"]) +
    bad_when_low(score_df["lfpr_16p_recovery"])
) / 5

# -----

```

```

# 3) Harder COVID shock relative to pre-COVID baseline
# -----
score_df["score_covid_shock"] = (
    bad_when_high(score_df["igs_total_drop_pre_to_covid_shock_only"]) +
    bad_when_high(score_df["igs_economy_drop_pre_to_covid_shock_only"]) +
    bad_when_high(score_df["poverty_increase_pre_to_covid_shock_only"]) +
    bad_when_high(score_df["unemp_increase_pre_to_covid_shock_only"]) +
    bad_when_high(score_df["income_drop_pre_to_covid_shock_only"]) +
    bad_when_high(score_df["lfpr_drop_pre_to_covid_shock_only"])
) / 6

# -----
# 4) Weak recovery since COVID
# -----
score_df["score_weak_recovery"] = (
    bad_when_high(score_df["weakrec_igs_total"]) +
    bad_when_high(score_df["weakrec_igs_economy"]) +
    bad_when_high(score_df["weakrec_poverty"]) +
    bad_when_high(score_df["weakrec_unemp"]) +
    bad_when_high(score_df["weakrec_income"]) +
    bad_when_high(score_df["weakrec_lfpr"])
) / 6

# -----
# Transparent final score
# Recovery status + recovery vulnerability remain the foundation
# -----
score_df["cluster_shortlist_score"] = (
    0.35 * score_df["score_low_igs_recovery"] +
    0.35 * score_df["score_econ_vulnerability_recovery"] +
    0.15 * score_df["score_covid_shock"] +
    0.15 * score_df["score_weak_recovery"]
)

# Write scores back into the full cluster frame
score_cols = [
    "score_low_igs_recovery",
    "score_econ_vulnerability_recovery",
    "score_covid_shock",
    "score_weak_recovery",
    "cluster_shortlist_score",
]
cluster_scored = cluster_scored.merge(
    score_df[["cluster_id"] + score_cols],
    on="cluster_id",
    how="left",
    validate="1:1",
)

# -----
# Final practical screen for enrichment candidates
# -----
cluster_shortlist = (
    cluster_scored
    .loc[
        cluster_scored["eligible_for_scoring"] &

```

```

        cluster_scored["low_igs_recovery_flag"]
    ]
    .sort_values(
        [
            "cluster_shortlist_score",
            "igs_total_recovery",
            "igs_economy_recovery",
            "latest_cluster_pop_total",
        ],
        ascending=[False, True, True, False]
    )
    .reset_index(drop=True)
)

cluster_shortlist["rank"] = np.arange(1, len(cluster_shortlist) + 1)

shortlist_view_cols = [
    "rank",
    "cluster_id",
    "display_state",
    "display_county",
    "n_cluster_tracts",
    "latest_cluster_pop_total",
    "igs_total_pre_covid",
    "igs_total_covid",
    "igs_total_recovery",
    "low_igs_tract_share_recovery",
    "igs_economy_recovery",
    "poverty_rate_recovery",
    "unemp_rate_recovery",
    "median_household_income_recovery",
    "lfpr_16p_recovery",
    "score_low_igs_recovery",
    "score_econ_vulnerability_recovery",
    "score_covid_shock",
    "score_weak_recovery",
    "cluster_shortlist_score",
]

print("Scoring universe shape:", score_df.shape)
print("Final shortlist shape:", cluster_shortlist.shape)
print()
print(cluster_shortlist[shortlist_view_cols].head(20))

# Optional: persist key outputs for later healthcare + business enrichment
OUT_DIR.mkdir(parents=True, exist_ok=True)
cluster_year_panel.to_parquet(OUT_DIR / "cluster_year_panel.parquet", index=False)
cluster_period_summary.to_parquet(OUT_DIR / "cluster_period_summary.parquet", index=False)
cluster_shortlist.to_parquet(OUT_DIR / "cluster_shortlist.parquet", index=False)

print()
print("Saved:")
print("-", OUT_DIR / "cluster_year_panel.parquet")
print("-", OUT_DIR / "cluster_period_summary.parquet")
print("-", OUT_DIR / "cluster_shortlist.parquet")

```

Scoring universe shape: (86, 66)

Final shortlist shape: (67, 55)

rank	display_county	n_cluster_tracts	latest_cluster_pop_total	cluster_id	display_state	igs_total_pre_covid	igs_total_covid	igs_total_recovery
0	1	Puerto Rico Utuado Municipio	8	cluster_75	Puerto Rico	27621.0	27.5	25.035923
1	2	Puerto Rico Mayagüez Municipio	25	cluster_73	Puerto Rico	70828.0	3	31.136263
2	3	Puerto Rico Río Grande Municipio	12	cluster_74	Puerto Rico	46051.0	2	25.284481
3	4	Ohio Cuyahoga County	13	cluster_64	Ohio	31886.0	33.47	34.225025
4	5	Michigan Wayne County	152	cluster_38	Michigan	319984.0	29.91185	34.793412
5	6	Illinois Cook County	9	cluster_29	Illinois	27412.0	32.908060	30.615724
6	7	Arizona Apache County	18	cluster_3	Arizona	65341.0	31.6201	34.156029
7	8	Illinois Cook County	33	cluster_25	Illinois	81348.0	29.989644	34.120468
8	9	Michigan Wayne County	9	cluster_39	Michigan	19947.0	27.93307	29.825030
9	10	Illinois Cook County	85	cluster_24	Illinois	187702.0	30.537359	34.342235
10	11	Tennessee Shelby County	96	cluster_77	Tennessee	268298.0	35.9003	35.813056
11	12	New Mexico McKinley County	33	cluster_49	New Mexico	70431.0	41.88	36.970752
12	13	California Los Angeles County	5	cluster_11	California	19914.0	3	35.686555
13	14	Ohio Cuyahoga County	107	cluster_63	Ohio	210162.0	35.09	38.303060
14	15	Wisconsin Milwaukee County	98	cluster_86	Wisconsin	225339.0	34.1	35.715526
15	16	Louisiana Orleans Parish	107	cluster_33	Louisiana	218876.0	39.817	37.108224
16	17	New York Kings County	15	cluster_51	New York	55602.0	39.08708	

1	39.677277	39.203114	
17	18	New York Kings County cluster_53	New York
Kings County	58	213577.0	34.98004
1	36.149592	37.669346	
18	19	North Carolina Robeson County cluster_62	North Carolina
Robeson County	35	116902.0	36.956
487	38.253102	35.994049	
19	20	California Los Angeles County cluster_8	California
Los Angeles County	148	569438.0	3
4.284506	35.928396	36.441923	

	low_igs_tract_share_recovery	igs_economy_recovery	poverty_rate_recovery
y unemp_rate_recovery	median_household_income_recovery	lfpr_16p_recovery	
score_low_igs_recovery \			
0	1.000000	15.289723	0.52533
2	0.264035	18417.685903	0.408071
0.985465			
1	1.000000	19.475426	0.53221
0	0.229062	19229.673889	0.415725
0.962209			
2	1.000000	17.443243	0.35665
4	0.165771	28512.143909	0.525166
0.979651			
3	1.000000	31.398833	0.37025
5	0.136437	36960.197388	0.590425
0.944767			
4	0.951435	29.645510	0.34875
3	0.161412	37886.810012	0.516598
0.906977			
5	1.000000	34.163963	0.23487
4	0.166658	51537.524910	0.543750
0.968023			
6	0.851852	28.709013	0.31357
4	0.099555	41738.341598	0.407386
0.831395			
7	0.959596	31.288560	0.30416
7	0.149948	43383.006594	0.586529
0.936047			
8	1.000000	27.539859	0.29727
0	0.092034	41703.874646	0.550155
0.973837			
9	0.900794	29.945366	0.30052
4	0.165977	46121.632254	0.577286
0.883721			
10	0.896057	35.120633	0.30034
8	0.118143	43461.126364	0.582458
0.848837			
11	0.757576	37.724026	0.34116
0	0.098756	45567.496445	0.484736
0.691860			
12	0.933333	35.223650	0.25748
0	0.098351	56430.097238	0.617167
0.895349			
13	0.852201	31.909575	0.32958
2	0.141196	38280.580628	0.551952
0.726744			

14		0.880952	35.118704	0.32114
5	0.091170		40889.939837	0.584413
0.837209				
15		0.841270	35.605207	0.27208
0	0.104481		49657.707522	0.582639
0.750000				
16		0.755556	32.280068	0.29726
4	0.144929		49551.621552	0.480014
0.604651				
17		0.870283	32.004432	0.28324
5	0.124499		52456.969477	0.559224
0.761628				
18		0.904762	43.200898	0.28025
8	0.062075		40796.621709	0.509912
0.863372				
19		0.918919	35.197972	0.25113
3	0.099923		54797.541906	0.611680
0.866279				

	score_econ_vulnerability_recovery	score_covid_shock	score_weak_recover
y cluster_shortlist_score			
0	0.990698	0.645349	0.54263
6	0.869855		
1	0.981395	0.515504	0.60658
9	0.848576		
2	0.939535	0.466085	0.53294
6	0.821570		
3	0.837209	0.438953	0.81007
8	0.811047		
4	0.923256	0.444767	0.43604
7	0.772703		
5	0.734884	0.543605	0.49806
2	0.752267		
6	0.879070	0.448643	0.50775
2	0.742122		
7	0.809302	0.450581	0.37403
1	0.734564		
8	0.795349	0.373062	0.38759
7	0.733314		
9	0.825581	0.373062	0.38759
7	0.712355		
10	0.765116	0.440891	0.54069
8	0.712122		
11	0.779070	0.595930	0.67829
5	0.705959		
12	0.558140	0.577519	0.73449
6	0.705523		
13	0.867442	0.452519	0.48255
8	0.698227		
14	0.730233	0.479651	0.43023
3	0.685087		
15	0.681395	0.679264	0.53100
8	0.682529		
16	0.827907	0.471899	0.69961
2	0.677122		
17	0.755814	0.508721	0.35271

3	0.660320			
18		0.602326	0.373062	0.58720
9	0.657035			
19		0.583721	0.442829	0.45155
0	0.641657			

Saved:

- data\processed\cluster_year_panel.parquet
- data\processed\cluster_period_summary.parquet
- data\processed\cluster_shortlist.parquet

Cell 6 — prepare enrichment-ready outputs for healthcare + business data

```
In [52]: # Cell 6 – prepare enrichment-ready outputs from the main shortlist
# IMPORTANT:
# This cell stays source-agnostic.
# It packages the shortlist for later healthcare + business enrichment only.

ENRICHMENT_TOP_K = min(15, len(cluster_shortlist))

# -----
# A. pick the top clusters to enrich next
# -----
enrichment_clusters = (
    cluster_shortlist
    .head(ENRICHMENT_TOP_K)
    .copy()
    .reset_index(drop=True)
)

enrichment_clusters["enrichment_rank"] = np.arange(1, len(enrichment_clusters) + 1)
enrichment_cluster_ids = enrichment_clusters["cluster_id"].astype(str).tolist()

print(f"Selected top {len(enrichment_clusters)} clusters for next-step enrichment")
print(
    enrichment_clusters[
        [
            "enrichment_rank",
            "rank",
            "cluster_id",
            "display_state",
            "display_county",
            "n_cluster_tracts",
            "latest_cluster_pop_total",
            "igs_total_recovery",
            "igs_economy_recovery",
            "poverty_rate_recovery",
            "unemp_rate_recovery",
            "median_household_income_recovery",
            "lfpr_16p_recovery",
            "cluster_shortlist_score",
        ]
    ].head(15)
)
```

```

# useful lookup for downstream merges
enrichment_rank_lookup = enrichment_clusters[
    ["cluster_id", "enrichment_rank", "rank", "cluster_shortlist_score"]
].drop_duplicates().copy()

# -----
# B. tract-level lookup table for external joins
# -----
enrichment_cluster_tracts = (
    tract_cluster_map
    .loc[tract_cluster_map["cluster_id"].astype(str).isin(enrichment_cluster
    .copy()
)

enrichment_cluster_tracts["geoid"] = normalize_geoid_series(enrichment_clust
enrichment_cluster_tracts = enrichment_cluster_tracts.dropna(subset=["geoid"]

enrichment_cluster_tracts["state_fips"] = enrichment_cluster_tracts["geoid"]
enrichment_cluster_tracts["county_fips"] = enrichment_cluster_tracts["geoid"]
enrichment_cluster_tracts["tract_code"] = enrichment_cluster_tracts["geoid"]

enrichment_cluster_tracts = (
    enrichment_cluster_tracts
    .merge(enrichment_rank_lookup, on="cluster_id", how="left", validate="ma
    .sort_values(["enrichment_rank", "cluster_id", "geoid"])
    .drop_duplicates(subset=["cluster_id", "geoid"])
    .reset_index(drop=True)
)

# -----
# C. county-level lookup table for external county joins
# -----
enrichment_cluster_counties = (
    enrichment_cluster_tracts
    .groupby(
        ["cluster_id", "enrichment_rank", "display_state", "display_county",
        as_index=False
    )
    .agg(
        n_cluster_tracts_in_county=("geoid", "nunique"),
        tract_list=("geoid", lambda s: sorted(set(s.astype(str))))
    )
    .sort_values(["enrichment_rank", "cluster_id", "county_fips"])
    .reset_index(drop=True)
)

cluster_county_counts = (
    enrichment_cluster_counties
    .groupby("cluster_id", as_index=False)
    .agg(n_cluster_counties=("county_fips", "nunique"))
)

# -----
# D. keep the full year-by-year history for shortlisted clusters
# -----

```

```

enrichment_cluster_year_panel = (
    cluster_year_panel
    .loc[cluster_year_panel["cluster_id"].astype(str).isin(enrichment_clusters["cluster_id"].astype(str))]
    .copy()
    .merge(enrichment_rank_lookup, on="cluster_id", how="left", validate="many")
    .sort_values(["enrichment_rank", "cluster_id", "year"])
    .reset_index(drop=True)
)

# -----
# E. tract-year panel for later tract/county joins
# -----
enrichment_tract_year_panel = (
    clustered_panel
    .loc[clustered_panel["cluster_id"].astype(str).isin(enrichment_clusters["cluster_id"].astype(str))]
    .copy()
)

enrichment_tract_year_panel["geoid"] = normalize_geoid_series(enrichment_tract_year_panel["geoid"])
enrichment_tract_year_panel = enrichment_tract_year_panel.dropna(subset=["geoid"])

enrichment_tract_year_panel["state_fips"] = enrichment_tract_year_panel["geoid"].str[0:2]
enrichment_tract_year_panel["county_fips"] = enrichment_tract_year_panel["geoid"].str[2:5]
enrichment_tract_year_panel["tract_code"] = enrichment_tract_year_panel["geoid"].str[5:]

enrichment_tract_year_panel = (
    enrichment_tract_year_panel
    .merge(enrichment_rank_lookup, on="cluster_id", how="left", validate="many")
    .sort_values(["enrichment_rank", "cluster_id", "geoid", "year"])
    .drop_duplicates(subset=["cluster_id", "geoid", "year"])
    .reset_index(drop=True)
)

# -----
# F. profile table
# -----
enrichment_cluster_profiles = (
    enrichment_clusters[
        [
            "enrichment_rank",
            "rank",
            "cluster_id",
            "display_state",
            "display_county",
            "n_cluster_tracts",
            "latest_cluster_pop_total",
            "igs_total_pre_covid",
            "igs_total_covid",
            "igs_total_recovery",
            "low_igs_tract_share_recovery",
            "igs_economy_recovery",
            "poverty_rate_recovery",
            "unemp_rate_recovery",
            "median_household_income_recovery",
            "lfpr_16p_recovery",
            "score_low_igs_recovery",
        ]
    ]
)

```

```

        "score_econ_vulnerability_recovery",
        "score_covid_shock",
        "score_weak_recovery",
        "cluster_shortlist_score",
    ]
]
.merge(cluster_county_counts, on="cluster_id", how="left", validate="1:1")
.copy()
)

# -----
# G. validation checks
# -----
expected_cluster_ids = set(enrichment_cluster_ids)
tract_cluster_ids = set(enrichment_cluster_tracts["cluster_id"].astype(str))
county_cluster_ids = set(enrichment_cluster_counties["cluster_id"].astype(str))
year_cluster_ids = set(enrichment_cluster_year_panel["cluster_id"].astype(str))
tract_year_cluster_ids = set(enrichment_tract_year_panel["cluster_id"].astype(str))
profile_cluster_ids = set(enrichment_cluster_profiles["cluster_id"].astype(str))

assert tract_cluster_ids == expected_cluster_ids, "Mismatch between enrichment cluster tracts and expected cluster IDs"
assert county_cluster_ids == expected_cluster_ids, "Mismatch between enrichment cluster counties and expected cluster IDs"
assert year_cluster_ids == expected_cluster_ids, "Mismatch between enrichment cluster year panel and expected cluster IDs"
assert tract_year_cluster_ids == expected_cluster_ids, "Mismatch between enrichment cluster year panel and expected cluster IDs"
assert profile_cluster_ids == expected_cluster_ids, "Mismatch between enrichment cluster profiles and expected cluster IDs"

expected_cluster_year_rows = len(expected_cluster_ids) * len(sorted(cluster_year_panel["year"].dropna().unique()))
if len(enrichment_cluster_year_panel) != expected_cluster_year_rows:
    print(
        f"Warning: enrichment_cluster_year_panel has {len(enrichment_cluster_year_panel)} rows, "
        f"expected about {expected_cluster_year_rows} if all selected clusters have data for all years"
    )

# optional tract-year completeness check
tract_counts = enrichment_cluster_tracts.groupby("cluster_id")["geoid"].nunique()
tract_year_counts = enrichment_tract_year_panel.groupby("cluster_id").size()
expected_years = len(sorted(enrichment_tract_year_panel["year"].dropna().unique()))
tract_year_check = pd.DataFrame({
    "n_cluster_tracts_lookup": tract_counts,
    "tract_year_rows": tract_year_counts
}).reset_index()
tract_year_check["expected_rows_if_full"] = tract_year_check["n_cluster_tracts_lookup"] * expected_years
tract_year_check["full_8yr_panel_flag"] = (
    tract_year_check["tract_year_rows"] == tract_year_check["expected_rows_if_full"]
)

# -----
# H. save outputs
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)

enrichment_clusters.to_parquet(OUT_DIR / "enrichment_clusters.parquet", index=False)
enrichment_cluster_profiles.to_parquet(OUT_DIR / "enrichment_cluster_profiles.parquet", index=False)
enrichment_cluster_tracts.to_parquet(OUT_DIR / "enrichment_cluster_tracts.parquet", index=False)
enrichment_cluster_counties.to_parquet(OUT_DIR / "enrichment_cluster_counties.parquet", index=False)
enrichment_cluster_year_panel.to_parquet(OUT_DIR / "enrichment_cluster_year_panel.parquet", index=False)

```

```

enrichment_tract_year_panel.to_parquet(OUT_DIR / "enrichment_tract_year_panel.parquet")
tract_year_check.to_parquet(OUT_DIR / "enrichment_tract_year_completeness.parquet")

enrichment_cluster_profiles.to_csv(OUT_DIR / "enrichment_cluster_profiles.csv")
enrichment_cluster_tracts.to_csv(OUT_DIR / "enrichment_cluster_tracts.csv",
enrichment_cluster_counties.to_csv(OUT_DIR / "enrichment_cluster_counties.csv",
tract_year_check.to_csv(OUT_DIR / "enrichment_tract_year_completeness.csv",

print("\nSaved enrichment package.")
print("enrichment_clusters shape:", enrichment_clusters.shape)
print("enrichment_cluster_profiles shape:", enrichment_cluster_profiles.shape)
print("enrichment_cluster_tracts shape:", enrichment_cluster_tracts.shape)
print("enrichment_cluster_counties shape:", enrichment_cluster_counties.shape)
print("enrichment_cluster_year_panel shape:", enrichment_cluster_year_panel.shape)
print("enrichment_tract_year_panel shape:", enrichment_tract_year_panel.shape)
print("tract_year_check shape:", tract_year_check.shape)
print()
print("Tract-year completeness check:")
print(
    tract_year_check.sort_values(["full_8yr_panel_flag", "cluster_id"], asce
)

```

Selected top 15 clusters for next-step enrichment.

enrichment_rank	rank	cluster_id	d
display_state	display_county	n_cluster_tracts	latest_cluster_pop_tot
al	igs_total_recovery	igs_economy_recovery	\
0	1	1	Puerto Rico Utuado Municipio cluster_75
Puerto Rico	Utuado Municipio	8	27621.
0	25.035923	15.289723	
1	2	2	Puerto Rico Mayagüez Municipio cluster_73
Puerto Rico	Mayagüez Municipio	25	70828.
0	31.136263	19.475426	
2	3	3	Puerto Rico Río Grande Municipio cluster_74
Puerto Rico	Río Grande Municipio	12	46051.
0	25.284481	17.443243	
3	4	4	Ohio Cuyahoga County cluster_64
Ohio	Cuyahoga County	13	31886.0
34.225025	31.398833		
4	5	5	Michigan Wayne County cluster_38
Michigan	Wayne County	152	319984.0
34.793412	29.645510		
5	6	6	Illinois Cook County cluster_29
Illinois	Cook County	9	27412.0
30.615724	34.163963		
6	7	7	Arizona Apache County cluster_3
Arizona	Apache County	18	65341.0
34.156029	28.709013		
7	8	8	Illinois Cook County cluster_25
Illinois	Cook County	33	81348.0
34.120468	31.288560		
8	9	9	Michigan Wayne County cluster_39
Michigan	Wayne County	9	19947.0
29.825030	27.539859		
9	10	10	Illinois Cook County cluster_24
Illinois	Cook County	85	187702.0
34.342235	29.945366		
10	11	11	Tennessee Shelby County cluster_77
Tennessee	Shelby County	96	268298.0
35.813056	35.120633		
11	12	12	New Mexico McKinley County cluster_49
New Mexico	McKinley County	33	70431.0
36.970752	37.724026		
12	13	13	California Los Angeles County cluster_11
California	Los Angeles County	5	19914.0
35.686555	35.223650		
13	14	14	Ohio Cuyahoga County cluster_63
Ohio	Cuyahoga County	107	210162.0
38.303060	31.909575		
14	15	15	Wisconsin Milwaukee County cluster_86
Wisconsin	Milwaukee County	98	225339.0
35.715526	35.118704		

poverty_rate_recovery	unemp_rate_recovery	median_household_income_recovery
lfpr_16p_recovery	cluster_shortlist_score	
0	0.525332	0.264035
5903	0.408071	0.869855
1	0.532210	0.229062
3889	0.415725	0.848576

2	0.356654	0.165771	28512.14
3909	0.525166	0.821570	
3	0.370255	0.136437	36960.19
7388	0.590425	0.811047	
4	0.348753	0.161412	37886.81
0012	0.516598	0.772703	
5	0.234874	0.166658	51537.52
4910	0.543750	0.752267	
6	0.313574	0.099555	41738.34
1598	0.407386	0.742122	
7	0.304167	0.149948	43383.00
6594	0.586529	0.734564	
8	0.297270	0.092034	41703.87
4646	0.550155	0.733314	
9	0.300524	0.165977	46121.63
2254	0.577286	0.712355	
10	0.300348	0.118143	43461.12
6364	0.582458	0.712122	
11	0.341160	0.098756	45567.49
6445	0.484736	0.705959	
12	0.257480	0.098351	56430.09
7238	0.617167	0.705523	
13	0.329582	0.141196	38280.58
0628	0.551952	0.698227	
14	0.321145	0.091170	40889.93
9837	0.584413	0.685087	

Saved enrichment package.

enrichment_clusters shape: (15, 56)

enrichment_cluster_profiles shape: (15, 22)

enrichment_cluster_tracts shape: (703, 12)

enrichment_cluster_counties shape: (15, 7)

enrichment_cluster_year_panel shape: (120, 18)

enrichment_tract_year_panel shape: (5624, 25)

tract_year_check shape: (15, 5)

Tract-year completeness check:

tract_year_rows	expected_rows_if_full	cluster_id	n_cluster_tracts_lookup
0	Arizona Apache County	cluster_3	18
144	144	True	
1	California Los Angeles County	cluster_11	5
40	40	True	
2	Illinois Cook County	cluster_24	85
680	680	True	
3	Illinois Cook County	cluster_25	33
264	264	True	
4	Illinois Cook County	cluster_29	9
72	72	True	
5	Michigan Wayne County	cluster_38	152
1216	1216	True	
6	Michigan Wayne County	cluster_39	9
72	72	True	
7	New Mexico McKinley County	cluster_49	33
264	264	True	
8	Ohio Cuyahoga County	cluster_63	107

856		856	True	
9	Ohio	Cuyahoga County	cluster_64	13
104		104	True	
10	Puerto Rico	Mayagüez Municipio	cluster_73	25
200		200	True	
11	Puerto Rico	Río Grande Municipio	cluster_74	12
96		96	True	
12	Puerto Rico	Utuado Municipio	cluster_75	8
64		64	True	
13	Tennessee	Shelby County	cluster_77	96
768		768	True	
14	Wisconsin	Milwaukee County	cluster_86	98
784		784	True	

Cell 7 — load County Health Rankings 2023 healthcare data from analytic_data2023_0.csv

```
In [53]: # Cell 7 – load and join multi-year County Health Rankings healthcare data (

from pathlib import Path
import pandas as pd
import numpy as np

HEALTHCARE_DIR = RAW_DIR / "healthcare"
HEALTHCARE_DIR.mkdir(parents=True, exist_ok=True)

# Only load actual County Health Rankings analytic files
healthcare_files = sorted(HEALTHCARE_DIR.glob("analytic_data*.csv"))

if len(healthcare_files) == 0:
    raise FileNotFoundError(
        f"No County Health Rankings analytic_data*.csv files found in {HEALTHCARE_DIR}"
    )

print("Healthcare files found:")
for p in healthcare_files:
    print("-", p.name)

# -----
# A. Helpers
# -----
def normalize_county_fips(series):
    s = series.astype("string").str.strip()
    s = s.str.replace(r"\.0$", "", regex=True)
    s = s.str.replace(r"\D", "", regex=True)
    s = s.str.zfill(5)
    return s.where(s.str.fullmatch(r"\d{5}"), pd.NA)

def find_first_matching_column(df, candidates):
    for c in candidates:
        if c in df.columns:
            return c
    return None

def find_first_startswith_column(df, prefixes):
```

```

lowered = {str(c).strip().lower(): c for c in df.columns}
for prefix in prefixes:
    prefix_l = str(prefix).strip().lower()
    for col_l, original_col in lowered.items():
        if col_l.startswith(prefix_l):
            return original_col
return None

def standardize_chr_file(chr_raw, source_name):
    chr_df = chr_raw.copy()

    fips_col = find_first_matching_column(chr_df, [
        "5-digit FIPS Code",
        "5 digit FIPS Code",
        "5-digit FIPS code",
        "FIPS Code",
    ])
    if fips_col is None:
        raise ValueError(f"{source_name}: could not find county FIPS column")

    year_col = find_first_matching_column(chr_df, [
        "Release Year",
        "release year",
        "Year",
    ])

    state_col = find_first_matching_column(chr_df, [
        "State Abbreviation",
        "State",
    ])

    name_col = find_first_matching_column(chr_df, [
        "Name",
        "County",
    ])

    chr_df["county_fips"] = normalize_county_fips(chr_df[fips_col])

    if year_col is not None:
        chr_df["year"] = pd.to_numeric(chr_df[year_col], errors="coerce").as
    else:
        chr_df["year"] = pd.NA

    chr_df["chr_state"] = chr_df[state_col].astype("string") if state_col is
    chr_df["chr_county"] = chr_df[name_col].astype("string") if name_col is

    health_measure_candidates = {
        "uninsured_rate": ["Uninsured raw value"],
        "uninsured_adults_rate": ["Uninsured Adults raw value"],
        "uninsured_children_rate": ["Uninsured Children raw value"],
        "primary_care_physicians_rate": ["Primary Care Physicians raw value"],
        "dentists_rate": ["Dentists raw value"],
        "mental_health_providers_rate": ["Mental Health Providers raw value"],
        "other_primary_care_providers_rate": ["Other Primary Care Providers"],
        "preventable_hospital_stays_rate": ["Preventable Hospital Stays raw"],
        "mammography_screening_rate": ["Mammography Screening raw value"],
    }

```

```

        "flu_vaccinations_rate": ["Flu Vaccinations raw value"],
    }

    resolved_measure_cols = {}

    for clean_name, candidates in health_measure_candidates.items():
        exact_col = find_first_matching_column(chr_df, candidates)
        if exact_col is not None:
            resolved_measure_cols[exact_col] = clean_name
            continue

        prefix_col = find_first_startswith_column(chr_df, candidates)
        if prefix_col is not None:
            resolved_measure_cols[prefix_col] = clean_name

    if len(resolved_measure_cols) == 0:
        possible_cols = [c for c in chr_df.columns if "raw value" in str(c)]
        raise ValueError(
            f"{source_name}: could not find expected healthcare measure columns"
            f"Example raw-value columns: {possible_cols[:25]}"
        )

    keep_cols = ["county_fips", "year", "chr_state", "chr_county"] + list(resolved_measure_cols)

    out = (
        chr_df[keep_cols]
        .copy()
        .rename(columns=resolved_measure_cols)
    )

    for col in out.columns:
        if col in ["county_fips", "year", "chr_state", "chr_county"]:
            continue
        out[col] = pd.to_numeric(out[col], errors="coerce")

    out["source_file"] = source_name

    out = (
        out
        .dropna(subset=["county_fips", "year"])
        .reset_index(drop=True)
    )

    return out

# -----
# B. Load and standardize all CHR files
# -----
healthcare_frames = []

for fp in healthcare_files:
    try:
        df_raw = pd.read_csv(fp, low_memory=False)
        # print(df_raw)
        df_std = standardize_chr_file(df_raw, fp.name)
        healthcare_frames.append(df_std)
    
```

```

        print(f"Loaded {fp.name}: {df_std.shape}")
    except Exception as e:
        print(f"Skipped {fp.name}: {e}")

if len(healthcare_frames) == 0:
    raise RuntimeError("No healthcare CSVs could be standardized successfully")

healthcare_standardized = pd.concat(healthcare_frames, ignore_index=True)

# Drop U.S. total and state rollup rows
healthcare_standardized = healthcare_standardized.loc[
    healthcare_standardized["county_fips"].notna()
    & (healthcare_standardized["county_fips"] != "00000")
    & (~healthcare_standardized["county_fips"].astype(str).str.endswith("00000"))
].copy()

# Keep the last-loaded version if a county-year repeats
healthcare_standardized = (
    healthcare_standardized
    .sort_values(["county_fips", "year", "source_file"])
    .drop_duplicates(subset=["county_fips", "year"], keep="last")
    .reset_index(drop=True)
)

healthcare_join_level = "county"

healthcare_metric_cols = [
    c for c in healthcare_standardized.columns
    if c not in ["county_fips", "year", "chr_state", "chr_county", "source_file"]
]

available_healthcare_years = sorted(
    healthcare_standardized["year"].dropna().astype(int).unique().tolist()
)

print("\nCombined healthcare shape:", healthcare_standardized.shape)
print("Available healthcare years:", available_healthcare_years)
print("Healthcare metric cols:", healthcare_metric_cols)

# -----
# C. Validate enrichment inputs from Cell 6
# -----
if "enrichment_clusters" not in globals() or len(enrichment_clusters) == 0:
    raise ValueError("enrichment_clusters is empty or missing. Rerun Cell 6")

if "enrichment_tract_year_panel" not in globals() or len(enrichment_tract_year_panel) == 0:
    raise ValueError("enrichment_tract_year_panel is empty or missing. Rerun Cell 6")

if "enrichment_cluster_year_panel" not in globals() or len(enrichment_cluster_year_panel) == 0:
    raise ValueError("enrichment_cluster_year_panel is empty or missing. Rerun Cell 6")

# -----
# D. Build YEAR-SPECIFIC county weights from shortlisted clusters
# FIX:
# - use actual year-specific tract populations
# - sum tract populations to county-year totals

```

```

# - compute county share within cluster-year
# -----
cluster_county_weights = (
    enrichment_tract_year_panel[
        ["cluster_id", "year", "geoid", "pop_total"]
    ]
    .copy()
)

cluster_county_weights["geoid"] = cluster_county_weights["geoid"].astype("str")
cluster_county_weights["county_fips"] = normalize_county_fips(
    cluster_county_weights["geoid"].str[:5]
)

cluster_county_weights["pop_total"] = pd.to_numeric(
    cluster_county_weights["pop_total"], errors="coerce"
)

cluster_county_weights = cluster_county_weights.loc[
    cluster_county_weights["year"].isin(available_healthcare_years)
].copy()

if len(cluster_county_weights) == 0:
    raise ValueError("No tract-year rows remain after restricting to healthcare years")

cluster_county_weights = (
    cluster_county_weights
    .dropna(subset=["cluster_id", "year", "county_fips", "pop_total"])
    .groupby(["cluster_id", "year", "county_fips"], as_index=False)
    .agg(cluster_county_pop=("pop_total", "sum"))
)

cluster_county_weights["cluster_total_pop_for_weights"] = (
    cluster_county_weights.groupby(["cluster_id", "year"])["cluster_county_pop"]
    .sum()
)

cluster_county_weights["cluster_county_weight"] = np.where(
    cluster_county_weights["cluster_total_pop_for_weights"] > 0,
    cluster_county_weights["cluster_county_pop"] / cluster_county_weights["cluster_total_pop_for_weights"],
    np.nan
)

cluster_county_weight_panel = cluster_county_weights.copy()

if len(cluster_county_weight_panel) == 0:
    raise ValueError("cluster_county_weight_panel is empty after building year-weighted county shares")

# -----
# E. Coverage audit before join
# -----
shortlist_fips = set(cluster_county_weight_panel["county_fips"].dropna().astype("str"))
healthcare_fips = set(healthcare_standardized["county_fips"].dropna().astype("str"))
overlap_fips = shortlist_fips & healthcare_fips

print("\nCoverage audit:")
print("Shortlist county_fips:", len(shortlist_fips))
print("Healthcare county_fips:", len(healthcare_fips))

```

```

print("Overlap county_fips:", len(overlap_fips))

missing_fips = sorted(list(shortlist_fips - healthcare_fips))
print("\nSample missing shortlist county_fips:")
print(missing_fips[:25])

healthcare_overlap_audit = pd.DataFrame({
    "metric": [
        "shortlist_county_fips",
        "healthcare_county_fips",
        "overlap_county_fips"
    ],
    "value": [
        len(shortlist_fips),
        len(healthcare_fips),
        len(overlap_fips)
    ]
})

# -----
# F. Join county healthcare into shortlisted clusters
# -----
healthcare_county_joined = (
    cluster_county_weight_panel
    .merge(
        healthcare_standardized,
        on=["county_fips", "year"],
        how="left",
        indicator=True
    )
    .copy()
)

healthcare_county_joined["matched_chr_county_flag"] = healthcare_county_joined["matched_chr_county_flag"]
healthcare_county_joined["county_has_any_healthcare_metric"] = (
    healthcare_county_joined[healthcare_metric_cols].notna().any(axis=1)
)

print("\nJoin result:")
print(healthcare_county_joined["_merge"].value_counts(dropna=False))

unmatched_counties = (
    healthcare_county_joined.loc[
        healthcare_county_joined["_merge"] == "left_only",
        ["cluster_id", "year", "county_fips"]
    ]
    .drop_duplicates()
    .sort_values(["cluster_id", "year", "county_fips"])
    .reset_index(drop=True)
)

print("\nUnmatched county-year examples:")
print(unmatched_counties.head(25))

# -----
# G. Aggregate county healthcare to cluster-year

```

```

# -----
cluster_year_meta = (
    enrichment_cluster_year_panel[
        ["cluster_id", "year", "display_state", "display_county", "n_cluster
    ]
    .drop_duplicates(subset=["cluster_id", "year"])
    .copy()
)

cluster_rows = []

for (cluster_id, year), grp in healthcare_county_joined.groupby(["cluster_id", "year"]):
    row = {
        "cluster_id": cluster_id,
        "year": int(year) if pd.notna(year) else pd.NA,
    }

    meta = cluster_year_meta.loc[
        (cluster_year_meta["cluster_id"] == cluster_id) &
        (cluster_year_meta["year"] == year)
    ]

    if len(meta) > 0:
        row["display_state"] = meta["display_state"].iloc[0]
        row["display_county"] = meta["display_county"].iloc[0]
        row["n_cluster_tracts"] = meta["n_cluster_tracts"].iloc[0]
        row["cluster_pop_total"] = meta["cluster_pop_total"].iloc[0]
    else:
        row["display_state"] = pd.NA
        row["display_county"] = pd.NA
        row["n_cluster_tracts"] = pd.NA
        row["cluster_pop_total"] = pd.NA

    weights = pd.to_numeric(grp["cluster_county_weight"], errors="coerce")

    row["cluster_county_count"] = grp["county_fips"].nunique()
    row["matched_chr_county_count"] = grp.loc[grp["matched_chr_county_flag"] == 1].nunique()
    row["matched_chr_weight_share"] = grp.loc[
        grp["matched_chr_county_flag"] & weights.notna(),
        "cluster_county_weight"
    ].sum()

    row["any_metric_weight_share"] = grp.loc[
        grp["county_has_any_healthcare_metric"] & weights.notna(),
        "cluster_county_weight"
    ].sum()

    for col in healthcare_metric_cols:
        values = pd.to_numeric(grp[col], errors="coerce")
        mask = values.notna() & weights.notna() & (weights > 0)

        if mask.sum() > 0:
            row[col] = np.average(values.loc[mask], weights=weights.loc[mask])
        elif values.notna().sum() > 0:
            row[col] = values.mean()
        else:
            row[col] = pd.NA

```

```

        row[col] = np.nan

    row["healthcare_any_data_flag"] = pd.Series(
        [row.get(col, np.nan) for col in healthcare_metric_cols]
    ).notna().any()

    cluster_rows.append(row)

if len(cluster_rows) == 0:
    raise ValueError("No cluster-year rows were produced for healthcare aggregation")

healthcare_cluster_panel = (
    pd.DataFrame(cluster_rows)
    .sort_values(["cluster_id", "year"])
    .reset_index(drop=True)
)

# Keep only clusters with actual healthcare data in summary
if healthcare_cluster_panel["healthcare_any_data_flag"].any():
    healthcare_cluster_summary = (
        healthcare_cluster_panel
        .loc[healthcare_cluster_panel["healthcare_any_data_flag"] == True]
        .groupby("cluster_id", as_index=False)
        .agg({
            "display_state": "first",
            "display_county": "first",
            "n_cluster_tracts": "max",
            "cluster_pop_total": "mean",
            "cluster_county_count": "max",
            "matched_chr_county_count": "max",
            "matched_chr_weight_share": "mean",
            "any_metric_weight_share": "mean",
            "year": "nunique",
            **{col: "mean" for col in healthcare_metric_cols}
        })
        .rename(columns={"year": "n_healthcare_years_with_any_data"})
    )
else:
    healthcare_cluster_summary = pd.DataFrame(
        columns=[
            "cluster_id",
            "display_state",
            "display_county",
            "n_cluster_tracts",
            "cluster_pop_total",
            "cluster_county_count",
            "matched_chr_county_count",
            "matched_chr_weight_share",
            "any_metric_weight_share",
            "n_healthcare_years_with_any_data",
        ] + healthcare_metric_cols
    )

print("\nHealthcare cluster panel rows with any data:")
print(healthcare_cluster_panel["healthcare_any_data_flag"].value_counts(drop=False))
print("Healthcare cluster summary shape:", healthcare_cluster_summary.shape)

```



```

# -----
# H. Save outputs
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)

healthcare_standardized.to_parquet(OUT_DIR / "healthcare_standardized.parquet")
healthcare_county_joined.to_parquet(OUT_DIR / "healthcare_county_joined.parquet")
healthcare_cluster_panel.to_parquet(OUT_DIR / "healthcare_cluster_panel.parquet")
healthcare_cluster_summary.to_parquet(OUT_DIR / "healthcare_cluster_summary.parquet")
unmatched_counties.to_parquet(OUT_DIR / "healthcare_unmatched_counties.parquet")
healthcare_overlap_audit.to_csv(OUT_DIR / "healthcare_overlap_audit.csv", index=True)

healthcare_standardized.to_csv(OUT_DIR / "healthcare_standardized.csv", index=True)
healthcare_cluster_summary.to_csv(OUT_DIR / "healthcare_cluster_summary.csv", index=True)
unmatched_counties.to_csv(OUT_DIR / "healthcare_unmatched_counties.csv", index=True)

print("\nSaved:")
print("-", OUT_DIR / "healthcare_standardized.parquet")
print("-", OUT_DIR / "healthcare_county_joined.parquet")
print("-", OUT_DIR / "healthcare_cluster_panel.parquet")
print("-", OUT_DIR / "healthcare_cluster_summary.parquet")
print("-", OUT_DIR / "healthcare_unmatched_counties.parquet")
print("-", OUT_DIR / "healthcare_overlap_audit.csv")
print("-", OUT_DIR / "healthcare_standardized.csv")
print("-", OUT_DIR / "healthcare_cluster_summary.csv")
print("-", OUT_DIR / "healthcare_unmatched_counties.csv")

print("\nHealthcare cluster panel preview:")
print(
    healthcare_cluster_panel[
        [
            "cluster_id",
            "year",
            "display_state",
            "display_county",
            "cluster_pop_total",
            "cluster_county_count",
            "matched_chr_county_count",
            "matched_chr_weight_share",
            "any_metric_weight_share",
            "healthcare_any_data_flag",
        ] + healthcare_metric_cols[:5]
    ].head(20)
)

```

Healthcare files found:

- analytic_data2017.csv
- analytic_data2018.csv
- analytic_data2019.csv
- analytic_data2020.csv
- analytic_data2021.csv
- analytic_data2022.csv
- analytic_data2023.csv

Loaded analytic_data2017.csv: (3187, 14)

Loaded analytic_data2018.csv: (3194, 14)

Loaded analytic_data2019.csv: (3194, 15)

Loaded analytic_data2020.csv: (3194, 15)

Loaded analytic_data2021.csv: (3194, 15)

Loaded analytic_data2022.csv: (3194, 15)

Loaded analytic_data2023.csv: (3194, 15)

Combined healthcare shape: (21987, 15)

Available healthcare years: [2017, 2018, 2019, 2020, 2021, 2022, 2023]

Healthcare metric cols: ['uninsured_rate', 'uninsured_adults_rate', 'uninsured_children_rate', 'primary_care_physicians_rate', 'dentists_rate', 'mental_health_providers_rate', 'other_primary_care_providers_rate', 'preventable_hospital_stays_rate', 'mammography_screening_rate', 'flu_vaccinations_rate']

Coverage audit:

Shortlist county_fips: 11

Healthcare county_fips: 3142

Overlap county_fips: 8

Sample missing shortlist county_fips:

['72097', '72119', '72141']

Join result:

_merge

both 84

left_only 21

right_only 0

Name: count, dtype: int64

Unmatched county-year examples:

			cluster_id	year	county_fips
0	Puerto Rico	Mayagüez Municipio	cluster_73	2017	72097
1	Puerto Rico	Mayagüez Municipio	cluster_73	2018	72097
2	Puerto Rico	Mayagüez Municipio	cluster_73	2019	72097
3	Puerto Rico	Mayagüez Municipio	cluster_73	2020	72097
4	Puerto Rico	Mayagüez Municipio	cluster_73	2021	72097
5	Puerto Rico	Mayagüez Municipio	cluster_73	2022	72097
6	Puerto Rico	Mayagüez Municipio	cluster_73	2023	72097
7	Puerto Rico	Río Grande Municipio	cluster_74	2017	72119
8	Puerto Rico	Río Grande Municipio	cluster_74	2018	72119
9	Puerto Rico	Río Grande Municipio	cluster_74	2019	72119
10	Puerto Rico	Río Grande Municipio	cluster_74	2020	72119
11	Puerto Rico	Río Grande Municipio	cluster_74	2021	72119
12	Puerto Rico	Río Grande Municipio	cluster_74	2022	72119
13	Puerto Rico	Río Grande Municipio	cluster_74	2023	72119
14	Puerto Rico	Utuado Municipio	cluster_75	2017	72141
15	Puerto Rico	Utuado Municipio	cluster_75	2018	72141

16	Puerto Rico	Utua	cluster_75	2019	72141
17	Puerto Rico	Utua	cluster_75	2020	72141
18	Puerto Rico	Utua	cluster_75	2021	72141
19	Puerto Rico	Utua	cluster_75	2022	72141
20	Puerto Rico	Utua	cluster_75	2023	72141

Healthcare cluster panel rows with any data:

healthcare_any_data_flag

True 84

False 21

Name: count, dtype: int64

Healthcare cluster summary shape: (12, 20)

Saved:

- data\processed\healthcare_standardized.parquet
- data\processed\healthcare_county_joined.parquet
- data\processed\healthcare_cluster_panel.parquet
- data\processed\healthcare_cluster_summary.parquet
- data\processed\healthcare_unmatched_counties.parquet
- data\processed\healthcare_overlap_audit.csv
- data\processed\healthcare_standardized.csv
- data\processed\healthcare_cluster_summary.csv
- data\processed\healthcare_unmatched_counties.csv

Healthcare cluster panel preview:

			cluster_id	year	display_state	di
splay_county	cluster_pop_total	cluster_county_count	matched_ch	county_co		
0	Arizona Apache County	cluster_3	2017	Arizona	A	
1	59847.0	1				
1	1.0					
1	Arizona Apache County	cluster_3	2018	Arizona	A	
1	60252.0	1				
1	1.0					
2	Arizona Apache County	cluster_3	2019	Arizona	A	
1	60948.0	1				
1	1.0					
3	Arizona Apache County	cluster_3	2020	Arizona	A	
1	71714.0	1				
1	1.0					
4	Arizona Apache County	cluster_3	2021	Arizona	A	
1	66473.0	1				
1	1.0					
5	Arizona Apache County	cluster_3	2022	Arizona	A	
1	66054.0	1				
1	1.0					
6	Arizona Apache County	cluster_3	2023	Arizona	A	
1	65680.0	1				
1	1.0					
7	California Los Angeles County	cluster_11	2017	California	Los An	
1	23130.0	1				
1	1.0					
8	California Los Angeles County	cluster_11	2018	California	Los An	
1	22941.0	1				
1	1.0					
9	California Los Angeles County	cluster_11	2019	California	Los An	

geles County	22432.0	1		
1	1.0			
10 California Los Angeles County cluster_11	2020	California	Los An	
geles County	21626.0	1		
1	1.0			
11 California Los Angeles County cluster_11	2021	California	Los An	
geles County	20770.0	1		
1	1.0			
12 California Los Angeles County cluster_11	2022	California	Los An	
geles County	20258.0	1		
1	1.0			
13 California Los Angeles County cluster_11	2023	California	Los An	
geles County	19958.0	1		
1	1.0			
14 Illinois Cook County cluster_24	2017	Illinois		
Cook County	178355.0	1		
1	1.0			
15 Illinois Cook County cluster_24	2018	Illinois		
Cook County	174286.0	1		
1	1.0			
16 Illinois Cook County cluster_24	2019	Illinois		
Cook County	171203.0	1		
1	1.0			
17 Illinois Cook County cluster_24	2020	Illinois		
Cook County	181444.0	1		
1	1.0			
18 Illinois Cook County cluster_24	2021	Illinois		
Cook County	183991.0	1		
1	1.0			
19 Illinois Cook County cluster_24	2022	Illinois		
Cook County	185035.0	1		
1	1.0			

	any_metric_weight_share	healthcare_any_data_flag	uninsured_rate	uninsured_adults_rate	uninsured_children_rate	primary_care_physicians_rate	dentists_rate
0	1.0	True	0.210486				
0.254249	0.125173		0.000292	0.0006			
16							
1	1.0	True	0.175198				
0.203082	0.120371		0.000294	0.0006			
57							
2	1.0	True	0.195142				
0.235324	0.116927		0.000328	0.0006			
84							
3	1.0	True	0.191727				
0.229970	0.112950		0.000321	0.0006			
54							
4	1.0	True	0.211884				
0.244723	0.142148		0.000390	0.0007			
37							
5	1.0	True	0.209902				
0.243381	0.138561		0.000431	0.0007			
10							
6	1.0	True	0.186161				
0.217248	0.118849		0.000390	0.0007			

92				
7	1.0	True	0.173704	
0.214741	0.060671		0.000721	0.0008
09				
8	1.0	True	0.124921	
0.153743	0.043855		0.000719	0.0008
34				
9	1.0	True	0.106733	
0.131414	0.036647		0.000723	0.0008
50				
10	1.0	True	0.100595	
0.123550	0.034665		0.000728	0.0008
64				
11	1.0	True	0.101961	
0.125476	0.033989		0.000733	0.0008
96				
12	1.0	True	0.111095	
0.135789	0.038542		0.000738	0.0009
11				
13	1.0	True	0.101865	
0.124670	0.033844		0.000744	0.0009
42				
14	1.0	True	0.140555	
0.176588	0.040261		0.000942	0.0008
47				
15	1.0	True	0.106296	
0.133566	0.030314		0.000942	0.0008
82				
16	1.0	True	0.094137	
0.117878	0.027753		0.000956	0.0008
95				
17	1.0	True	0.099723	
0.123408	0.032616		0.000933	0.0009
11				
18	1.0	True	0.101789	
0.123949	0.038905		0.000948	0.0009
45				
19	1.0	True	0.108698	
0.131163	0.044418		0.000956	0.0009
56				

In [54]: `### Cell 7.5 – healthcare feasibility decision checkpoint`

In [55]: `# Cell 7.5 – healthcare feasibility decision checkpoint
Purpose:
make the healthcare-source mismatch explicit before any healthcare scoring

A. cluster-level healthcare feasibility summary

healthcare_feasibility = (
 healthcare_cluster_panel
 .groupby("cluster_id", as_index=False)
 .agg(
 display_state=("display_state", "first"),
 display_county=("display_county", "first"),`

```

        n_cluster_tracts=("n_cluster_tracts", "max"),
        cluster_pop_latest_proxy=("cluster_pop_total", "max"),
        n_years_in_panel=("year", "nunique"),
        n_years_with_any_healthcare_data=("healthcare_any_data_flag", lambda
        mean_matched_chr_weight_share=("matched_chr_weight_share", "mean"),
        mean_any_metric_weight_share=("any_metric_weight_share", "mean"),
    )
    .copy()
)

healthcare_feasibility["healthcare_usable_flag"] = (
    (healthcare_feasibility["n_years_with_any_healthcare_data"] >= 2) &
    (healthcare_feasibility["mean_any_metric_weight_share"] > 0)
)

healthcare_feasibility = (
    healthcare_feasibility
    .merge(
        enrichment_clusters[
            ["cluster_id", "enrichment_rank", "rank", "cluster_shortlist_score"]
        ].drop_duplicates(),
        on="cluster_id",
        how="left",
        validate="1:1"
    )
    .sort_values(["enrichment_rank", "cluster_id"])
    .reset_index(drop=True)
)

# -----
# B. overall decision stats
# -----
n_shortlisted_clusters = len(enrichment_clusters)
n_usable_healthcare_clusters = int(healthcare_feasibility["healthcare_usable_flag"] > 0)

decision_summary = pd.DataFrame({
    "metric": [
        "shortlisted_clusters",
        "clusters_with_usable_healthcare_data",
        "clusters_without_usable_healthcare_data",
        "share_usable"
    ],
    "value": [
        n_shortlisted_clusters,
        n_usable_healthcare_clusters,
        n_shortlisted_clusters - n_usable_healthcare_clusters,
        (n_usable_healthcare_clusters / n_shortlisted_clusters) if n_shortlisted_clusters > 0 else 0
    ]
})

# -----
# C. explicit notebook decision
# -----
if n_usable_healthcare_clusters <= 1:
    healthcare_decision_message = (
        "County Health Rankings is not sufficient for cross-cluster healthcare decision making"
    )

```

```

        "on the current shortlist. The shortlist is mostly Puerto Rico, while
        "matches one shortlisted cluster. Do NOT build the final healthcare
        "this source as-is."
    )
    recommended_next_step = (
        "Choose one of two branches: "
        "(1) keep the current shortlist and switch to a healthcare source that
        "(2) keep CHR and create a mainland-compatible healthcare branch for
    )
else:
    healthcare_decision_message = (
        "County Health Rankings has enough overlap to support preliminary he
        "across multiple shortlisted clusters."
    )
    recommended_next_step = (
        "Proceed to Cell 8 using recovery-period healthcare metrics only, with
    )

print("Healthcare decision checkpoint")
print("-" * 60)
print(healthcare_decision_message)
print()
print("Recommended next step:")
print(recommended_next_step)

print("\nDecision summary:")
print(decision_summary)

print("\nCluster-level healthcare feasibility:")
print(
    healthcare_feasibility[
        [
            "enrichment_rank",
            "cluster_id",
            "display_state",
            "display_county",
            "n_years_in_panel",
            "n_years_with_any_healthcare_data",
            "mean_matched_chr_weight_share",
            "mean_any_metric_weight_share",
            "healthcare_usable_flag",
        ]
    ]
)

# -----
# D. Save outputs
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)

healthcare_feasibility.to_parquet(OUT_DIR / "healthcare_feasibility.parquet")
healthcare_feasibility.to_csv(OUT_DIR / "healthcare_feasibility.csv", index=False)
decision_summary.to_csv(OUT_DIR / "healthcare_decision_summary.csv", index=False)

print("\nSaved:")
print("-", OUT_DIR / "healthcare_feasibility.parquet")

```

```
print("-", OUT_DIR / "healthcare_feasibility.csv")  
print("-", OUT_DIR / "healthcare_decision_summary.csv")
```


Healthcare decision checkpoint

County Health Rankings has enough overlap to support preliminary healthcare comparison across multiple shortlisted clusters.

Recommended next step:

Proceed to Cell 8 using recovery-period healthcare metrics only, with coverage filters.

Decision summary:

	metric	value
0	shortlisted_clusters	15.0
1	clusters_with_usable_healthcare_data	12.0
2	clusters_without_usable_healthcare_data	3.0
3	share_usable	0.8

Cluster-level healthcare feasibility:

enrichment_rank	cluster_id	display			
_state	display_county	n_years_in_panel	n_years_with_any_healthcare_data	mean_matched_chr_weight_share	\
0	1	Puerto Rico Utuado Municipio	cluster_75	Puert	
o Rico	Utuado Municipio	7			
0		0.0			
1	2	Puerto Rico Mayagüez Municipio	cluster_73	Puert	
o Rico	Mayagüez Municipio	7			
0		0.0			
2	3	Puerto Rico Río Grande Municipio	cluster_74	Puert	
o Rico	Río Grande Municipio	7			
0		0.0			
3	4	Ohio Cuyahoga County	cluster_64		
Ohio	Cuyahoga County	7			
7		1.0			
4	5	Michigan Wayne County	cluster_38	Mi	
chigan	Wayne County	7			
7		1.0			
5	6	Illinois Cook County	cluster_29	Il	
linois	Cook County	7			
7		1.0			
6	7	Arizona Apache County	cluster_3	A	
izona	Apache County	7			
7		1.0			
7	8	Illinois Cook County	cluster_25	Il	
linois	Cook County	7			
7		1.0			
8	9	Michigan Wayne County	cluster_39	Mi	
chigan	Wayne County	7			
7		1.0			
9	10	Illinois Cook County	cluster_24	Il	
linois	Cook County	7			
7		1.0			
10	11	Tennessee Shelby County	cluster_77	Ten	
nessee	Shelby County	7			
7		1.0			
11	12	New Mexico McKinley County	cluster_49	New	
Mexico	McKinley County	7			
7		1.0			

id	state	county	cluster	weight_share	usable_flag
12	California	Los Angeles County	cluster_11	1.0	False
13	California	Los Angeles County	cluster_11	1.0	False
14	Ohio	Cuyahoga County	cluster_63	1.0	True
15	Wisconsin	Milwaukee County	cluster_86	1.0	True

id	mean_any_metric_weight_share	healthcare_usable_flag
0	0.0	False
1	0.0	False
2	0.0	False
3	1.0	True
4	1.0	True
5	1.0	True
6	1.0	True
7	1.0	True
8	1.0	True
9	1.0	True
10	1.0	True
11	1.0	True
12	1.0	True
13	1.0	True
14	1.0	True

Saved:

- data\processed\healthcare_feasibility.parquet
- data\processed\healthcare_feasibility.csv
- data\processed\healthcare_decision_summary.csv

```
In [56]: # Cell 8 – score healthcare need on the healthcare-covered subset (recovery
# IMPORTANT:
# - Uses County Health Rankings only where coverage is actually usable
# - Uses recovery-period overlap years only (given CHR availability, this sh
# - Scores metrics within-year, then averages across recovery years
# so cross-year raw-value scale changes do not distort the comparison

import pandas as pd
import numpy as np

# -----
# A. Helpers
# -----
def bad_when_high(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return s.rank(pct=True, method="average")

def bad_when_low(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return (-s).rank(pct=True, method="average")

# -----
# B. Basic validation
# -----
```

```

if "healthcare_cluster_panel" not in globals() or len(healthcare_cluster_panel) == 0:
    raise ValueError("healthcare_cluster_panel is empty or missing. Rerun Cell 4")

if "cluster_shortlist" not in globals() or len(cluster_shortlist) == 0:
    raise ValueError("cluster_shortlist is empty or missing. Rerun Cell 5")

if "healthcare_metric_cols" not in globals() or len(healthcare_metric_cols) == 0:
    raise ValueError("healthcare_metric_cols is empty or missing. Rerun Cell 6")

# -----
# C. Determine usable recovery years for healthcare scoring
# -----
recovery_years_from_main = sorted(
    cluster_year_panel.loc[
        cluster_year_panel["analysis_period"] == "recovery",
        "year"
    ].dropna().astype(int).unique().tolist()
)

healthcare_years_available = sorted(
    healthcare_cluster_panel["year"].dropna().astype(int).unique().tolist()
)

healthcare_recovery_years = sorted(set(recovery_years_from_main) & set(healthcare_years_available))

if len(healthcare_recovery_years) == 0:
    raise ValueError(
        "No overlap between recovery years in the main panel and available healthcare years"
    )

# With CHR through 2023 and recovery = 2022–2024, this should usually be [2022, 2023, 2024]
print("Main recovery years:", recovery_years_from_main)
print("Healthcare years available:", healthcare_years_available)
print("Healthcare recovery years used for scoring:", healthcare_recovery_years)

# -----
# D. Metric directions
# higher is worse vs lower is worse
# -----
healthcare_metric_direction_map = {
    "uninsured_rate": "high_is_bad",
    "uninsured_adults_rate": "high_is_bad",
    "uninsured_children_rate": "high_is_bad",
    "primary_care_physicians_rate": "low_is_bad",
    "dentists_rate": "low_is_bad",
    "mental_health_providers_rate": "low_is_bad",
    "other_primary_care_providers_rate": "low_is_bad",
    "preventable_hospital_stays_rate": "high_is_bad",
    "mammography_screening_rate": "low_is_bad",
    "flu_vaccinations_rate": "low_is_bad",
}

healthcare_scoring_metrics = [
    c for c in healthcare_metric_cols
    if c in healthcare_metric_direction_map
]

```

```

if len(healthcare_scoring_metrics) == 0:
    raise ValueError("No healthcare scoring metrics were found in healthcare

healthcare_metric_directions = pd.DataFrame({
    "metric": healthcare_scoring_metrics,
    "direction": [healthcare_metric_direction_map[c] for c in healthcare_sco
})

# -----
# E. Build recovery-period healthcare base
# -----
MIN_MATCHED_WEIGHT_SHARE = 0.50
MIN_ANY_METRIC_WEIGHT_SHARE = 0.50
MIN_METRICS_PRESENT_PER_YEAR = 4
MIN_RECOVERY_HEALTHCARE_YEARS = min(2, len(healthcare_recovery_years))

healthcare_recovery_panel = (
    healthcare_cluster_panel.loc[
        healthcare_cluster_panel["year"].isin(healthcare_recovery_years)
    ]
    .copy()
    .sort_values(["cluster_id", "year"])
    .reset_index(drop=True)
)

healthcare_recovery_panel["n_nonmissing_healthcare_metrics"] = (
    healthcare_recovery_panel[healthcare_scoring_metrics].notna().sum(axis=1
)

healthcare_recovery_panel["recovery_healthcare_row_usable_flag"] = (
    healthcare_recovery_panel["healthcare_any_data_flag"].fillna(False)
    & (pd.to_numeric(healthcare_recovery_panel["matched_chr_weight_share"],
    & (pd.to_numeric(healthcare_recovery_panel["any_metric_weight_share"], e
    & (pd.to_numeric(healthcare_recovery_panel["n_nonmissing_healthcare_metr
)

# cluster-level coverage summary over recovery years
healthcare_recovery_coverage = (
    healthcare_recovery_panel
    .groupby("cluster_id", as_index=False)
    .agg(
        display_state=("display_state", "first"),
        display_county=("display_county", "first"),
        n_cluster_tracts=("n_cluster_tracts", "max"),
        n_recovery_healthcare_years=("year", "nunique"),
        n_usable_recovery_healthcare_years=("recovery_healthcare_row_usable_
        mean_matched_chr_weight_share=("matched_chr_weight_share", "mean"),
        mean_any_metric_weight_share=("any_metric_weight_share", "mean"),
        mean_nonmissing_healthcare_metrics=("n_nonmissing_healthcare_metrics
    )
    .copy()
)

healthcare_recovery_coverage["healthcare_eligible_flag"] = (
    (healthcare_recovery_coverage["n_usable_recovery_healthcare_years"] >= M

```

```

        & (healthcare_recovery_coverage["mean_any_metric_weight_share"] >= MIN_A
    )

eligible_healthcare_cluster_ids = set(
    healthcare_recovery_coverage.loc[
        healthcare_recovery_coverage["healthcare_eligible_flag"],
        "cluster_id"
    ].astype(str).tolist()
)

print("\nHealthcare recovery coverage summary:")
print("Recovery healthcare rows:", healthcare_recovery_panel.shape[0])
print("Eligible healthcare clusters:", len(eligible_healthcare_cluster_ids))
print("Min usable recovery years required:", MIN_RECOVERY_HEALTHCARE_YEARS)
print()

print(
    healthcare_recovery_coverage.sort_values(
        ["healthcare_eligible_flag", "n_usable_recovery_healthcare_years", "
        ascending=[False, False, False, True]
    )
)

# -----
# F. Score healthcare need within each year
# (rank within year, then average across years)
# -----
healthcare_scored_yearly = (
    healthcare_recovery_panel.loc[
        healthcare_recovery_panel["cluster_id"].astype(str).isin(eligible_he
        & healthcare_recovery_panel["recovery_healthcare_row_usable_flag"]
    ]
    .copy()
)

if len(healthcare_scored_yearly) == 0:
    print("\nNo clusters met healthcare eligibility for scoring.")
    healthcare_cluster_scored = pd.DataFrame(columns=[
        "cluster_id",
        "display_state",
        "display_county",
        "n_cluster_tracts",
        "n_recovery_healthcare_years_scored",
        "mean_matched_chr_weight_share",
        "mean_any_metric_weight_share",
        "mean_nonmissing_healthcare_metrics",
        "healthcare_need_score",
        "healthcare_need_rank",
    ] + healthcare_scoring_metrics)
else:
    metric_score_cols = []

    for metric in healthcare_scoring_metrics:
        direction = healthcare_metric_direction_map[metric]
        score_col = f"{metric}_need_score"
        metric_score_cols.append(score_col)

```

```

        if direction == "high_is_bad":
            healthcare_scored_yearly[score_col] = (
                healthcare_scored_yearly
                .groupby("year")[metric]
                .transform(bad_when_high)
            )
        elif direction == "low_is_bad":
            healthcare_scored_yearly[score_col] = (
                healthcare_scored_yearly
                .groupby("year")[metric]
                .transform(bad_when_low)
            )
        else:
            raise ValueError(f"Unexpected direction for {metric}: {direction}")

healthcare_scored_yearly["healthcare_need_score_year"] = (
    healthcare_scored_yearly[metric_score_cols].mean(axis=1, skipna=True)
)

# -----
# G. Aggregate to cluster-level healthcare score
# -----
healthcare_cluster_scored = (
    healthcare_scored_yearly
    .groupby("cluster_id", as_index=False)
    .agg(
        display_state=("display_state", "first"),
        display_county=("display_county", "first"),
        n_cluster_tracts=("n_cluster_tracts", "max"),
        n_recovery_healthcare_years_scored=("year", "nunique"),
        mean_matched_chr_weight_share=("matched_chr_weight_share", "mean"),
        mean_any_metric_weight_share=("any_metric_weight_share", "mean"),
        mean_nonmissing_healthcare_metrics=("n_nonmissing_healthcare_met
        healthcare_need_score=("healthcare_need_score_year", "mean"),
        **{metric: (metric, "mean") for metric in healthcare_scoring_met
    )
    .copy()
)

healthcare_cluster_scored["healthcare_need_rank"] = (
    healthcare_cluster_scored["healthcare_need_score"]
    .rank(method="first", ascending=False)
    .astype(int)
)

healthcare_cluster_scored = healthcare_cluster_scored.sort_values(
    ["healthcare_need_score", "cluster_id"],
    ascending=[False, True]
).reset_index(drop=True)

# -----
# H. Merge healthcare back into the shortlist
# -----
cluster_shortlist_with_healthcare = (
    cluster_shortlist

```

```

.merge(
    healthcare_recovery_coverage[
        [
            "cluster_id",
            "n_recovery_healthcare_years",
            "n_usable_recovery_healthcare_years",
            "mean_matched_chr_weight_share",
            "mean_any_metric_weight_share",
            "mean_nonmissing_healthcare_metrics",
            "healthcare_eligible_flag",
        ]
    ],
    on="cluster_id",
    how="left",
    validate="1:1"
)
.merge(
    healthcare_cluster_scored[
        ["cluster_id", "healthcare_need_score", "healthcare_need_rank"]
    ] if len(healthcare_cluster_scored) > 0 else pd.DataFrame(columns=["cluster_id",
    on="cluster_id",
    how="left",
    validate="1:1"
)
.copy()
)

cluster_shortlist_with_healthcare["healthcare_eligible_flag"] = (
    cluster_shortlist_with_healthcare["healthcare_eligible_flag"].fillna(False)
)

# preliminary combined score before business enrichment
# keep the shortlist foundation primary, then let healthcare refine among cc
SHORTLIST_WEIGHT = 0.70
HEALTHCARE_WEIGHT = 0.30

cluster_shortlist_with_healthcare["preliminary_combined_score"] = np.where(
    cluster_shortlist_with_healthcare["healthcare_eligible_flag"]
    & cluster_shortlist_with_healthcare["healthcare_need_score"].notna(),
    (SHORTLIST_WEIGHT * cluster_shortlist_with_healthcare["cluster_shortlist_score"]
    + (HEALTHCARE_WEIGHT * cluster_shortlist_with_healthcare["healthcare_need_score"]
    np.nan
)

healthcare_comparison_set = (
    cluster_shortlist_with_healthcare
    .loc[cluster_shortlist_with_healthcare["healthcare_eligible_flag"]]
    .sort_values(
        [
            "preliminary_combined_score",
            "healthcare_need_score",
            "cluster_shortlist_score",
            "latest_cluster_pop_total",
        ],
        ascending=[False, False, False, False]
    )
)

```

```

        .reset_index(drop=True)
    )

    if len(healthcare_comparison_set) > 0:
        healthcare_comparison_set["healthcare_comparison_rank"] = np.arange(1, len(healthcare_comparison_set) + 1)

    # -----
    # I. Diagnostics
    # -----
    print("\nHealthcare metric directions:")
    print(healthcare_metric_directions)

    print("\nHealthcare-scored yearly panel shape:", healthcare_scored_yearly.shape)
    print("Healthcare cluster scored shape:", healthcare_cluster_scored.shape)
    print("Shortlist with healthcare shape:", cluster_shortlist_with_healthcare.shape)
    print("Healthcare comparison set shape:", healthcare_comparison_set.shape)

    print("\nTop healthcare-covered clusters:")
    view_cols = [
        "rank",
        "cluster_id",
        "display_state",
        "display_county",
        "n_cluster_tracts",
        "latest_cluster_pop_total",
        "cluster_shortlist_score",
        "n_usable_recovery_healthcare_years",
        "mean_matched_chr_weight_share",
        "mean_any_metric_weight_share",
        "healthcare_need_score",
        "preliminary_combined_score",
    ]

    if len(healthcare_comparison_set) > 0:
        print(healthcare_comparison_set[view_cols].head(20))
    else:
        print("No healthcare-eligible clusters were available for comparison.")

    print("\nShortlist clusters without usable healthcare coverage:")
    print(
        cluster_shortlist_with_healthcare.loc[
            ~cluster_shortlist_with_healthcare["healthcare_eligible_flag"],
            ["rank", "cluster_id", "display_state", "display_county", "cluster_score"],
        ].sort_values(["rank", "cluster_id"]).reset_index(drop=True)
    )

    # -----
    # J. Save outputs
    # -----
    OUT_DIR.mkdir(parents=True, exist_ok=True)

    healthcare_metric_directions.to_csv(OUT_DIR / "healthcare_metric_directions.csv")
    healthcare_recovery_panel.to_parquet(OUT_DIR / "healthcare_recovery_panel.parquet")
    healthcare_recovery_coverage.to_parquet(OUT_DIR / "healthcare_recovery_coverage.parquet")
    healthcare_scored_yearly.to_parquet(OUT_DIR / "healthcare_scored_yearly.parquet")
    healthcare_cluster_scored.to_parquet(OUT_DIR / "healthcare_cluster_scored.parquet")

```



```

cluster_shortlist_with_healthcare.to_parquet(OUT_DIR / "cluster_shortlist_with_healthcare_comparison_set.to_parquet(OUT_DIR / "healthcare_comparison_set.pa

cluster_shortlist_with_healthcare.to_csv(OUT_DIR / "cluster_shortlist_with_healthcare_comparison_set.to_csv(OUT_DIR / "healthcare_comparison_set.csv",

print("\nSaved:")
print("-", OUT_DIR / "healthcare_metric_directions.csv")
print("-", OUT_DIR / "healthcare_recovery_panel.parquet")
print("-", OUT_DIR / "healthcare_recovery_coverage.parquet")
print("-", OUT_DIR / "healthcare_scored_yearly.parquet")
print("-", OUT_DIR / "healthcare_cluster_scored.parquet")
print("-", OUT_DIR / "cluster_shortlist_with_healthcare.parquet")
print("-", OUT_DIR / "healthcare_comparison_set.parquet")
print("-", OUT_DIR / "cluster_shortlist_with_healthcare.csv")
print("-", OUT_DIR / "healthcare_comparison_set.csv")

#12 eligible out of the 15 enrichment clusters
# not "12 eligible out of the full 67 shortlist clusters"

```

Main recovery years: [2022, 2023, 2024]
 Healthcare years available: [2017, 2018, 2019, 2020, 2021, 2022, 2023]
 Healthcare recovery years used for scoring: [2022, 2023]

Healthcare recovery coverage summary:
 Recovery healthcare rows: 30
 Eligible healthcare clusters: 12
 Min usable recovery years required: 2

play_county	n_cluster_tracts	n_recovery_healthcare_years	cluster_id display_state	dis
0	Arizona Apache County	cluster_3	Arizona	Ap
ache County	18		2	
2				
1	California Los Angeles County	cluster_11	California	Los Ang
eles County	5		2	
2				
2	Illinois Cook County	cluster_24	Illinois	
Cook County	85		2	
2				
3	Illinois Cook County	cluster_25	Illinois	
Cook County	33		2	
2				
4	Illinois Cook County	cluster_29	Illinois	
Cook County	9		2	
2				
5	Michigan Wayne County	cluster_38	Michigan	W
ayne County	152		2	
2				
6	Michigan Wayne County	cluster_39	Michigan	W
ayne County	9		2	
2				
7	New Mexico McKinley County	cluster_49	New Mexico	McKi
nley County	33		2	
2				
8	Ohio Cuyahoga County	cluster_63	Ohio	Cuya
hoga County	107		2	
2				
9	Ohio Cuyahoga County	cluster_64	Ohio	Cuya
hoga County	13		2	
2				
13	Tennessee Shelby County	cluster_77	Tennessee	Sh
elby County	96		2	
2				
14	Wisconsin Milwaukee County	cluster_86	Wisconsin	Milwa
ukee County	98		2	
2				
10	Puerto Rico Mayagüez Municipio	cluster_73	Puerto Rico	Mayagüe
z Municipio	25		2	
0				
11	Puerto Rico Río Grande Municipio	cluster_74	Puerto Rico	Río Grand
e Municipio	12		2	
0				
12	Puerto Rico Utuado Municipio	cluster_75	Puerto Rico	Utuad
o Municipio	8		2	

0

	mean_matched_chr_weight_share	mean_any_metric_weight_share	mean_nonmis
0	1.0	1.0	
10.0	True		
1	1.0	1.0	
10.0	True		
2	1.0	1.0	
10.0	True		
3	1.0	1.0	
10.0	True		
4	1.0	1.0	
10.0	True		
5	1.0	1.0	
10.0	True		
6	1.0	1.0	
10.0	True		
7	1.0	1.0	
10.0	True		
8	1.0	1.0	
10.0	True		
9	1.0	1.0	
10.0	True		
13	1.0	1.0	
10.0	True		
14	1.0	1.0	
10.0	True		
10	0.0	0.0	
0.0	False		
11	0.0	0.0	
0.0	False		
12	0.0	0.0	
0.0	False		

Healthcare metric directions:

	metric	direction
0	uninsured_rate	high_is_bad
1	uninsured_adults_rate	high_is_bad
2	uninsured_children_rate	high_is_bad
3	primary_care_physicians_rate	low_is_bad
4	dentists_rate	low_is_bad
5	mental_health_providers_rate	low_is_bad
6	other_primary_care_providers_rate	low_is_bad
7	preventable_hospital_stays_rate	high_is_bad
8	mammography_screening_rate	low_is_bad
9	flu_vaccinations_rate	low_is_bad

Healthcare-scored yearly panel shape: (24, 34)

Healthcare cluster scored shape: (12, 20)

Shortlist with healthcare shape: (67, 74)

Healthcare comparison set shape: (12, 75)

Top healthcare-covered clusters:

rank	cluster_id	display_state	di
splay_county	n_cluster_tracts	latest_cluster_pop_total	cluster_shortlist_

score	n_usable_recovery_healthcare_years	\				
0	7	Arizona	Apache County	cluster_3	Arizona	A
42122			18	65341.0		0.7
				2.0		
1	12	New Mexico	McKinley County	cluster_49	New Mexico	McK
05959			33	70431.0		0.7
				2.0		
2	5	Michigan	Wayne County	cluster_38	Michigan	
72703			152	319984.0		0.7
				2.0		
3	11	Tennessee	Shelby County	cluster_77	Tennessee	S
12122			96	268298.0		0.7
				2.0		
4	6	Illinois	Cook County	cluster_29	Illinois	
2267			9	27412.0		0.75
				2.0		
5	13	California	Los Angeles County	cluster_11	California	Los An
05523			5	19914.0		0.7
				2.0		
6	9	Michigan	Wayne County	cluster_39	Michigan	
33314			9	19947.0		0.7
				2.0		
7	8	Illinois	Cook County	cluster_25	Illinois	
4564			33	81348.0		0.73
				2.0		
8	10	Illinois	Cook County	cluster_24	Illinois	
2355			85	187702.0		0.71
				2.0		
9	4	Ohio	Cuyahoga County	cluster_64	Ohio	Cuy
11047			13	31886.0		0.8
				2.0		
10	15	Wisconsin	Milwaukee County	cluster_86	Wisconsin	Milw
85087			98	225339.0		0.6
				2.0		
11	14	Ohio	Cuyahoga County	cluster_63	Ohio	Cuy
98227			107	210162.0		0.6
				2.0		

	mean_matched_chr_weight_share	mean_any_metric_weight_share	healthcare_
need_score	preliminary_combined_score		
0	1.0		1.0
0.879167	0.783235		
1	1.0		1.0
0.822917	0.741047		
2	1.0		1.0
0.550000	0.705892		
3	1.0		1.0
0.645833	0.692235		
4	1.0		1.0
0.510417	0.679712		
5	1.0		1.0
0.616667	0.678866		
6	1.0		1.0
0.550000	0.678320		
7	1.0		1.0
0.510417	0.667320		

8	1.0	1.0
0.510417	0.651773	
9	1.0	1.0
0.260417	0.645858	
10	1.0	1.0
0.383333	0.594561	
11	1.0	1.0
0.260417	0.566884	

Shortlist clusters without usable healthcare coverage:

rank	cluster_id	display_
state	display_county	cluster_shortlist_score
0	1	Puerto Rico Utuado Municipio cluster_75
Rico	Utuado Municipio	0.869855
1	2	Puerto Rico Mayagüez Municipio cluster_73
Rico	Mayagüez Municipio	0.848576
2	3	Puerto Rico Río Grande Municipio cluster_74
Rico	Río Grande Municipio	0.821570
3	16	Louisiana Orleans Parish cluster_33
siana	Orleans Parish	0.682529
4	17	New York Kings County cluster_51
York	Kings County	0.677122
5	18	New York Kings County cluster_53
York	Kings County	0.660320
6	19	North Carolina Robeson County cluster_62
olina	Robeson County	0.657035
7	20	California Los Angeles County cluster_8
ornia	Los Angeles County	0.641657
8	21	Texas Harris County cluster_83
Texas	Harris County	0.627558
9	22	Illinois Cook County cluster_26
inois	Cook County	0.622965
10	23	Arkansas Chicot County cluster_6
ansas	Chicot County	0.622703
11	24	New York Kings County cluster_59
York	Kings County	0.611308
12	25	Maryland Baltimore city cluster_34
yland	Baltimore city	0.608983
13	26	Texas Harris County cluster_78
Texas	Harris County	0.604855
14	27	California Los Angeles County cluster_7
ornia	Los Angeles County	0.599709
15	28	New York Kings County cluster_58
York	Kings County	0.591512
16	29	Mississippi Washington County cluster_40
sippi	Washington County	0.590203
17	30	Michigan Genesee County cluster_37
higan	Genesee County	0.588692
18	31	New York Kings County cluster_57
York	Kings County	0.587791
19	32	Illinois Cook County cluster_27
inois	Cook County	0.586279
20	33	Missouri St. Louis city cluster_41
souri	St. Louis city	0.574622
21	34	New Mexico San Juan County cluster_50
exico	San Juan County	0.565233

22	35	New York Kings County cluster_54	New
York		Kings County 0.563547	
23	36	Florida Palm Beach County cluster_17	Fl
orida		Palm Beach County 0.559157	
24	37	Illinois Cook County cluster_28	Ill
inois		Cook County 0.556715	
25	38	Texas Harris County cluster_79	
Texas		Harris County 0.547558	
26	39	Texas Harris County cluster_81	
Texas		Harris County 0.547122	
27	40	Kentucky Lawrence County cluster_32	Ken
tucky		Lawrence County 0.541134	
28	41	Indiana Lake County cluster_30	In
diana		Lake County 0.534622	
29	42	California Los Angeles County cluster_14	Calif
ornia		Los Angeles County 0.532122	
30	43	Nevada Clark County cluster_43	N
evada		Clark County 0.520436	
31	44	Pennsylvania Allegheny County cluster_69	Pennsylv
vania		Allegheny County 0.518750	
32	45	Pennsylvania Allegheny County cluster_71	Pennsylv
vania		Allegheny County 0.518721	
33	46	New York Kings County cluster_55	New
York		Kings County 0.517936	
34	47	Nevada Clark County cluster_45	N
evada		Clark County 0.516395	
35	48	California Los Angeles County cluster_12	Calif
ornia		Los Angeles County 0.503343	
36	49	Alabama Mobile County cluster_1	Al
abama		Mobile County 0.502936	
37	50	Arizona Navajo County cluster_5	Ar
izona		Navajo County 0.480727	
38	51	Pennsylvania Allegheny County cluster_70	Pennsylv
vania		Allegheny County 0.471860	
39	52	Nevada Clark County cluster_44	N
evada		Clark County 0.469913	
40	53	Ohio Mahoning County cluster_65	
Ohio		Mahoning County 0.453401	
41	54	New York Monroe County cluster_61	New
York		Monroe County 0.452878	
42	55	California Los Angeles County cluster_9	Calif
ornia		Los Angeles County 0.451017	
43	56	New York Kings County cluster_56	New
York		Kings County 0.426715	
44	57	Georgia Bibb County cluster_23	Ge
orgia		Bibb County 0.422703	
45	58	Kansas Wyandotte County cluster_31	K
ansas		Wyandotte County 0.412471	
46	59	California Los Angeles County cluster_13	Calif
ornia		Los Angeles County 0.408401	
47	60	Texas Harris County cluster_80	
Texas		Harris County 0.402791	
48	61	Texas Harris County cluster_82	
Texas		Harris County 0.399244	
49	62	District of Columbia District of Columbia ... District of Col	
umbia		District of Columbia 0.388750	

50	63	California Los Angeles County cluster_15	Calif
ornia		Los Angeles County	0.363895
51	64	Oklahoma Tulsa County cluster_67	Okl
ahoma		Tulsa County	0.362529
52	65	California Los Angeles County cluster_10	Calif
ornia		Los Angeles County	0.356657
53	66	Alabama Mobile County cluster_2	Al
abama		Mobile County	0.333314
54	67	Florida Palm Beach County cluster_21	Fl
orida		Palm Beach County	0.281831

Saved:

- data\processed\healthcare_metric_directions.csv
- data\processed\healthcare_recovery_panel.parquet
- data\processed\healthcare_recovery_coverage.parquet
- data\processed\healthcare_scored_yearly.parquet
- data\processed\healthcare_cluster_scored.parquet
- data\processed\cluster_shortlist_with_healthcare.parquet
- data\processed\healthcare_comparison_set.parquet
- data\processed\cluster_shortlist_with_healthcare.csv
- data\processed\healthcare_comparison_set.csv

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\1322174038.py:277: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
cluster_shortlist_with_healthcare["healthcare_eligible_flag"].fillna(False)
```

Cell 9A — first storytelling visual:

```
In [57]: # Cell 9A – first storytelling visual:
# ranked comparison of the healthcare-covered finalist clusters

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# A. Validate inputs
# -----
if "healthcare_comparison_set" not in globals() or len(healthcare_comparison_set) == 0:
    raise ValueError(
        "healthcare_comparison_set is empty or missing. Run Cell 8 first."
    )

# -----
# B. Build a compact plotting table
# -----
storytelling_overview = (
    healthcare_comparison_set[
        [
            "healthcare_comparison_rank",
            "rank",
            "cluster_id",
```

```

        "display_state",
        "display_county",
        "n_cluster_tracts",
        "latest_cluster_pop_total",
        "cluster_shortlist_score",
        "healthcare_need_score",
        "preliminary_combined_score",
    ]
]
.copy()
.sort_values(
    ["preliminary_combined_score", "healthcare_need_score", "cluster_sho
    ascending=[False, False, False]
)
.reset_index(drop=True)
)

storytelling_overview["story_rank"] = np.arange(1, len(storytelling_overview

storytelling_overview["cluster_label"] = (
    storytelling_overview["display_county"].astype(str)
    + ", "
    + storytelling_overview["display_state"].astype(str)
)

# shorter label if you want cluster number visible too
storytelling_overview["cluster_label_short"] = (
    storytelling_overview["cluster_label"]
    + " | "
    + storytelling_overview["cluster_id"].astype(str).str.extract(r"(cluster
)

# -----
# C. Show the table you will use for storytelling
# -----
storytelling_view_cols = [
    "story_rank",
    "healthcare_comparison_rank",
    "rank",
    "cluster_label_short",
    "n_cluster_tracts",
    "latest_cluster_pop_total",
    "cluster_shortlist_score",
    "healthcare_need_score",
    "preliminary_combined_score",
]

print("Healthcare-covered finalist overview:")
display(storytelling_overview[storytelling_view_cols])

# -----
# D. Plot the first comparison visual
# -----
plot_df = storytelling_overview.sort_values("preliminary_combined_score", as
y = np.arange(len(plot_df))

```



```

fig, ax = plt.subplots(figsize=(12, max(7, len(plot_df) * 0.55)))

bar_h = 0.22

ax.barh(
    y - bar_h,
    plot_df["cluster_shortlist_score"],
    height=bar_h,
    label="Shortlist score"
)

ax.barh(
    y,
    plot_df["healthcare_need_score"],
    height=bar_h,
    label="Healthcare need score"
)

ax.barh(
    y + bar_h,
    plot_df["preliminary_combined_score"],
    height=bar_h,
    label="Preliminary combined score"
)

ax.set_yticks(y)
ax.set_yticklabels(plot_df["cluster_label_short"])
ax.set_xlabel("Score")
ax.set_ylabel("Cluster")
ax.set_title("Healthcare-Covered Finalist Clusters: Shortlist vs Healthcare")
ax.legend()
ax.grid(axis="x", alpha=0.25)

plt.tight_layout()
plt.show()

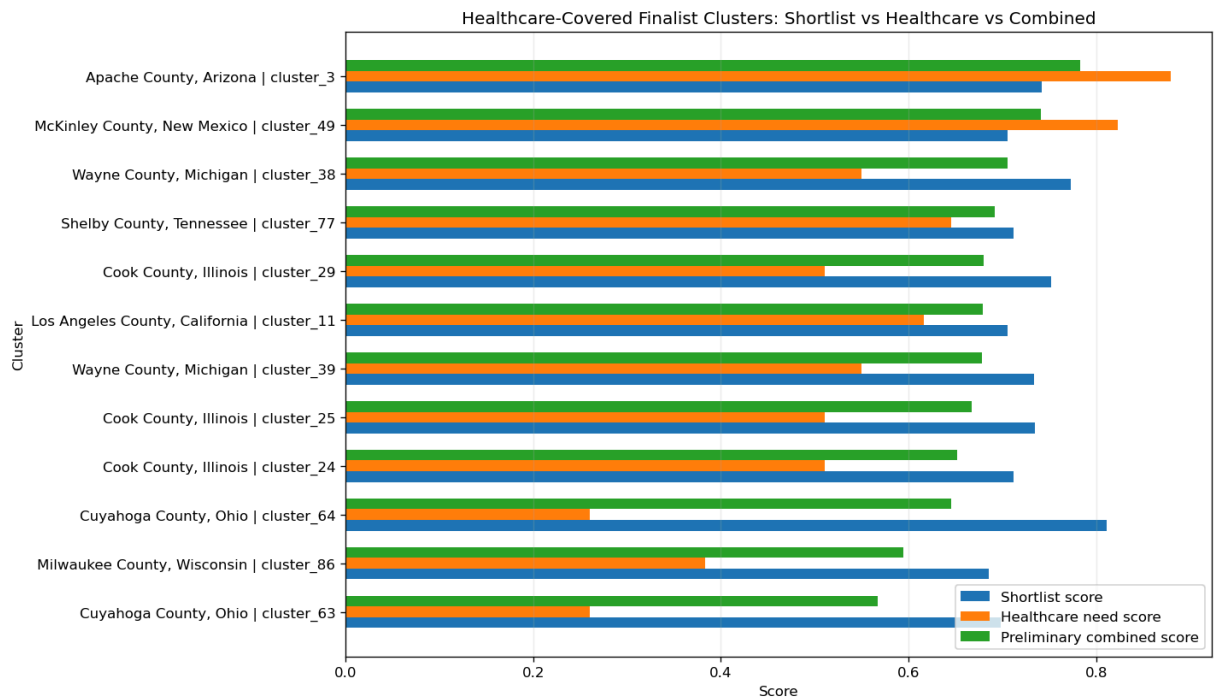
# -----
# E. Optional save for later slide/table use
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)
storytelling_overview.to_csv(OUT_DIR / "storytelling_overview_finalists.csv")

print("\nSaved:")
print("-", OUT_DIR / "storytelling_overview_finalists.csv")

```

Healthcare-covered finalist overview:

	story_rank	healthcare_comparison_rank	rank	cluster_label_short	n_cluster
0	1		1	7	Apache County, Arizona cluster_3
1	2		2	12	McKinley County, New Mexico cluster_49
2	3		3	5	Wayne County, Michigan cluster_38
3	4		4	11	Shelby County, Tennessee cluster_77
4	5		5	6	Cook County, Illinois cluster_29
5	6		6	13	Los Angeles County, California cluster_11
6	7		7	9	Wayne County, Michigan cluster_39
7	8		8	8	Cook County, Illinois cluster_25
8	9		9	10	Cook County, Illinois cluster_24
9	10		10	4	Cuyahoga County, Ohio cluster_64
10	11		11	15	Milwaukee County, Wisconsin cluster_86
11	12		12	14	Cuyahoga County, Ohio cluster_63



Saved:

- data\processed\storytelling_overview_finalists.csv

Cell 9C — national overview map with U.S. basemap + cluster locator dots

```
In [58]: # Cell 9C — national overview map with U.S. basemap + cluster locator dots
# Purpose:
# show where the selected clusters are in the U.S. in a presentation-friendly way

import geopandas as gpd
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import zipfile
import requests
from pathlib import Path

# -----
# A. Validate inputs
# -----
if "adjacency_gdf" not in globals() or len(adjacency_gdf) == 0:
    raise ValueError("adjacency_gdf is missing or empty. Rerun Cell 2 first.")

if "enrichment_clusters" not in globals() or len(enrichment_clusters) == 0:
    raise ValueError("enrichment_clusters is missing or empty. Rerun Cell 6")

# -----
# B. Choose which cluster set to show
# -----
MAP_CLUSTER_SOURCE = "enrichment_clusters"
# MAP_CLUSTER_SOURCE = "healthcare_comparison_set"
```

```

if MAP_CLUSTER_SOURCE == "healthcare_comparison_set":
    if "healthcare_comparison_set" not in globals() or len(healthcare_compar
        raise ValueError("healthcare_comparison_set is missing or empty. Run
    map_clusters_df = healthcare_comparison_set.copy()
else:
    map_clusters_df = enrichment_clusters.copy()

map_cluster_ids = set(map_clusters_df["cluster_id"].astype(str).tolist())

# -----
# C. Build dissolved cluster geometries
# -----
map_gdf = adjacency_gdf.loc[
    adjacency_gdf["cluster_id"].astype(str).isin(map_cluster_ids)
].copy()

if len(map_gdf) == 0:
    raise ValueError("No geometries found for selected clusters.")

map_gdf = gpd.GeoDataFrame(map_gdf, geometry="geometry", crs=adjacency_gdf.crs)

cluster_boundaries = (
    map_gdf[["cluster_id", "geometry"]]
    .dissolve(by="cluster_id", as_index=False)
)

cluster_meta_cols = ["cluster_id", "display_state", "display_county"]
for c in ["enrichment_rank", "rank", "cluster_shortlist_score"]:
    if c in map_clusters_df.columns:
        cluster_meta_cols.append(c)

cluster_meta = (
    map_clusters_df[cluster_meta_cols]
    .drop_duplicates(subset=["cluster_id"])
    .copy()
)

cluster_boundaries = cluster_boundaries.merge(
    cluster_meta,
    on="cluster_id",
    how="left",
    validate="1:1"
)

# healthcare eligibility if available
if "cluster_shortlist_with_healthcare" in globals() and len(cluster_shortlis
    eligibility_df = (
        cluster_shortlist_with_healthcare[["cluster_id", "healthcare_eligibl
        .drop_duplicates(subset=["cluster_id"])
        .copy()
    )
    cluster_boundaries = cluster_boundaries.merge(
        eligibility_df,
        on="cluster_id",
        how="left",
        validate="1:1"

```

```

    )
else:
    cluster_boundaries["healthcare_eligible_flag"] = False

cluster_boundaries["healthcare_eligible_flag"] = (
    cluster_boundaries["healthcare_eligible_flag"].fillna(False).astype(bool)
)

cluster_boundaries["label_point"] = cluster_boundaries.geometry.representati
cluster_boundaries["lon"] = cluster_boundaries["label_point"].x
cluster_boundaries["lat"] = cluster_boundaries["label_point"].y

# -----
# D. Load U.S. state basemap (Census TIGER states)
# -----
STATE_BASE_URL = "https://www2.census.gov/geo/tiger/TIGER2024/STATE/"
STATE_ROOT_DIR = RAW_DIR / "tl_2024_us_state"
STATE_ROOT_DIR.mkdir(parents=True, exist_ok=True)

state_zip_name = "tl_2024_us_state.zip"
state_shp_name = "tl_2024_us_state.shp"
state_zip_path = STATE_ROOT_DIR / state_zip_name
state_extract_dir = STATE_ROOT_DIR / "tl_2024_us_state"
state_shp_path = state_extract_dir / state_shp_name

if not state_shp_path.exists():
    state_extract_dir.mkdir(parents=True, exist_ok=True)

    if not state_zip_path.exists():
        url = f"{STATE_BASE_URL}{state_zip_name}"
        print(f"Downloading {url}")
        r = requests.get(url, timeout=120)
        r.raise_for_status()
        with open(state_zip_path, "wb") as f:
            f.write(r.content)

        print(f"Extracting {state_zip_name}")
        with zipfile.ZipFile(state_zip_path, "r") as zf:
            zf.extractall(state_extract_dir)

states_gdf = gpd.read_file(state_shp_path).to_crs(4326)

# Keep only states / DC / PR, drop territories not needed for this slide
drop_names = {
    "Alaska", "Hawaii", "Guam", "American Samoa",
    "Commonwealth of the Northern Mariana Islands",
    "United States Virgin Islands"
}
states_gdf = states_gdf.loc[~states_gdf["NAME"].isin(drop_names)].copy()

conus_states = states_gdf.loc[states_gdf["NAME"] != "Puerto Rico"].copy()
pr_state = states_gdf.loc[states_gdf["NAME"] == "Puerto Rico"].copy()

# -----
# E. Split points into CONUS vs Puerto Rico
# -----

```

```

conus_points = cluster_boundaries.loc[cluster_boundaries["display_state"] !=
pr_points = cluster_boundaries.loc[cluster_boundaries["display_state"] == "F

# -----
# F. Plot
# -----
fig = plt.figure(figsize=(16, 10))

# Main U.S. panel
ax_main = fig.add_axes([0.04, 0.08, 0.72, 0.84])
conus_states.boundary.plot(ax=ax_main, linewidth=0.8, color="lightgray")

# Healthcare-eligible vs not
conus_yes = conus_points.loc[conus_points["healthcare_eligible_flag"]].copy()
conus_no = conus_points.loc[~conus_points["healthcare_eligible_flag"]].copy()

if len(conus_yes) > 0:
    ax_main.scatter(
        conus_yes["lon"], conus_yes["lat"],
        s=140, zorder=3, edgecolor="black", linewidth=0.8,
        label="Healthcare-eligible"
    )

if len(conus_no) > 0:
    ax_main.scatter(
        conus_no["lon"], conus_no["lat"],
        s=140, zorder=3, edgecolor="black", linewidth=1.0,
        facecolors="white", label="Not healthcare-eligible"
    )

for _, row in conus_points.iterrows():
    label_txt = ""
    if "enrichment_rank" in row and pd.notna(row["enrichment_rank"]):
        label_txt = str(int(row["enrichment_rank"]))

    if label_txt:
        ax_main.text(
            row["lon"], row["lat"], label_txt,
            ha="center", va="center",
            fontsize=8, fontweight="bold",
            zorder=4
        )

ax_main.set_title("Selected Community Clusters – U.S. Overview", fontsize=15)
ax_main.set_xlim(-126, -66)
ax_main.set_ylim(24, 50)
ax_main.set_axis_off()

# Puerto Rico inset
ax_pr = fig.add_axes([0.79, 0.12, 0.16, 0.18])
if len(pr_state) > 0:
    pr_state.boundary.plot(ax=ax_pr, linewidth=0.8, color="lightgray")

pr_yes = pr_points.loc[pr_points["healthcare_eligible_flag"]].copy()
pr_no = pr_points.loc[~pr_points["healthcare_eligible_flag"]].copy()

```

```

if len(pr_yes) > 0:
    ax_pr.scatter(
        pr_yes["lon"], pr_yes["lat"],
        s=120, zorder=3, edgecolor="black", linewidth=0.8
    )

if len(pr_no) > 0:
    ax_pr.scatter(
        pr_no["lon"], pr_no["lat"],
        s=120, zorder=3, edgecolor="black", linewidth=1.0,
        facecolors="white"
    )

for _, row in pr_points.iterrows():
    label_txt = ""
    if "enrichment_rank" in row and pd.notna(row["enrichment_rank"]):
        label_txt = str(int(row["enrichment_rank"]))

    if label_txt:
        ax_pr.text(
            row["lon"], row["lat"], label_txt,
            ha="center", va="center",
            fontsize=8, fontweight="bold",
            zorder=4
        )

if len(pr_state) > 0:
    minx, miny, maxx, maxy = pr_state.total_bounds
    ax_pr.set_xlim(minx - 0.2, maxx + 0.2)
    ax_pr.set_ylim(miny - 0.15, maxy + 0.15)

ax_pr.set_title("Puerto Rico inset", fontsize=11)
ax_pr.set_axis_off()

fig.suptitle("Selected Tract Clusters for Community Comparison", fontsize=18)

# Legend
legend_handles = [
    mpatches.Patch(label="State outlines = U.S. basemap"),
]

if len(conus_yes) + len(pr_yes) > 0:
    legend_handles.append(
        plt.Line2D([0], [0], marker='o', color='w',
                    markerfacecolor='C0', markeredgcolor='black',
                    markersize=10, label='Healthcare-eligible cluster')
    )

if len(conus_no) + len(pr_no) > 0:
    legend_handles.append(
        plt.Line2D([0], [0], marker='o', color='w',
                    markerfacecolor='white', markeredgcolor='black',
                    markersize=10, label='Not healthcare-eligible cluster')
    )

fig.legend(

```

```

        handles=legend_handles,
        loc="lower left",
        bbox_to_anchor=(0.02, 0.02),
        frameon=True
    )

plt.show()

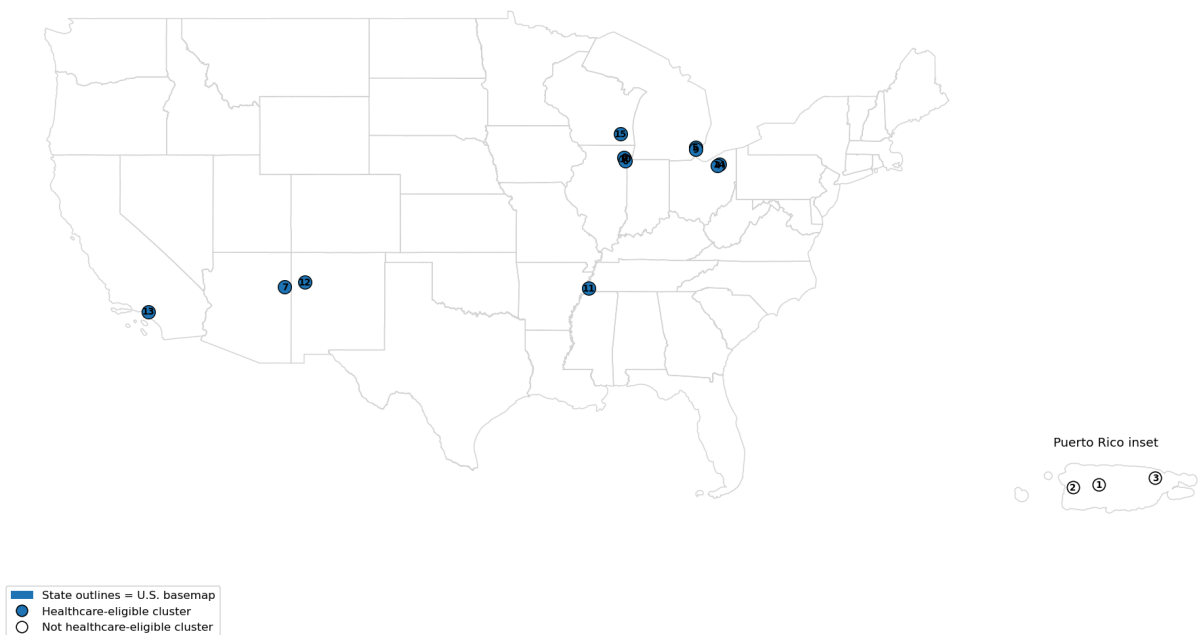
# -----
# G. Printed lookup table
# -----
lookup_cols = [
    c for c in [
        "enrichment_rank",
        "cluster_id",
        "display_state",
        "display_county",
        "healthcare_eligible_flag",
        "cluster_shortlist_score"
    ]
    if c in cluster_boundaries.columns
]

print("Mapped cluster lookup:")
display(
    cluster_boundaries[lookup_cols]
    .sort_values(
        ["enrichment_rank", "cluster_id"] if "enrichment_rank" in cluster_bco
    )
    .reset_index(drop=True)
)

```

Selected Tract Clusters for Community Comparison

Selected Community Clusters — U.S. Overview



Mapped cluster lookup:

enrichment_rank	cluster_id	display_state	display_county	healthcare_eligib
0	1	Puerto Rico Utuaao Municipio cluster_75	Puerto Rico	Utuaao Municipio
1	2	Puerto Rico Mayagüez Municipio cluster_73	Puerto Rico	Mayagüez Municipio
2	3	Puerto Rico Río Grande Municipio cluster_74	Puerto Rico	Río Grande Municipio
3	4	Ohio Cuyahoga County cluster_64	Ohio	Cuyahoga County
4	5	Michigan Wayne County cluster_38	Michigan	Wayne County
5	6	Illinois Cook County cluster_29	Illinois	Cook County
6	7	Arizona Apache County cluster_3	Arizona	Apache County
7	8	Illinois Cook County cluster_25	Illinois	Cook County
8	9	Michigan Wayne County cluster_39	Michigan	Wayne County
9	10	Illinois Cook County cluster_24	Illinois	Cook County
10	11	Tennessee Shelby County cluster_77	Tennessee	Shelby County

	enrichment_rank	cluster_id	display_state	display_county	healthcare_eligib
11	12	New Mexico McKinley County cluster_49	New Mexico	McKinley County	
12	13	California Los Angeles County cluster_11	California	Los Angeles County	
13	14	Ohio Cuyahoga County cluster_63	Ohio	Cuyahoga County	
14	15	Wisconsin Milwaukee County cluster_86	Wisconsin	Milwaukee County	

Cell 9D — ranked score comparison chart for healthcare-covered finalist clusters

```
In [59]: # Cell 9D (vertical version) – ranked score comparison chart

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# A. Validate inputs
# -----
if "healthcare_comparison_set" not in globals() or len(healthcare_comparison_set) == 0:
    raise ValueError(
        "healthcare_comparison_set is empty or missing. Run Cell 8 first."
    )

# -----
# B. Build ranked plotting table
# -----
ranked_score_df = (
    healthcare_comparison_set[
        [
            "healthcare_comparison_rank",
            "rank",
            "cluster_id",
            "display_state",
            "display_county",
            "n_cluster_tracts",
            "latest_cluster_pop_total",
            "cluster_shortlist_score",
            "healthcare_need_score",
        ]
    ]
)
```

```

        "preliminary_combined_score",
    ]
]
    .copy()
    .sort_values(
        ["preliminary_combined_score", "healthcare_need_score", "cluster_sho
        ascending=[False, False, False]
    )
    .reset_index(drop=True)
)

ranked_score_df["final_rank"] = np.arange(1, len(ranked_score_df) + 1)

ranked_score_df["cluster_label"] = (
    "#" + ranked_score_df["final_rank"].astype(str)
    + "\n"
    + ranked_score_df["display_county"].astype(str)
)

print("Ranked healthcare-covered comparison set:")
display(
    ranked_score_df[
        [
            "final_rank",
            "rank",
            "healthcare_comparison_rank",
            "display_county",
            "display_state",
            "n_cluster_tracts",
            "latest_cluster_pop_total",
            "cluster_shortlist_score",
            "healthcare_need_score",
            "preliminary_combined_score",
        ]
    ]
)

# -----
# C. Plot ranked vertical grouped bar chart
# -----

plot_df = ranked_score_df.copy()
x = np.arange(len(plot_df))
bar_w = 0.24

fig, ax = plt.subplots(figsize=(15, 8))

ax.bar(
    x - bar_w,
    plot_df["cluster_shortlist_score"],
    width=bar_w,
    label="Shortlist score"
)

ax.bar(
    x,
    plot_df["healthcare_need_score"],

```

```

        width=bar_w,
        label="Healthcare need score"
    )

    ax.bar(
        x + bar_w,
        plot_df["preliminary_combined_score"],
        width=bar_w,
        label="Combined score"
    )

    # rank labels above combined bars
    for xi, val, rank in zip(x, plot_df["preliminary_combined_score"], plot_df["
        ax.text(xi + bar_w, val + 0.01, f"#{rank}", ha="center", va="bottom", fc

    ax.set_xticks(x)
    ax.set_xticklabels(plot_df["cluster_label"], rotation=35, ha="right")
    ax.set_ylabel("Score")
    ax.set_xlabel("Community cluster")
    ax.set_title("Ranked Comparison of Healthcare-Covered Finalist Clusters")
    ax.grid(axis="y", alpha=0.25)
    ax.legend(frameon=True, loc="upper right")

    ymax = max(
        plot_df["cluster_shortlist_score"].max(),
        plot_df["healthcare_need_score"].max(),
        plot_df["preliminary_combined_score"].max()
    )
    ax.set_ylim(0, ymax + 0.12)

    plt.tight_layout()
    plt.show()

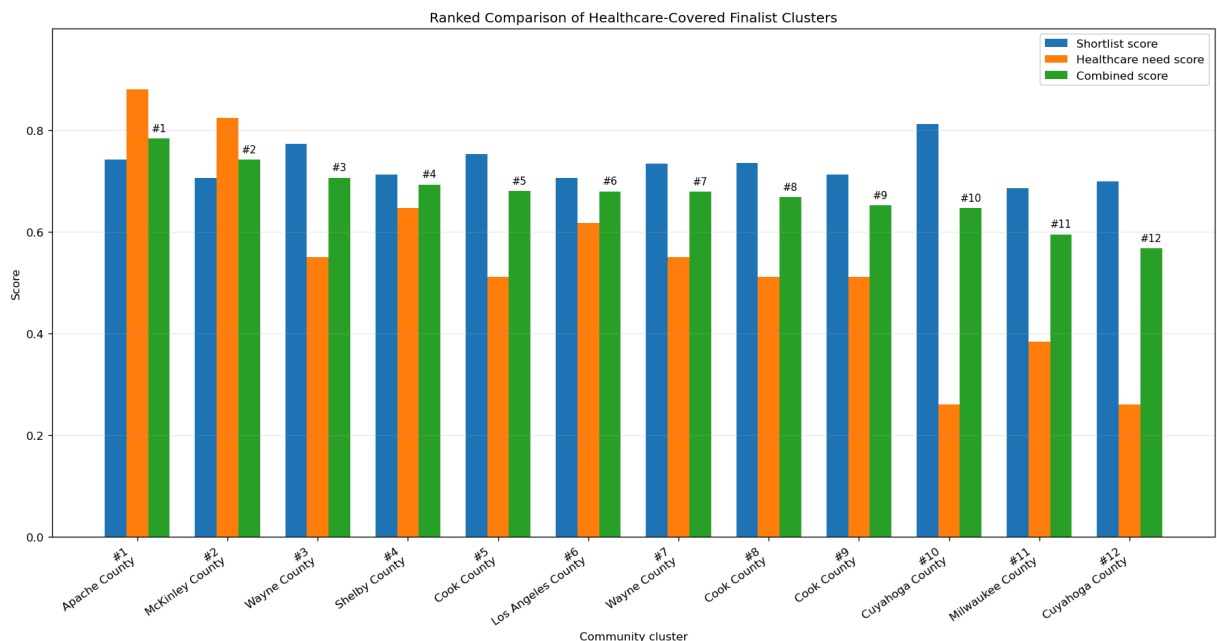
    # -----
    # D. Optional save
    # -----
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    ranked_score_df.to_csv(OUT_DIR / "ranked_score_comparison_chart_data.csv", i

    print("\nSaved:")
    print("-", OUT_DIR / "ranked_score_comparison_chart_data.csv")

```

Ranked healthcare-covered comparison set:

	final_rank	rank	healthcare_comparison_rank	display_county	display_state
0	1	7	1	Apache County	Arizona
1	2	12	2	McKinley County	New Mexico
2	3	5	3	Wayne County	Michigan
3	4	11	4	Shelby County	Tennessee
4	5	6	5	Cook County	Illinois
5	6	13	6	Los Angeles County	California
6	7	9	7	Wayne County	Michigan
7	8	8	8	Cook County	Illinois
8	9	10	9	Cook County	Illinois
9	10	4	10	Cuyahoga County	Ohio
10	11	15	11	Milwaukee County	Wisconsin
11	12	14	12	Cuyahoga County	Ohio



Saved:

- data\processed\ranked_score_comparison_chart_data.csv

Cell 9E — small-multiples trend view for finalist clusters

```
In [60]: # Cell 9E (refined) – top-4 finalist trend view with direct line-end labels
# Changes:
# - only top 4 finalists
# - no legend
```

```

# - direct labels at the recovery end of each line
# - cleaner, more slide-ready layout

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# A. Validate inputs
# -----
if "healthcare_comparison_set" not in globals() or len(healthcare_comparison_set) == 0:
    raise ValueError("healthcare_comparison_set is empty or missing. Run Cell 3")

if "cluster_period_summary" not in globals() or len(cluster_period_summary) == 0:
    raise ValueError("cluster_period_summary is empty or missing. Run Cell 4")

# -----
# B. Settings
# -----
N_FINALISTS_FOR_TRENDS = min(4, len(healthcare_comparison_set))

PERIOD_ORDER = ["pre_covid", "covid", "recovery"]
PERIOD_LABELS = ["Pre-COVID", "COVID", "Recovery"]

trend_metric_map = {
    "igs_total": "IGS Total",
    "igs_economy": "IGS Economy",
    "poverty_rate": "Poverty Rate",
    "unemp_rate": "Unemployment Rate",
    "median_household_income": "Median Income",
}

# Shorter labels for direct annotation
def short_cluster_label(rank, county, state):
    state_short = {
        "Arizona": "AZ",
        "New Mexico": "NM",
        "Michigan": "MI",
        "Tennessee": "TN",
        "Illinois": "IL",
        "California": "CA",
        "Ohio": "OH",
        "Wisconsin": "WI",
    }.get(state, state)
    return f"#{int(rank)} {county}, {state_short}"

# -----
# C. Select top finalists
# -----
finalists_for_trends = (
    healthcare_comparison_set
    .sort_values(
        ["preliminary_combined_score", "healthcare_need_score", "cluster_score"],
        ascending=[False, False, False]
    )
    .head(N_FINALISTS_FOR_TRENDS)

```

```

        .copy()
        .reset_index(drop=True)
    )

finalists_for_trends["final_trend_rank"] = np.arange(1, len(finalists_for_trends))
finalists_for_trends["cluster_label"] = finalists_for_trends.apply(
    lambda r: short_cluster_label(r["final_trend_rank"], r["display_county"]),
    axis=1
)

print("Top finalists used in the trend view:")
display(
    finalists_for_trends[
        [
            "final_trend_rank",
            "display_county",
            "display_state",
            "cluster_shortlist_score",
            "healthcare_need_score",
            "preliminary_combined_score",
        ]
    ]
)

# -----
# D. Build long table
# -----

trend_rows = []

for _, finalist in finalists_for_trends.iterrows():
    cluster_id = finalist["cluster_id"]
    cluster_label = finalist["cluster_label"]

    row = cluster_period_summary.loc[cluster_id]
    if len(row) == 0:
        continue

    row = row.iloc[0]

    for metric, metric_label in trend_metric_map.items():
        vals = []
        for period in PERIOD_ORDER:
            col = f"{metric}_{period}"
            vals.append(pd.to_numeric(row.get(col, np.nan), errors="coerce"))

        for i, period in enumerate(PERIOD_ORDER):
            trend_rows.append({
                "cluster_id": cluster_id,
                "cluster_label": cluster_label,
                "metric": metric,
                "metric_label": metric_label,
                "period": period,
                "period_label": PERIOD_LABELS[i],
                "value": vals[i],
            })

```

```

trend_long_df = pd.DataFrame(trend_rows)

if len(trend_long_df) == 0:
    raise ValueError("No trend data was built for plotting.")

# -----
# E. Plot: one panel per metric, finalists as lines
# -----
fig, axes = plt.subplots(1, 5, figsize=(18, 5), squeeze=False)
axes = axes[0]
x = np.arange(len(PERIOD_ORDER))

for ax, (metric, metric_label) in zip(axes, trend_metric_map.items()):
    sub = trend_long_df.loc[trend_long_df["metric"] == metric].copy()

    # plot each finalist
    line_data = []
    for cluster_label, grp in sub.groupby("cluster_label", sort=False):
        grp = grp.set_index("period").reindex(PERIOD_ORDER).reset_index()

        line, = ax.plot(
            x,
            grp["value"],
            marker="o",
            linewidth=2
        )

        # store for direct labeling
        y_end = grp.loc[grp["period"] == "recovery", "value"].iloc[0]
        line_data.append({
            "cluster_label": cluster_label,
            "y_end": y_end,
            "color": line.get_color(),
        })

    # direct labels at the recovery end
    # sort by end value so labels are less likely to collide
    line_data = sorted(line_data, key=lambda d: d["y_end"])

    # simple vertical nudging if labels are too close
    adjusted_y = []
    min_gap = 0.03 * (sub["value"].max() - sub["value"].min()) if sub["value"].nunique() > 1 else 0
    if min_gap == 0:
        min_gap = 0.01

    for item in line_data:
        y = item["y_end"]
        if adjusted_y and (y - adjusted_y[-1] < min_gap):
            y = adjusted_y[-1] + min_gap
        adjusted_y.append(y)
        item["y_label"] = y

    for item in line_data:
        ax.text(
            2.08,
            item["y_label"],

```



```

        item["cluster_label"],
        color=item["color"],
        fontsize=8,
        va="center"
    )

    ax.set_title(metric_label, fontsize=11)
    ax.set_xticks(x)
    ax.set_xticklabels(PERIOD_LABELS, rotation=0)
    ax.grid(axis="y", alpha=0.25)
    ax.set_xlim(-0.1, 2.95)

    if metric == "median_household_income":
        ax.ticklabel_format(style="plain", axis="y")
        ax.get_yaxis().set_major_formatter(
            plt.FuncFormatter(lambda v, pos: f"{int(v):,}")
        )

axes[0].set_ylabel("Value")

fig.suptitle(
    "Top 4 Finalist Cluster Trends Across Pre-COVID, COVID, and Recovery",
    fontsize=16,
    y=1.03
)

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()

# -----
# F. Optional save
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)
trend_long_df.to_csv(OUT_DIR / "top4_finalist_metric_first_trends.csv", index=False)

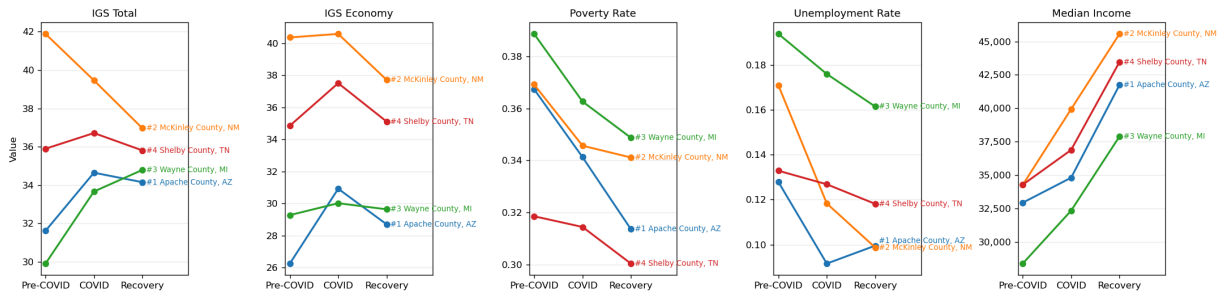
print("Saved:")
print("-", OUT_DIR / "top4_finalist_metric_first_trends.csv")

```

Top finalists used in the trend view:

	final_trend_rank	display_county	display_state	cluster_shortlist_score	health_index
0	1	Apache County	Arizona	0.742122	
1	2	McKinley County	New Mexico	0.705959	
2	3	Wayne County	Michigan	0.772703	
3	4	Shelby County	Tennessee	0.712122	

Top 4 Finalist Cluster Trends Across Pre-COVID, COVID, and Recovery



Saved:

- data\processed\top4_finalist_metric_first_trends.csv

Cell 9F — healthcare access comparison heatmap across finalist clusters

```
In [61]: # Cell 9F – healthcare access comparison heatmap across finalist clusters
# FIXED:
# avoid pandas index-alignment NaNs by assigning numpy arrays directly

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# A. Validate inputs
# -----
if "healthcare_cluster_scored" not in globals() or len(healthcare_cluster_scored) == 0:
    raise ValueError("healthcare_cluster_scored is empty or missing. Run Cell 9E")

if "healthcare_comparison_set" not in globals() or len(healthcare_comparison_set) == 0:
    raise ValueError("healthcare_comparison_set is empty or missing. Run Cell 9E")

# -----
# B. Metric setup
# higher = worse access / higher need
# -----
healthcare_access_metrics = [
    "uninsured_rate",
    "uninsured_adults_rate",
    "uninsured_children_rate",
    "primary_care_physicians_rate",
    "dentists_rate",
    "mental_health_providers_rate",
    "other_primary_care_providers_rate",
    "preventable_hospital_stays_rate",
    "mammography_screening_rate",
    "flu_vaccinations_rate",
]

metric_label_map = {
    "uninsured_rate": "Uninsured",
    "uninsured_adults_rate": "Uninsured Adults",
    "uninsured_children_rate": "Uninsured Children",
```

```

    "primary_care_physicians_rate": "Primary Care Access",
    "dentists_rate": "Dental Access",
    "mental_health_providers_rate": "Mental Health Access",
    "other_primary_care_providers_rate": "Other PCP Access",
    "preventable_hospital_stays_rate": "Preventable Hospital Stays",
    "mammography_screening_rate": "Mammography Screening",
    "flu_vaccinations_rate": "Flu Vaccinations",
}

high_is_bad = {
    "uninsured_rate",
    "uninsured_adults_rate",
    "uninsured_children_rate",
    "preventable_hospital_stays_rate",
}

low_is_bad = {
    "primary_care_physicians_rate",
    "dentists_rate",
    "mental_health_providers_rate",
    "other_primary_care_providers_rate",
    "mammography_screening_rate",
    "flu_vaccinations_rate",
}

# -----
# C. Build finalist healthcare table
# -----
heatmap_df = (
    healthcare_cluster_scored
    .merge(
        healthcare_comparison_set[
            ["cluster_id", "healthcare_comparison_rank", "preliminary_combin
        ].drop_duplicates(),
        on="cluster_id",
        how="inner",
        validate="1:1"
    )
    .copy()
)

heatmap_df = heatmap_df.sort_values(
    ["healthcare_comparison_rank", "healthcare_need_score"],
    ascending=[True, False]
).reset_index(drop=True)

heatmap_df["cluster_label"] = (
    "#" + heatmap_df["healthcare_comparison_rank"].astype(int).astype(str)
    + " - "
    + heatmap_df["display_county"].astype(str)
    + ", "
    + heatmap_df["display_state"].astype(str)
)

print("Preview of healthcare values used in the heatmap:")
display(

```

```

heatmap_df[
    ["cluster_label", "healthcare_need_score"] + healthcare_access_metrics
].head(12)
)

# -----
# D. Convert all metrics so higher = worse access / higher need
# then percentile-rank them across finalists
# -----
oriented = pd.DataFrame(index=heatmap_df["cluster_label"].tolist())

for metric in healthcare_access_metrics:
    vals = pd.to_numeric(heatmap_df[metric], errors="coerce")

    if metric in high_is_bad:
        ranked = vals.rank(pct=True, method="average")
    elif metric in low_is_bad:
        ranked = (-vals).rank(pct=True, method="average")
    else:
        ranked = pd.Series(np.nan, index=vals.index)

    # IMPORTANT FIX: assign raw values, not index-aligned series
    oriented[metric_label_map[metric]] = ranked.to_numpy()

# keep healthcare need score as summary column
oriented["Overall Healthcare Need"] = pd.to_numeric(
    heatmap_df["healthcare_need_score"], errors="coerce"
).to_numpy()

# move summary column first
ordered_cols = ["Overall Healthcare Need"] + [
    c for c in oriented.columns if c != "Overall Healthcare Need"
]
oriented = oriented[ordered_cols]

# -----
# E. Plot heatmap
# -----
fig, ax = plt.subplots(figsize=(13, max(6, len(oriented) * 0.55)))

im = ax.imshow(oriented.values, aspect="auto")

ax.set_xticks(np.arange(oriented.shape[1]))
ax.set_xticklabels(oriented.columns, rotation=45, ha="right")

ax.set_yticks(np.arange(oriented.shape[0]))
ax.set_yticklabels(oriented.index)

ax.set_title("Healthcare Access / Need Comparison Across Finalist Clusters")
ax.set_xlabel("Healthcare access metrics (higher = worse access / higher need)")
ax.set_ylabel("Finalist cluster")

cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Relative need intensity")

plt.tight_layout()

```

```
plt.show()

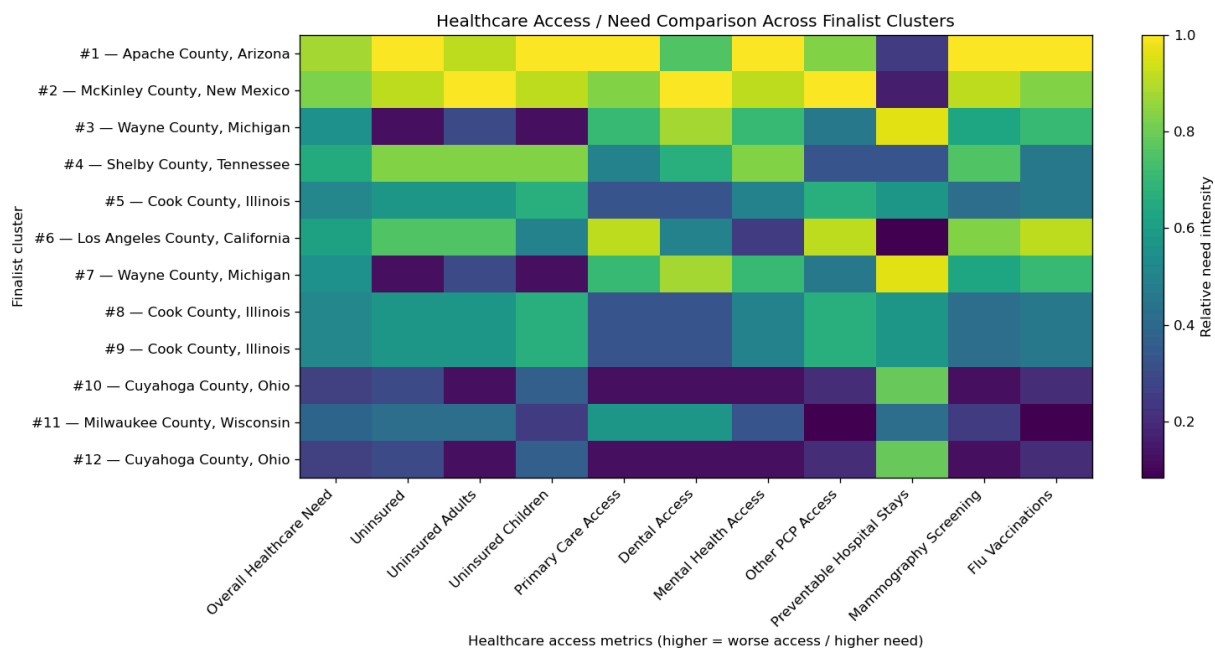
# -----
# F. Printed table
# -----
print("Healthcare access comparison table (relative scores):")
display(oriented.round(3))

# -----
# G. Optional save
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)
oriented.to_csv(OUT_DIR / "healthcare_access_heatmap_table.csv")

print("\nSaved:")
print("-", OUT_DIR / "healthcare_access_heatmap_table.csv")
```

Preview of healthcare values used in the heatmap:

	cluster_label	healthcare_need_score	uninsured_rate	uninsured_adults_rate
0	#1 — Apache County, Arizona	0.879167	0.198031	0.230315
1	#2 — McKinley County, New Mexico	0.822917	0.188087	0.241622
2	#3 — Wayne County, Michigan	0.550000	0.070757	0.087738
3	#4 — Shelby County, Tennessee	0.645833	0.131008	0.161872
4	#5 — Cook County, Illinois	0.510417	0.105727	0.129626
5	#6 — Los Angeles County, California	0.616667	0.106480	0.130229
6	#7 — Wayne County, Michigan	0.550000	0.070757	0.087738
7	#8 — Cook County, Illinois	0.510417	0.105727	0.129626
8	#9 — Cook County, Illinois	0.510417	0.105727	0.129626
9	#10 — Cuyahoga County, Ohio	0.260417	0.073992	0.087338
10	#11 — Milwaukee County, Wisconsin	0.383333	0.084279	0.103360
11	#12 — Cuyahoga County, Ohio	0.260417	0.073992	0.087338



Healthcare access comparison table (relative scores):

	Overall Healthcare Need	Uninsured	Uninsured Adults	Uninsured Children	Primary Care Access	Dental Access	Me He Acc
#1 — Apache County, Arizona	0.879	1.000	0.917	1.000	1.000	0.750	1
#2 — McKinley County, New Mexico	0.823	0.917	1.000	0.917	0.833	1.000	0
#3 — Wayne County, Michigan	0.550	0.125	0.292	0.125	0.708	0.875	0
#4 — Shelby County, Tennessee	0.646	0.833	0.833	0.833	0.500	0.667	0
#5 — Cook County, Illinois	0.510	0.583	0.583	0.667	0.333	0.333	0
#6 — Los Angeles County, California	0.617	0.750	0.750	0.500	0.917	0.500	0
#7 — Wayne County, Michigan	0.550	0.125	0.292	0.125	0.708	0.875	0
#8 — Cook County, Illinois	0.510	0.583	0.583	0.667	0.333	0.333	0
#9 — Cook County, Illinois	0.510	0.583	0.583	0.667	0.333	0.333	0
#10 — Cuyahoga County, Ohio	0.260	0.292	0.125	0.375	0.125	0.125	0
#11 — Milwaukee County, Wisconsin	0.383	0.417	0.417	0.250	0.583	0.583	0
#12 — Cuyahoga County, Ohio	0.260	0.292	0.125	0.375	0.125	0.125	0

Saved:

- data\processed\healthcare_access_heatmap_table.csv

```
In [62]: # Cell 9F – healthcare access comparison heatmap across finalist clusters
# FIXED:
# avoid pandas index-alignment NaNs by assigning numpy arrays directly

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# A. Validate inputs
# -----
if "healthcare_cluster_scored" not in globals() or len(healthcare_cluster_scored) == 0:
    raise ValueError("healthcare_cluster_scored is empty or missing. Run Cell 9E")

if "healthcare_comparison_set" not in globals() or len(healthcare_comparison_set) == 0:
    raise ValueError("healthcare_comparison_set is empty or missing. Run Cell 9D")

# -----
# B. Metric setup
# higher = worse access / higher need
# -----
healthcare_access_metrics = [
    "uninsured_rate",
    "uninsured_adults_rate",
    "uninsured_children_rate",
    "primary_care_physicians_rate",
    "dentists_rate",
    "mental_health_providers_rate",
    "other_primary_care_providers_rate",
    "preventable_hospital_stays_rate",
    "mammography_screening_rate",
    "flu_vaccinations_rate",
]

metric_label_map = {
    "uninsured_rate": "Uninsured",
    "uninsured_adults_rate": "Uninsured Adults",
    "uninsured_children_rate": "Uninsured Children",
    "primary_care_physicians_rate": "Primary Care Access",
    "dentists_rate": "Dental Access",
    "mental_health_providers_rate": "Mental Health Access",
    "other_primary_care_providers_rate": "Other PCP Access",
    "preventable_hospital_stays_rate": "Preventable Hospital Stays",
    "mammography_screening_rate": "Mammography Screening",
    "flu_vaccinations_rate": "Flu Vaccinations",
}

high_is_bad = {
    "uninsured_rate",
    "uninsured_adults_rate",
    "uninsured_children_rate",
    "preventable_hospital_stays_rate",
}
```

```

low_is_bad = {
    "primary_care_physicians_rate",
    "dentists_rate",
    "mental_health_providers_rate",
    "other_primary_care_providers_rate",
    "mammography_screening_rate",
    "flu_vaccinations_rate",
}

# -----
# C. Build finalist healthcare table
# -----
heatmap_df = (
    healthcare_cluster_scored
    .merge(
        healthcare_comparison_set[
            ["cluster_id", "healthcare_comparison_rank", "preliminary_combi
        ].drop_duplicates(),
        on="cluster_id",
        how="inner",
        validate="1:1"
    )
    .copy()
)

heatmap_df = heatmap_df.sort_values(
    ["healthcare_comparison_rank", "healthcare_need_score"],
    ascending=[True, False]
).reset_index(drop=True)

heatmap_df["cluster_label"] = (
    "#" + heatmap_df["healthcare_comparison_rank"].astype(int).astype(str)
    + " - "
    + heatmap_df["display_county"].astype(str)
    + ", "
    + heatmap_df["display_state"].astype(str)
)

print("Preview of healthcare values used in the heatmap:")
display(
    heatmap_df[
        ["cluster_label", "healthcare_need_score"] + healthcare_access_metri
    ].head(12)
)

# -----
# D. Convert all metrics so higher = worse access / higher need
# then percentile-rank them across finalists
# -----
oriented = pd.DataFrame(index=heatmap_df["cluster_label"].tolist())

for metric in healthcare_access_metrics:
    vals = pd.to_numeric(heatmap_df[metric], errors="coerce")

    if metric in high_is_bad:

```

```

        ranked = vals.rank(pct=True, method="average")
    elif metric in low_is_bad:
        ranked = (-vals).rank(pct=True, method="average")
    else:
        ranked = pd.Series(np.nan, index=vals.index)

    # IMPORTANT FIX: assign raw values, not index-aligned series
    oriented[metric_label_map[metric]] = ranked.to_numpy()

# keep healthcare need score as summary column
oriented["Overall Healthcare Need"] = pd.to_numeric(
    heatmap_df["healthcare_need_score"], errors="coerce"
).to_numpy()

# move summary column first
ordered_cols = ["Overall Healthcare Need"] + [
    c for c in oriented.columns if c != "Overall Healthcare Need"
]
oriented = oriented[ordered_cols]

# -----
# E. Plot heatmap
# -----
fig, ax = plt.subplots(figsize=(13, max(6, len(oriented) * 0.55)))

im = ax.imshow(oriented.values, aspect="auto")

ax.set_xticks(np.arange(oriented.shape[1]))
ax.set_xticklabels(oriented.columns, rotation=45, ha="right")

ax.set_yticks(np.arange(oriented.shape[0]))
ax.set_yticklabels(oriented.index)

ax.set_title("Healthcare Access / Need Comparison Across Finalist Clusters")
ax.set_xlabel("Healthcare access metrics (higher = worse access / higher need)")
ax.set_ylabel("Finalist cluster")

cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Relative need intensity")

plt.tight_layout()
plt.show()

# -----
# F. Printed table
# -----
print("Healthcare access comparison table (relative scores):")
display(oriented.round(3))

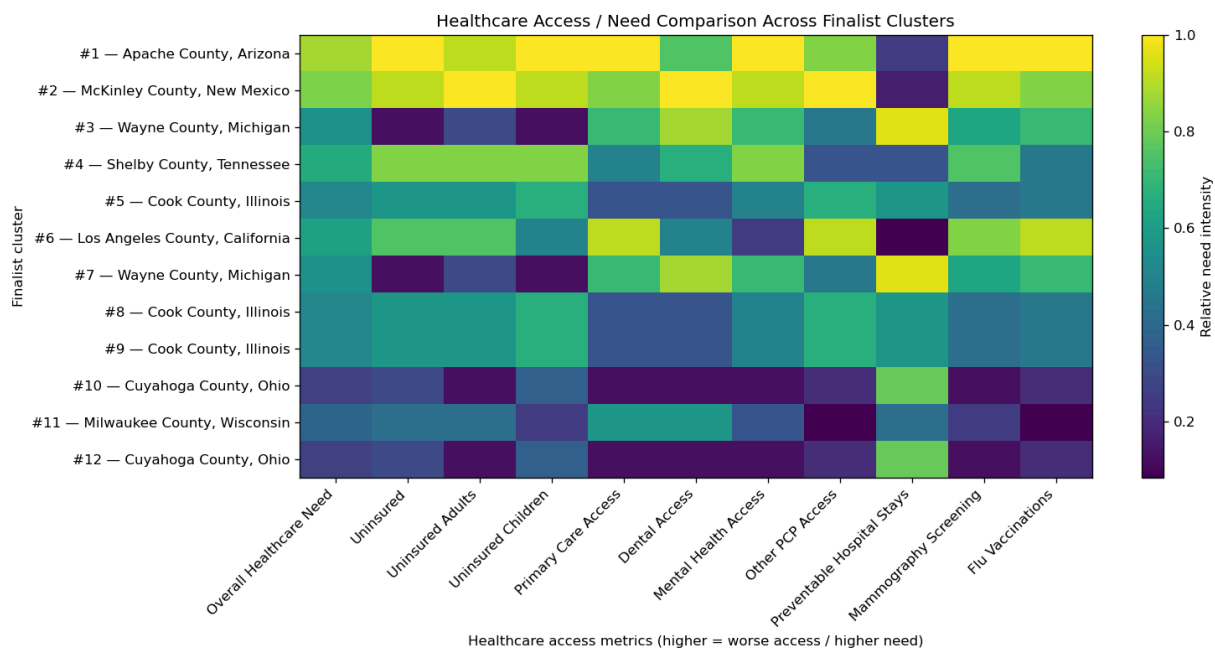
# -----
# G. Optional save
# -----
OUT_DIR.mkdir(parents=True, exist_ok=True)
oriented.to_csv(OUT_DIR / "healthcare_access_heatmap_table.csv")

```

```
print("\nSaved:")
print("-", OUT_DIR / "healthcare_access_heatmap_table.csv")
```

Preview of healthcare values used in the heatmap:

	cluster_label	healthcare_need_score	uninsured_rate	uninsured_adults_rate
0	#1 — Apache County, Arizona	0.879167	0.198031	0.230315
1	#2 — McKinley County, New Mexico	0.822917	0.188087	0.241622
2	#3 — Wayne County, Michigan	0.550000	0.070757	0.087738
3	#4 — Shelby County, Tennessee	0.645833	0.131008	0.161872
4	#5 — Cook County, Illinois	0.510417	0.105727	0.129626
5	#6 — Los Angeles County, California	0.616667	0.106480	0.130229
6	#7 — Wayne County, Michigan	0.550000	0.070757	0.087738
7	#8 — Cook County, Illinois	0.510417	0.105727	0.129626
8	#9 — Cook County, Illinois	0.510417	0.105727	0.129626
9	#10 — Cuyahoga County, Ohio	0.260417	0.073992	0.087338
10	#11 — Milwaukee County, Wisconsin	0.383333	0.084279	0.103360
11	#12 — Cuyahoga County, Ohio	0.260417	0.073992	0.087338



Healthcare access comparison table (relative scores):

	Overall Healthcare Need	Uninsured	Uninsured Adults	Uninsured Children	Primary Care Access	Dental Access	Me He Acc
#1 — Apache County, Arizona	0.879	1.000	0.917	1.000	1.000	0.750	1
#2 — McKinley County, New Mexico	0.823	0.917	1.000	0.917	0.833	1.000	0
#3 — Wayne County, Michigan	0.550	0.125	0.292	0.125	0.708	0.875	0
#4 — Shelby County, Tennessee	0.646	0.833	0.833	0.833	0.500	0.667	0
#5 — Cook County, Illinois	0.510	0.583	0.583	0.667	0.333	0.333	0
#6 — Los Angeles County, California	0.617	0.750	0.750	0.500	0.917	0.500	0
#7 — Wayne County, Michigan	0.550	0.125	0.292	0.125	0.708	0.875	0
#8 — Cook County, Illinois	0.510	0.583	0.583	0.667	0.333	0.333	0
#9 — Cook County, Illinois	0.510	0.583	0.583	0.667	0.333	0.333	0
#10 — Cuyahoga County, Ohio	0.260	0.292	0.125	0.375	0.125	0.125	0
#11 — Milwaukee County, Wisconsin	0.383	0.417	0.417	0.250	0.583	0.583	0
#12 — Cuyahoga County, Ohio	0.260	0.292	0.125	0.375	0.125	0.125	0

Saved:

- data\processed\healthcare_access_heatmap_table.csv

```
In [63]: # Cell 10A – shared prep for finalist visuals

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import geopandas as gpd

# -----
# basic checks
# -----
required_globals = [
    "healthcare_comparison_set",
    "healthcare_cluster_scored",
    "cluster_period_summary",
    "analysis_panel",
    "healthcare_standardized",
]
for name in required_globals:
    if name not in globals() or len(globals()[name]) == 0:
        raise ValueError(f"{name} is missing or empty. Rerun the earlier cell")

# -----
# helpers
# -----
def pct_rank_high(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return s.rank(pct=True, method="average")

def pct_rank_low(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return (-s).rank(pct=True, method="average")

def safe_weighted_mean(values: pd.Series, weights: pd.Series):
    values = pd.to_numeric(values, errors="coerce")
    weights = pd.to_numeric(weights, errors="coerce")
    mask = values.notna() & weights.notna() & (weights > 0)
    if mask.sum() == 0:
        valid = values.notna()
        return values.loc[valid].mean() if valid.sum() > 0 else np.nan
    return np.average(values.loc[mask], weights=weights.loc[mask])

STATE_ABBR = {
    "Alabama": "AL", "Alaska": "AK", "Arizona": "AZ", "Arkansas": "AR", "California":
    "Connecticut": "CT", "Delaware": "DE", "District of Columbia": "DC", "Florida":
    "Hawaii": "HI", "Idaho": "ID", "Illinois": "IL", "Indiana": "IN", "Iowa": "IA", "K
    "Kentucky": "KY", "Louisiana": "LA", "Maine": "ME", "Maryland": "MD", "Massachus
    "Michigan": "MI", "Minnesota": "MN", "Mississippi": "MS", "Missouri": "MO", "Mor
    "Nebraska": "NE", "Nevada": "NV", "New Hampshire": "NH", "New Jersey": "NJ", "Ne
    "New York": "NY", "North Carolina": "NC", "North Dakota": "ND", "Ohio": "OH", "C
    "Oregon": "OR", "Pennsylvania": "PA", "Rhode Island": "RI", "South Carolina":
    "Tennessee": "TN", "Texas": "TX", "Utah": "UT", "Vermont": "VT", "Virginia": "VA"
    "West Virginia": "WV", "Wisconsin": "WI", "Wyoming": "WY", "Puerto Rico": "PR"
}
```

```

TOP_FINALISTS_N = min(4, len(healthcare_comparison_set))

# -----
# top finalists
# -----
finalist_eval_df = (
    healthcare_comparison_set
    .sort_values(
        ["preliminary_combined_score", "healthcare_need_score", "cluster_score"],
        ascending=[False, False, False]
    )
    .head(TOP_FINALISTS_N)
    .copy()
    .reset_index(drop=True)
)

finalist_eval_df["final_rank"] = np.arange(1, len(finalist_eval_df) + 1)
finalist_eval_df["cluster_label"] = (
    "#" + finalist_eval_df["final_rank"].astype(str)
    + " - "
    + finalist_eval_df["display_county"].astype(str)
    + ", "
    + finalist_eval_df["display_state"].astype(str)
)

# attach detailed healthcare metrics
health_cols = [
    "cluster_id",
    "uninsured_rate",
    "uninsured_adults_rate",
    "uninsured_children_rate",
    "primary_care_physicians_rate",
    "dentists_rate",
    "mental_health_providers_rate",
    "other_primary_care_providers_rate",
    "preventable_hospital_stays_rate",
    "mammography_screening_rate",
    "flu_vaccinations_rate",
]
finalist_eval_df = finalist_eval_df.merge(
    healthcare_cluster_scored[health_cols],
    on="cluster_id",
    how="left",
    validate="1:1"
)

# -----
# healthcare composites
# -----
all_health = healthcare_cluster_scored.copy()

uninsured_burden_all = (
    pct_rank_high(all_health["uninsured_rate"]) +
    pct_rank_high(all_health["uninsured_adults_rate"]) +
    pct_rank_high(all_health["uninsured_children_rate"])
)

```



```

) / 3

provider_gap_all = (
    pct_rank_low(all_health["primary_care_physicians_rate"]) +
    pct_rank_low(all_health["dentists_rate"]) +
    pct_rank_low(all_health["mental_health_providers_rate"]) +
    pct_rank_low(all_health["other_primary_care_providers_rate"])
) / 4

preventive_gap_all = (
    pct_rank_low(all_health["mammography_screening_rate"]) +
    pct_rank_low(all_health["flu_vaccinations_rate"])
) / 2

preventable_stays_all = pct_rank_high(all_health["preventable_hospital_stays"])

health_composites = pd.DataFrame({
    "cluster_id": all_health["cluster_id"],
    "uninsured_burden_score": uninsured_burden_all,
    "provider_access_gap_score": provider_gap_all,
    "preventive_care_gap_score": preventive_gap_all,
    "preventable_stays_score": preventable_stays_all,
})

finalist_eval_df = finalist_eval_df.merge(
    health_composites,
    on="cluster_id",
    how="left",
    validate="1:1"
)

# -----
# recovery-period economic context
# -----
recovery_cols = [
    "cluster_id",
    "igs_total_recovery",
    "igs_economy_recovery",
    "poverty_rate_recovery",
    "unemp_rate_recovery",
    "median_household_income_recovery",
]

finalist_eval_df = finalist_eval_df.merge(
    cluster_period_summary[recovery_cols],
    on="cluster_id",
    how="left",
    validate="1:1"
)

print("Prepared finalist_eval_df:")
display(
    finalist_eval_df[
        [
            "final_rank", "cluster_label",
            "cluster_shortlist_score", "healthcare_need_score", "preliminary",
            "uninsured_burden_score", "provider_access_gap_score",

```

```

        "preventive_care_gap_score", "preventable_stays_score"
    ]
]
)

```

Prepared finalist_eval_df:

	final_rank	cluster_label	cluster_shortlist_score	healthcare_need_score	preli
0	1	#1 — Apache County, Arizona	0.742122	0.879167	
1	2	#2 — McKinley County, New Mexico	0.705959	0.822917	
2	3	#3 — Wayne County, Michigan	0.772703	0.550000	
3	4	#4 — Shelby County, Tennessee	0.712122	0.645833	

Cell 10B — finalist decision matrix

In [64]: *# Cell 10B — finalist decision matrix*

```

matrix_cols = {
    "score_low_igs_recovery": "Low IGS",
    "score_econ_vulnerability_recovery": "Economic Vulnerability",
    "cluster_shortlist_score": "Shortlist Score",
    "healthcare_need_score": "Healthcare Need",
    "uninsured_burden_score": "Uninsured Burden",
    "provider_access_gap_score": "Provider Access Gap",
    "preventive_care_gap_score": "Preventive Care Gap",
    "preventable_stays_score": "Preventable Stays",
    "preliminary_combined_score": "Combined Score",
}

decision_matrix = (
    finalist_eval_df[
        ["cluster_label"] + list(matrix_cols.keys())
    ]
    .rename(columns=matrix_cols)
    .set_index("cluster_label")
    .copy()
)

fig, ax = plt.subplots(figsize=(12, max(4.5, len(decision_matrix) * 0.9)))
im = ax.imshow(decision_matrix.values, aspect="auto")

ax.set_xticks(np.arange(decision_matrix.shape[1]))
ax.set_xticklabels(decision_matrix.columns, rotation=40, ha="right")
ax.set_yticks(np.arange(decision_matrix.shape[0]))

```

```

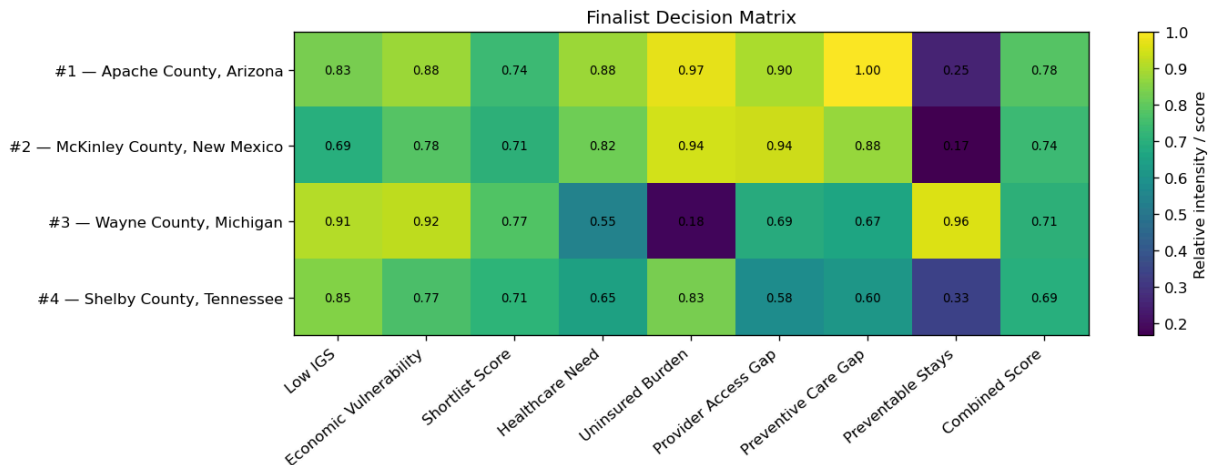
ax.set_yticklabels(decision_matrix.index)

for i in range(decision_matrix.shape[0]):
    for j in range(decision_matrix.shape[1]):
        val = decision_matrix.iloc[i, j]
        ax.text(j, i, f"{val:.2f}", ha="center", va="center", fontsize=8)

ax.set_title("Finalist Decision Matrix")
cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Relative intensity / score")

plt.tight_layout()
plt.show()

```



Cell 10D — healthcare profile grouped bar chart for the top finalists

In [65]: # Cell 10D — healthcare profile grouped bar chart for the top finalists

```

profile_cols = [
    "uninsured_burden_score",
    "provider_access_gap_score",
    "preventive_care_gap_score",
    "preventable_stays_score",
]

profile_labels = {
    "uninsured_burden_score": "Uninsured Burden",
    "provider_access_gap_score": "Provider Access Gap",
    "preventive_care_gap_score": "Preventive Care Gap",
    "preventable_stays_score": "Preventable Stays",
}

plot_df = finalist_eval_df[["cluster_label"] + profile_cols].copy()

x = np.arange(len(profile_cols))
bar_w = 0.18

fig, ax = plt.subplots(figsize=(14, 7))

```

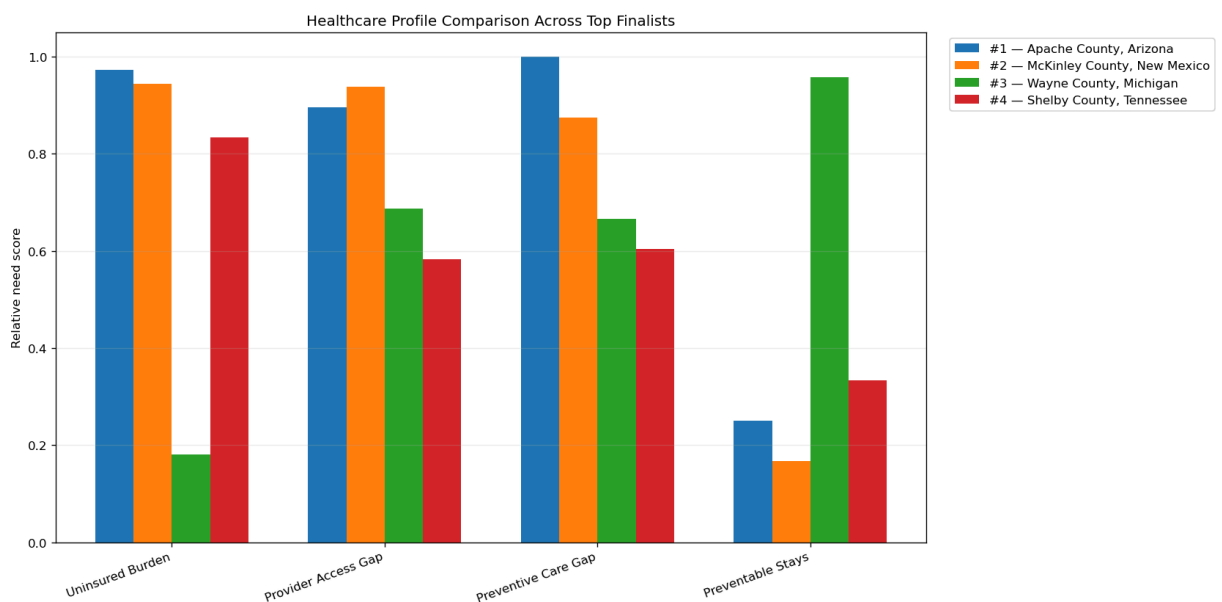
```

for i, (_, row) in enumerate(plot_df.iterrows()):
    ax.bar(
        x + (i - (len(plot_df)-1)/2) * bar_w,
        row[profile_cols].values,
        width=bar_w,
        label=row["cluster_label"]
    )

ax.set_xticks(x)
ax.set_xticklabels([profile_labels[c] for c in profile_cols], rotation=20, h
ax.set_ylabel("Relative need score")
ax.set_title("Healthcare Profile Comparison Across Top Finalists")
ax.grid(axis="y", alpha=0.25)
ax.legend(frameon=True, bbox_to_anchor=(1.02, 1), loc="upper left")

plt.tight_layout()
plt.show()

```



Cell 10E — "Why here?" place-based slide for the top 2 candidates

Cell 10F — provisional solution-fit visual

```

In [66]: # Cell 10F — provisional solution-fit visual
# IMPORTANT:
# this is a proxy-based fit view using current econ + healthcare patterns or
# it should be updated once the business layer is added.

solution_fit = finalist_eval_df[["cluster_label"]].copy()

solution_fit["Mobile / Telehealth Support"] = (
    0.55 * finalist_eval_df["provider_access_gap_score"] +
    0.45 * finalist_eval_df["healthcare_need_score"]
)

```

```

solution_fit["Benefits / Navigation"] = (
    0.60 * finalist_eval_df["uninsured_burden_score"] +
    0.40 * finalist_eval_df["score_econ_vulnerability_recovery"]
)

solution_fit["Preventive Outreach"] = (
    0.60 * finalist_eval_df["preventive_care_gap_score"] +
    0.40 * finalist_eval_df["uninsured_burden_score"]
)

solution_fit["Transportation / Access Support"] = (
    0.50 * finalist_eval_df["provider_access_gap_score"] +
    0.50 * finalist_eval_df["preventable_stays_score"]
)

solution_fit["Health-Support Retail"] = (
    0.50 * finalist_eval_df["preventive_care_gap_score"] +
    0.50 * finalist_eval_df["score_econ_vulnerability_recovery"]
)

solution_fit = solution_fit.set_index("cluster_label")

fig, ax = plt.subplots(figsize=(12, max(4.5, len(solution_fit) * 0.9)))
im = ax.imshow(solution_fit.values, aspect="auto")

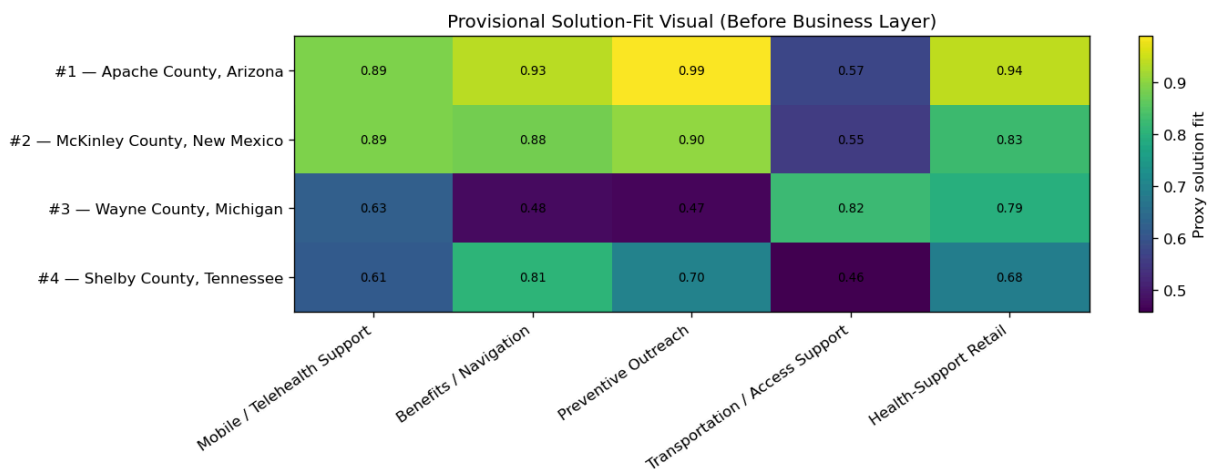
ax.set_xticks(np.arange(solution_fit.shape[1]))
ax.set_xticklabels(solution_fit.columns, rotation=35, ha="right")
ax.set_yticks(np.arange(solution_fit.shape[0]))
ax.set_yticklabels(solution_fit.index)

for i in range(solution_fit.shape[0]):
    for j in range(solution_fit.shape[1]):
        ax.text(j, i, f"{solution_fit.iloc[i, j]:.2f}", ha="center", va="center")

ax.set_title("Provisional Solution-Fit Visual (Before Business Layer)")
cbar = plt.colorbar(im, ax=ax)
cbar.set_label("Proxy solution fit")

plt.tight_layout()
plt.show()

```



```

In [67]: # =====
# SETUP CELL – load processed data + shared helpers
# Run this once before the 4 slide cells below.
# =====

from pathlib import Path
import textwrap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch

plt.rcParams["figure.dpi"] = 120
plt.rcParams["savefig.dpi"] = 160

# -----
# Locate processed directory
# -----
PROC_CANDIDATES = [
    Path("src/data/processed"),
    Path("data/processed"),
    Path.cwd() / "src/data/processed",
    Path.cwd() / "data/processed",
]

PROCESSED_DIR = next((p for p in PROC_CANDIDATES if p.exists()), None)
if PROCESSED_DIR is None:
    raise FileNotFoundError("Could not find src/data/processed or data/processed")

OUT_DIR = PROCESSED_DIR / "presentation_ready"
OUT_DIR.mkdir(parents=True, exist_ok=True)

# -----
# Load core files
# -----
analysis_panel = pd.read_parquet(PROCESSED_DIR / "eda_panel_clean.parquet").
cluster_period_summary = pd.read_parquet(PROCESSED_DIR / "cluster_period_summary.parquet").
healthcare_standardized = pd.read_parquet(PROCESSED_DIR / "healthcare_standardized.parquet").
healthcare_cluster_scored = pd.read_parquet(PROCESSED_DIR / "healthcare_cluster_scored.parquet").

# -----
# Add analysis_period if missing
# -----
def assign_period(year):
    year = int(year)
    if year in [2017, 2018, 2019]:
        return "pre_covid"
    if year in [2020, 2021]:
        return "covid"
    if year in [2022, 2023, 2024]:
        return "recovery"
    return np.nan

if "analysis_period" not in analysis_panel.columns:

```

```

analysis_panel["analysis_period"] = analysis_panel["year"].map(assign_pe

analysis_panel["geoid"] = analysis_panel["geoid"].astype(str).str.zfill(11)
analysis_panel["state_fips"] = analysis_panel["geoid"].str[:2]

healthcare_standardized["county_fips"] = (
    healthcare_standardized["county_fips"].astype(str).str.zfill(5)
)
healthcare_standardized["state_fips"] = healthcare_standardized["county_fips

# -----
# Ensure healthcare rows have labels
# -----
if ("display_state" not in healthcare_cluster_scored.columns) or ("display_c
    healthcare_cluster_scored = healthcare_cluster_scored.merge(
        cluster_period_summary[["cluster_id", "display_state", "display_cour
        on="cluster_id",
        how="left",
        validate="1:1",
    )

# -----
# Helper functions
# -----
def bad_when_high(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return s.rank(pct=True, method="average")

def bad_when_low(series: pd.Series) -> pd.Series:
    s = pd.to_numeric(series, errors="coerce")
    return (-s).rank(pct=True, method="average")

def weighted_avg(df: pd.DataFrame, value_col: str, weight_col: str = "pop_to
    tmp = df[[value_col, weight_col]].copy()
    tmp[value_col] = pd.to_numeric(tmp[value_col], errors="coerce")
    tmp[weight_col] = pd.to_numeric(tmp[weight_col], errors="coerce")
    tmp = tmp.dropna()
    tmp = tmp[tmp[weight_col] > 0]
    if tmp.empty:
        return np.nan
    return np.average(tmp[value_col], weights=tmp[weight_col])

def percentile_of_value(series: pd.Series, value: float, high_is_bad: bool =
    s = pd.to_numeric(series, errors="coerce").dropna()
    if s.empty or pd.isna(value):
        return np.nan
    return float((s <= value).mean()) if high_is_bad else float((s >= value)

def mean_of_cols(df: pd.DataFrame, cols: list[str]) -> float:
    vals = df[cols].apply(pd.to_numeric, errors="coerce")
    vals = vals.stack(dropna=True)
    if len(vals) == 0:
        return np.nan
    return float(vals.mean())

def wrap(text, width=34):

```

```

    return "\n".join(textwrap.wrap(str(text), width=width))

def draw_box(ax, x, y, w, h, title, body, face="#f7f7f7", edge="#999999", titlefont="serif", bodyfont="sans-serif", align="left", baseline="top",
             patch = FancyBboxPatch(
                 (x, y), w, h,
                 boxstyle="round,pad=0.012,rounding_size=0.02",
                 linewidth=1.2, edgecolor=edge, facecolor=face
             )
    ax.add_patch(patch)
    ax.text(x + 0.02*w, y + h - 0.08*h, title, fontsize=13, weight="bold", fontstyle=titlefont, align="left", baseline="top")
    ax.text(x + 0.02*w, y + h - 0.20*h, body, fontsize=body_fs, fontstyle=bodyfont, align="left", baseline="top")

def find_cluster(summary_df, state_name, county_name):
    tmp = summary_df.loc[
        (summary_df["display_state"] == state_name) &
        (summary_df["display_county"] == county_name)
    ].copy()
    if tmp.empty:
        raise ValueError(f"Could not find cluster row for {county_name}, {state_name}")
    # choose the lowest recovery IGS row if duplicates ever exist
    sort_cols = [c for c in ["igs_total_recovery", "cluster_id"] if c in tmp.columns]
    if sort_cols:
        tmp = tmp.sort_values(sort_cols, ascending=[True] * len(sort_cols))
    return tmp.iloc[0]

# -----
# Finalist anchors
# -----
apache_row = find_cluster(cluster_period_summary, "Arizona", "Apache County")
mck_row = find_cluster(cluster_period_summary, "New Mexico", "McKinley County")

APACHE_CLUSTER_ID = apache_row["cluster_id"]
MCK_CLUSTER_ID = mck_row["cluster_id"]

# -----
# Build cluster health composites
# -----
provider_cols = [
    "primary_care_physicians_rate",
    "dentists_rate",
    "mental_health_providers_rate",
    "other_primary_care_providers_rate",
]
uninsured_cols = [
    "uninsured_rate",
    "uninsured_adults_rate",
    "uninsured_children_rate",
]
preventive_cols = [
    "mammography_screening_rate",
    "flu_vaccinations_rate",
]

cluster_health_composites = pd.DataFrame({
    "cluster_id": healthcare_cluster_scored["cluster_id"],
    "uninsured_burden_score": (

```



```

        bad_when_high(healthcare_cluster_scored["uninsured_rate"]) +
        bad_when_high(healthcare_cluster_scored["uninsured_adults_rate"]) +
        bad_when_high(healthcare_cluster_scored["uninsured_children_rate"])
    ) / 3,
    "provider_access_gap_score": (
        bad_when_low(healthcare_cluster_scored["primary_care_physicians_rate"]) +
        bad_when_low(healthcare_cluster_scored["dentists_rate"]) +
        bad_when_low(healthcare_cluster_scored["mental_health_providers_rate"]) +
        bad_when_low(healthcare_cluster_scored["other_primary_care_providers_rate"])
    ) / 4,
    "preventive_care_gap_score": (
        bad_when_low(healthcare_cluster_scored["mammography_screening_rate"]) +
        bad_when_low(healthcare_cluster_scored["flu_vaccinations_rate"])
    ) / 2,
    "preventable_stays_score": bad_when_high(healthcare_cluster_scored["preventable_stays_rate"])
})

healthcare_cluster_scored = healthcare_cluster_scored.merge(
    cluster_health_composites,
    on="cluster_id",
    how="left",
    validate="1:1",
)

apache_health = healthcare_cluster_scored.loc[
    healthcare_cluster_scored["cluster_id"] == APACHE_CLUSTER_ID
].iloc[0]

mck_health = healthcare_cluster_scored.loc[
    healthcare_cluster_scored["cluster_id"] == MCK_CLUSTER_ID
].iloc[0]

# -----
# Optional sourced Apache context
# Replace only if you want different sourced values
# -----
PLACE_CONTEXT = {
    "population_2024": 64800,
    "land_area_sq_miles": 11198.31,
    "population_density_2020": 5.9,
    "broadband_subscription_pct": 60.0,
    "under65_uninsured_pct": 13.0,
    "labor_force_pct": 41.7,
    "poverty_pct": 29.3,
    "employer_establishments_2023": 442,
}

print("Loaded from:", PROCESSED_DIR)
print("Will save outputs to:", OUT_DIR)
print("Apache cluster:", APACHE_CLUSTER_ID)
print("McKinley cluster:", MCK_CLUSTER_ID)

```

Loaded from: data\processed
 Will save outputs to: data\processed\presentation_ready
 Apache cluster: Arizona | Apache County | cluster_3
 McKinley cluster: New Mexico | McKinley County | cluster_49

```

In [68]: # =====
# 1) APACHE-SPECIFIC BENCHMARK SLIDE
# Apache vs Arizona vs U.S. vs McKinley
# =====

# -----
# A. Recovery-period economic raw values
# -----
econ_metric_map = {
    "igs_total_recovery": "IGS Total",
    "igs_economy_recovery": "IGS Economy",
    "poverty_rate_recovery": "Poverty",
    "unemp_rate_recovery": "Unemployment",
    "median_household_income_recovery": "Median Income",
    "lfpr_16p_recovery": "LFPR",
}

recovery_panel = analysis_panel.loc[analysis_panel["analysis_period"] == "recovery"]
az_recovery = recovery_panel.loc[recovery_panel["state_fips"] == "04"].copy()

apache_econ_raw = pd.Series({
    "IGS Total": apache_row["igs_total_recovery"],
    "IGS Economy": apache_row["igs_economy_recovery"],
    "Poverty": apache_row["poverty_rate_recovery"],
    "Unemployment": apache_row["unemp_rate_recovery"],
    "Median Income": apache_row["median_household_income_recovery"],
    "LFPR": apache_row["lfpr_16p_recovery"],
})

mck_econ_raw = pd.Series({
    "IGS Total": mck_row["igs_total_recovery"],
    "IGS Economy": mck_row["igs_economy_recovery"],
    "Poverty": mck_row["poverty_rate_recovery"],
    "Unemployment": mck_row["unemp_rate_recovery"],
    "Median Income": mck_row["median_household_income_recovery"],
    "LFPR": mck_row["lfpr_16p_recovery"],
})

az_econ_raw = pd.Series({
    "IGS Total": weighted_avg(az_recovery, "igs_total"),
    "IGS Economy": weighted_avg(az_recovery, "igs_economy"),
    "Poverty": weighted_avg(az_recovery, "poverty_rate"),
    "Unemployment": weighted_avg(az_recovery, "unemp_rate"),
    "Median Income": weighted_avg(az_recovery, "median_household_income"),
    "LFPR": weighted_avg(az_recovery, "lfpr_16p"),
})

us_econ_raw = pd.Series({
    "IGS Total": weighted_avg(recovery_panel, "igs_total"),
    "IGS Economy": weighted_avg(recovery_panel, "igs_economy"),
    "Poverty": weighted_avg(recovery_panel, "poverty_rate"),
    "Unemployment": weighted_avg(recovery_panel, "unemp_rate"),
    "Median Income": weighted_avg(recovery_panel, "median_household_income"),
    "LFPR": weighted_avg(recovery_panel, "lfpr_16p"),
})

```

```

# -----
# B. Healthcare benchmark raw values
# 2022–2023 overlap years only
# -----
health_bench = healthcare_standardized.loc[
    healthcare_standardized["year"].isin([2022, 2023])
].copy()

apache_chr = health_bench.loc[health_bench["county_fips"] == "04001"].copy()
mck_chr = health_bench.loc[health_bench["county_fips"] == "35031"].copy()
az_chr = health_bench.loc[health_bench["state_fips"] == "04"].copy()
us_chr = health_bench.copy()

def uninsured_burden_raw(df):
    return mean_of_cols(df, uninsured_cols)

def provider_access_gap_index(df, ref_df):
    scores = []
    for col in provider_cols:
        v = mean_of_cols(df, [col])
        scores.append(percentile_of_value(ref_df[col], v, high_is_bad=False))
    return float(np.nanmean(scores))

apache_health_raw = pd.Series({
    "Uninsured Burden": uninsured_burden_raw(apache_chr),
    "Provider Access Gap": provider_access_gap_index(apache_chr, health_bench)
})

mck_health_raw = pd.Series({
    "Uninsured Burden": uninsured_burden_raw(mck_chr),
    "Provider Access Gap": provider_access_gap_index(mck_chr, health_bench)
})

az_health_raw = pd.Series({
    "Uninsured Burden": uninsured_burden_raw(az_chr),
    "Provider Access Gap": provider_access_gap_index(az_chr, health_bench)
})

us_health_raw = pd.Series({
    "Uninsured Burden": uninsured_burden_raw(us_chr),
    "Provider Access Gap": provider_access_gap_index(us_chr, health_bench)
})

# -----
# C. Combined raw benchmark table
# -----
benchmark_raw = pd.concat(
    [
        pd.concat([apache_econ_raw, apache_health_raw]).rename("Apache County"),
        pd.concat([az_econ_raw, az_health_raw]).rename("Arizona"),
        pd.concat([us_econ_raw, us_health_raw]).rename("U.S."),
        pd.concat([mck_econ_raw, mck_health_raw]).rename("McKinley County"),
    ],
    axis=1,
)

```

```

# -----
# D. Severity-score benchmark (higher = worse)
# Uses national distributions as reference
# -----
benchmark_scores = pd.DataFrame(index=benchmark_raw.index)

for group_name, raw_series in benchmark_raw.items():
    out = {}
    out["IGS Total"] = percentile_of_value(recovery_panel["igs_total"], raw_series)
    out["IGS Economy"] = percentile_of_value(recovery_panel["igs_economy"], raw_series)
    out["Poverty"] = percentile_of_value(recovery_panel["poverty_rate"], raw_series)
    out["Unemployment"] = percentile_of_value(recovery_panel["unemp_rate"], raw_series)
    out["Median Income"] = percentile_of_value(recovery_panel["median_household_income"], raw_series)
    out["LFPR"] = percentile_of_value(recovery_panel["lfpr_16p"], raw_series)

    if group_name == "Apache County":
        src = apache_chr
    elif group_name == "Arizona":
        src = az_chr
    elif group_name == "U.S.":
        src = us_chr
    else:
        src = mck_chr

    uninsured_parts = []
    for col in uninsured_cols:
        uninsured_parts.append(
            percentile_of_value(health_bench[col], mean_of_cols(src, [col]),
                               [uninsured_cols])
        )
    out["Uninsured Burden"] = float(np.nanmean(uninsured_parts))

    provider_parts = []
    for col in provider_cols:
        provider_parts.append(
            percentile_of_value(health_bench[col], mean_of_cols(src, [col]),
                               [provider_cols])
        )
    out["Provider Access Gap"] = float(np.nanmean(provider_parts))

    benchmark_scores[group_name] = pd.Series(out)

# -----
# E. Pretty formatting
# -----
def fmt_metric(metric, v):
    if pd.isna(v):
        return "NA"
    if metric in ["Poverty", "Unemployment", "LFPR", "Uninsured Burden"]:
        return f"{100*v:.1f}%"
    if metric == "Median Income":
        return f"${v:,.0f}"
    if metric == "Provider Access Gap":
        return f"{v:.2f}"
    return f"{v:.1f}"

benchmark_raw_fmt = benchmark_raw.copy()

```

```

for r in benchmark_raw_fmt.index:
    for c in benchmark_raw_fmt.columns:
        benchmark_raw_fmt.loc[r, c] = fmt_metric(r, benchmark_raw_fmt.loc[r,

# -----
# F. Plot (VERTICAL VERSION)
# -----
plot_df = benchmark_scores.loc[
    [
        "IGS Total", "IGS Economy", "Poverty", "Unemployment",
        "Median Income", "LFPR", "Uninsured Burden", "Provider Access Gap"
    ]
].copy()

fig, (ax1, ax2) = plt.subplots(
    1, 2, figsize=(18, 9),
    gridspec_kw={"width_ratios": [1.7, 1.0]}
)

# left: grouped vertical bars
groups = ["Apache County", "Arizona", "U.S.", "McKinley County"]
x = np.arange(len(plot_df.index))
bar_w = 0.18
offsets = np.array([-1.5, -0.5, 0.5, 1.5]) * bar_w

for i, g in enumerate(groups):
    ax1.bar(x + offsets[i], plot_df[g].values, width=bar_w, label=g)

ax1.set_xticks(x)
ax1.set_xticklabels(plot_df.index, rotation=30, ha="right")
ax1.set_ylim(0, 1.05)
ax1.set_ylabel("Relative severity score (higher = worse)")
ax1.set_title("Apache benchmark vs Arizona, U.S., and McKinley")
ax1.grid(axis="y", alpha=0.25)
ax1.legend(loc="upper right", frameon=True)

# right: raw-value table
ax2.axis("off")
tbl = ax2.table(
    cellText=benchmark_raw_fmt.values,
    rowLabels=benchmark_raw_fmt.index,
    colLabels=benchmark_raw_fmt.columns,
    loc="center",
    cellLoc="center",
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(9)
tbl.scale(1.08, 1.55)
ax2.set_title("Raw values used in the benchmark", pad=14)

fig.suptitle("Slide 1 – Apache County Benchmark", fontsize=18, y=0.98)
fig.text(
    0.02, 0.02,
    "Economic metrics use recovery-period values; healthcare uses 2022–2023"
    "Severity scores are direction-aligned percentiles against national dist"
    fontsize=10

```

```
)  
plt.tight_layout(rect=[0, 0.05, 1, 0.95])  
plt.show()
```

```
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.
  vals = vals.stack(dropna=True)
```

```
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
```


lence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
lence this warning.
    vals = vals.stack(dropna=True)
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWa
rning: The previous implementation of stack is deprecated and will be remove
d in a future version of pandas. See the What's New notes for pandas 2.1.0 f
or details. Specify future_stack=True to adopt the new implementation and si
```

lence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\4057870243.py:105: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
vals = vals.stack(dropna=True)
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\1932022422.py:170: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value '34.2' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```
benchmark_raw_fmt.loc[r, c] = fmt_metric(r, benchmark_raw_fmt.loc[r, c])
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\1932022422.py:170: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value '50.7' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```
benchmark_raw_fmt.loc[r, c] = fmt_metric(r, benchmark_raw_fmt.loc[r, c])
```

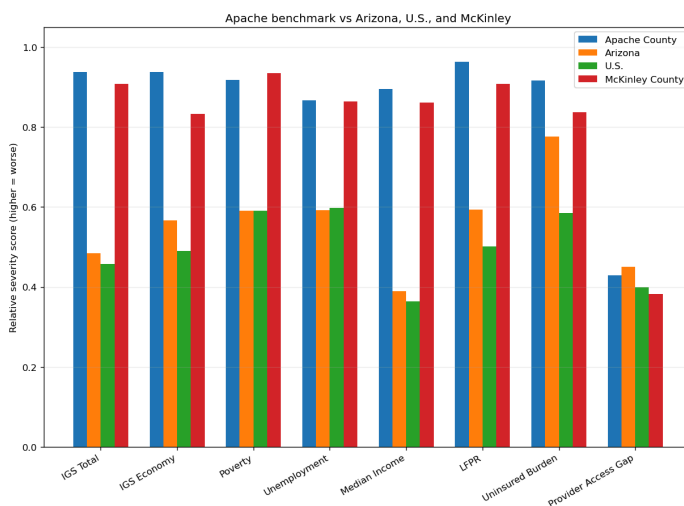
C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\1932022422.py:170: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value '51.1' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```
benchmark_raw_fmt.loc[r, c] = fmt_metric(r, benchmark_raw_fmt.loc[r, c])
```

C:\Users\jabba\AppData\Local\Temp\ipykernel_3872\1932022422.py:170: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value '37.0' has dtype incompatible with float64, please explicitly cast to a compatible dtype first.

```
benchmark_raw_fmt.loc[r, c] = fmt_metric(r, benchmark_raw_fmt.loc[r, c])
```

Slide 1 — Apache County Benchmark



Raw values used in the benchmark

	Apache County	Arizona	U.S.	McKinley County
IGS Total	34.2	50.7	51.1	37.0
IGS Economy	28.7	48.7	51.8	37.7
Poverty	31.4%	13.0%	13.0%	34.1%
Unemployment	10.0%	5.4%	5.4%	9.9%
Median Income	\$41,738	\$84,160	\$86,867	\$45,567
LFPR	40.7%	61.3%	63.5%	48.5%
Uninsured Burden	18.6%	14.1%	10.9%	16.8%
Provider Access Gap	0.43	0.45	0.40	0.38

Economic metrics use recovery-period values; healthcare uses 2022-2023 overlap years. Severity scores are direction-aligned percentiles against national distributions.

```
In [69]: # =====
# 2) CLEAN "WHY APACHE?" STORY SLIDE
# Plain-English, judge-facing summary
# =====

if "benchmark_scores" not in globals():
    raise RuntimeError("Run the benchmark cell first.")

# -----
```

```

# A. Useful story signals
# -----
apache_vs_az_worse = int((benchmark_scores["Apache County"] > benchmark_scores["Arizona"])
apache_vs_us_worse = int((benchmark_scores["Apache County"] > benchmark_scores["United States"])

health_rank = healthcare_cluster_scored["healthcare_need_score"].rank(ascending=True)
apache_health_rank = int(
    health_rank.loc[healthcare_cluster_scored["cluster_id"] == APACHE_CLUSTER_ID]
)
n_healthcare_finalists = int(len(healthcare_cluster_scored))

geo_text = (
    f"Apache County is a very large, low-density place. "
    f"It spans about {PLACE_CONTEXT['land_area_sq_miles']:.0f} square miles "
    f"with only {PLACE_CONTEXT['population_density_2020']:.1f} people per square mile. "
    f"Our selected cluster covers {int(apache_row['n_cluster_tracts'])} tracts "
    f"and about {int(apache_row['latest_cluster_pop_total']):,} residents."
)

econ_text = (
    f"Apache still looks economically fragile in recovery: "
    f"IGS Total = {apache_row['igs_total_recovery']:.1f}, "
    f"IGS Economy = {apache_row['igs_economy_recovery']:.1f}, "
    f"poverty = {apache_row['poverty_rate_recovery']:.1%}, "
    f"unemployment = {apache_row['unemp_rate_recovery']:.1%}, "
    f"median income = ${apache_row['median_household_income_recovery']:.0f} "
    f"LFPFR = {apache_row['lfpr_16p_recovery']:.1%}. "
    f"Across the 8 benchmark metrics, Apache is worse than Arizona on "
    f"{apache_vs_az_worse}/8 and worse than the U.S. on {apache_vs_us_worse}/8."
)

health_text = (
    f"Healthcare access is also a major part of the story. "
    f"Apache ranks #{apache_health_rank} of {n_healthcare_finalists} health clusters "
    f"on overall healthcare need. Its uninsured burden score is "
    f"{apache_health['uninsured_burden_score']:.2f} and provider access gap "
    f"{apache_health['provider_access_gap_score']:.2f}, both pointing to a significant need."
)

fit_text = (
    f"That combination makes Apache a strong fit for a logistics/access solution. "
    f"When distance is large, providers are limited, broadband is only "
    f"{PLACE_CONTEXT['broadband_subscription_pct']:.1f}% of households, "
    f"and the county has only {PLACE_CONTEXT['employer_establishments_2023']} "
    f"a local healthcare logistics business is easier to defend than a general store."
)

# -----
# B. Plot story slide
# -----
fig, ax = plt.subplots(figsize=(16, 9))
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis("off")

ax.text(0.03, 0.95, "Slide 2 – Why Apache County?", fontsize=24, weight="bold")

```

```

ax.text(
    0.03, 0.90,
    "A plain-English version of the story for judges",
    fontsize=13, color="dimgray", va="top"
)

draw_box(
    ax, 0.03, 0.56, 0.45, 0.27,
    "1) Large geography + low density",
    wrap(geo_text, 52),
    face="#eef6ff", edge="#86a8d8"
)

draw_box(
    ax, 0.52, 0.56, 0.45, 0.27,
    "2) Persistent economic vulnerability",
    wrap(econ_text, 54),
    face="#fff5e8", edge="#d9ad65"
)

draw_box(
    ax, 0.03, 0.22, 0.45, 0.27,
    "3) Healthcare access is weak",
    wrap(health_text, 52),
    face="#edf9ef", edge="#7dbb83"
)

draw_box(
    ax, 0.52, 0.22, 0.45, 0.27,
    "4) Why the solution fits the place",
    wrap(fit_text, 54),
    face="#f8eeff", edge="#b392c6"
)

bottom_callout = (
    "Judge-ready takeaway:\n"
    "Apache County was not chosen just because it scored poorly. "
    "It was chosen because it combines low inclusive growth, persistent econ
    "clear healthcare access burden, and a place-based operating environment
)

draw_box(
    ax, 0.03, 0.04, 0.94, 0.12,
    "Bottom line",
    wrap(bottom_callout, 130),
    face="#f4f4f4", edge="#888888", body_fs=12
)

plt.tight_layout()
plt.show()

fig.savefig(OUT_DIR / "slide_02_why_apache_story.png", bbox_inches="tight")
print("Saved:")
print("-", OUT_DIR / "slide_02_why_apache_story.png")

```

Slide 2 — Why Apache County?

A plain-English version of the story for judges

1) Large geography + low density

Apache County is a very large, low-density place. It spans about 11,190 square miles with only 5.9 people per square mile. Our selected cluster covers 18 tracts and about 65,341 residents.

2) Persistent economic vulnerability

Apache still looks economically fragile in recovery: IGS Total = 34.2, IGS Economy = 28.7, poverty = 31.4%, unemployment = 10.0%, median income = \$41,738, LFPR = 40.7%. Across the 8 benchmark metrics, Apache is worse than Arizona on 7/8 and worse than the U.S. on 8/8.

3) Healthcare access is weak

Healthcare access is also a major part of the story. Apache ranks #1 of 12 healthcare-covered finalists on overall healthcare need. Its uninsured burden score is 0.97 and provider access gap score is 0.90, both pointing to a strong access problem.

4) Why the solution fits the place

That combination makes Apache a strong fit for a logistics/access solution. When distance is large, providers are limited, broadband is only 60.0% of households, and the county has only 442 employer establishments, a local healthcare logistics business is easier to defend than a generic clinic-first solution.

Bottom line

Judge-ready takeaway: Apache County was not chosen just because it scored poorly. It was chosen because it combines low inclusive growth, persistent economic strain, clear healthcare access burden, and a place-based operating environment where a logistics/access business makes sense.

Saved:

- data\processed\presentation_ready\slide_02_why_apache_story.png

```
In [70]: # =====
# 3) BUSINESS-LAYER EVIDENCE SLIDE
# Why AI + rural healthcare logistics / drone-assisted delivery fits Apache
# =====

# -----
# A. Build evidence rows
# -----
biz_evidence_df = pd.DataFrame([
    {
        "Apache signal": "Large geography / low density",
        "Evidence": (
            f"{PLACE_CONTEXT['land_area_sq_miles']:.0f} sq mi and "
            f"{PLACE_CONTEXT['population_density_2020']:.1f} people/sq mi."
        ),
        "Why a normal urban solution fits less well": (
            "A clinic-first or storefront-only model still leaves residents
        ),
        "Why an AI + logistics model fits": (
            "AI routing, delivery batching, and selective last-mile delivery
        ),
    },
    {
        "Apache signal": "Economic fragility",
        "Evidence": (
            f"Recovery poverty = {apache_row['poverty_rate_recovery']:.1%};
            f"median income = ${apache_row['median_household_income_recovery']:.0f};
            f"LFPR = {apache_row['lfpr_16p_recovery']:.1%}."
        ),
        "Why a normal urban solution fits less well": (
            "Residents may delay care or medication refills when transportat
        ),
        "Why an AI + logistics model fits": (
```



```

        "A lower-overhead access business can reduce missed pickups, mis
    ),
},
{
    "Apache signal": "Insurance / affordability pressure",
    "Evidence": (
        f"QuickFacts uninsured under 65 = {PLACE_CONTEXT['under65_uninsu
        f"finalist uninsured burden score = {apache_health['uninsured_bu
    ),
    "Why a normal urban solution fits less well": (
        "The problem is not only provider supply. It is also whether pec
    ),
    "Why an AI + logistics model fits": (
        "Medication reminders, refill-risk flags, and low-friction deliv
    ),
},
{
    "Apache signal": "Provider access gaps",
    "Evidence": (
        f"Provider access gap score = {apache_health['provider_access_ga
        f"healthcare need score = {apache_health['healthcare_need_score'
    ),
    "Why a normal urban solution fits less well": (
        "When providers are sparse, residents often need coordination ac
    ),
    "Why an AI + logistics model fits": (
        "Navigation support, telehealth setup, and delivery from existin
    ),
},
{
    "Apache signal": "Thin service / infrastructure environment",
    "Evidence": (
        f"{PLACE_CONTEXT['broadband_subscription_pct']:.1f}% household b
        f"{PLACE_CONTEXT['employer_establishments_2023']:,} employer est
    ),
    "Why a normal urban solution fits less well": (
        "App-only or telehealth-only models leave some households behind
    ),
    "Why an AI + logistics model fits": (
        "A hybrid model can combine AI dispatch, human navigators, physi
    ),
},
1)

# Wrap text for display
display_df = biz_evidence_df.copy()
for col in display_df.columns:
    display_df[col] = display_df[col].map(lambda x: wrap(x, 28))

# -----
# B. Plot as a judge-facing evidence table
# -----
fig, ax = plt.subplots(figsize=(19, 10))
ax.axis("off")
ax.set_title(
    "Slide 3 – Business-Layer Evidence for Apache County",

```



```

        fontsize=20, pad=18
    )

tbl = ax.table(
    cellText=display_df.values,
    colLabels=display_df.columns,
    loc="center",
    cellLoc="left",
    colLoc="center",
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(10)
tbl.scale(1.0, 2.2)

# Header styling
for (r, c), cell in tbl.get_celld().items():
    if r == 0:
        cell.set_text_props(weight="bold")
        cell.set_facecolor("#e8eef7")
    else:
        if c == 0:
            cell.set_facecolor("#f4f8fc")
        elif c == 1:
            cell.set_facecolor("#fbfbfb")
        elif c == 2:
            cell.set_facecolor("#fff8ef")
        elif c == 3:
            cell.set_facecolor("#eef9f0")

fig.text(
    0.01, 0.02,
    "Recommended framing: Apache does not just need more healthcare supply.  

    "It needs a local business that reduces the friction of reaching care.",
    fontsize=12
)

plt.tight_layout(rect=[0, 0.04, 1, 0.96])
plt.show()

biz_evidence_df.to_csv(OUT_DIR / "apache_business_layer_evidence.csv", index=False)
fig.savefig(OUT_DIR / "slide_03_business_layer_evidence.png", bbox_inches="tight")

print("Saved:")
print("-", OUT_DIR / "apache_business_layer_evidence.csv")
print("-", OUT_DIR / "slide_03_business_layer_evidence.png")

```

Slide 3 — Business-Layer Evidence for Apache County

Apache signal	Evidence	Why a normal urban solution fits less well	Why an AI + logistics model fits
Large geography / low density	11,198 sq mi and 5.9 people/sq mi.	A clinic-first or storefront-only model still leaves many people or medication refills when transportation, time, and provider supply. It is also relatively thin.	AI routing, delivery batching, and selective last-mile delivery solve the access access problem.
Economic fragility	Recovery poverty = 31.4%; median income = \$41,738; 65% of the population measured under	the problem is not only	business can reduce missed pickups, missed refills, and medication reminders.
Insurance / affordability pressure	65 = 13.0%, finalist uninsured burden score =	When residents are unable	refill-risk flags, and low-cost options to improve delivery from existing
Provider access gaps	Provider access gap score = 0.90; healthcare need score = 0.88.	residents often need	coordination across longer distances, and the local commercial base is
Thin service / infrastructure environment	60.0% household broadband; 442 employer establishments.	models leave some households	behind, and the local commercial base is relatively thin.

Recommended framing: Apache does not just need more healthcare supply. It needs a local business that reduces the friction of reaching care.

Saved:

- data\processed\presentation_ready\apache_business_layer_evidence.csv
- data\processed\presentation_ready\slide_03_business_layer_evidence.png

```
In [71]: # =====
# 4) IMPLEMENTATION + IMPACT METRICS SLIDE
# What the business is, who it serves, how it works, AI role, KPIs
# =====

BUSINESS_NAME = "Apache HealthLink AI"

slide_sections = {
    "What the business is": (
        f"{BUSINESS_NAME} is a rural healthcare logistics small business. "
        f"It helps residents get medications, stay on schedule, and stay cor
    ),
    "Who the first customers are": (
        "Start with residents who face the highest access friction: "
        "chronic-condition patients, households far from pharmacies/clinics, "
        "and people with repeated refill or appointment barriers."
    ),
    "How it operates": (
        "Hub-and-spoke model. Partner pharmacies and clinics hand off medica
        "The business manages delivery scheduling, pickup points, reminder c
    ),
    "Where AI fits": (
        "AI is the operations engine: route optimization, refill-risk flags, "
        "and mode selection (ground delivery, pickup point, telehealth assis
    ),
    "Why drones are only one part": (
        "Do not pitch this as 'just drones.' Pitch it as a hybrid healthcare
        "Use ground delivery and pickup points by default; use a certified p
    ),
    "What should improve": (
        "Medication access should become faster and more reliable, missed re
        "and residents should have fewer logistics-related missed appointmer
    ),
}
```

```

kpis = pd.DataFrame({
    "Metric": [
        "On-time medication delivery rate",
        "Refill adherence / on-time refill rate",
        "Missed-appointment reduction",
        "Successful telehealth-support completions",
        "Cost per successful delivery",
        "Repeat-customer retention",
    ],
    "How to measure it": [
        "Completed on schedule / total scheduled deliveries",
        "Share of targeted refills delivered or picked up on time",
        "Before-vs-after comparison for customers enrolled in the service",
        "Completed assisted telehealth visits / scheduled assisted visits",
        "Total operating cost / completed deliveries",
        "Share of customers using the service again within 90 days",
    ]
})

# -----
# Plot slide
# -----
fig, ax = plt.subplots(figsize=(17, 10))
ax.set_xlim(0, 1)
ax.set_ylim(0, 1)
ax.axis("off")

ax.text(0.03, 0.96, "Slide 4 – Implementation + Impact Metrics", fontsize=24)
ax.text(0.03, 0.91, f"Business recommendation: {BUSINESS_NAME}", fontsize=14)

# 2 x 3 grid of boxes
positions = [
    (0.03, 0.63, 0.30, 0.20, "What the business is"),
    (0.35, 0.63, 0.30, 0.20, "Who the first customers are"),
    (0.67, 0.63, 0.30, 0.20, "How it operates"),
    (0.03, 0.37, 0.30, 0.20, "Where AI fits"),
    (0.35, 0.37, 0.30, 0.20, "Why drones are only one part"),
    (0.67, 0.37, 0.30, 0.20, "What should improve"),
]

box_colors = ["#eef6ff", "#fff5e8", "#edf9ef", "#f8eefb", "#f5f5f5", "#fff9e8"]
box_edges = ["#86a8d8", "#d9ad65", "#7dbb83", "#b392c6", "#9d9d9d", "#d8b04c"]

for i, (x, y, w, h, key) in enumerate(positions):
    draw_box(
        ax, x, y, w, h,
        key,
        wrap(slide_sections[key], 42),
        face=box_colors[i],
        edge=box_edges[i],
        body_fs=10.5
    )

# KPI box
kpi_text = "\n".join(

```

```

    [f"• {m}" for m in kpis["Metric"].tolist()]
)
draw_box(
    ax, 0.03, 0.07, 0.42, 0.20,
    "Year-1 success metrics",
    wrap(kpi_text, 42),
    face="#f3f8f2", edge="#78a36d", body_fs=11
)

impact_text = (
    "Pilot suggestion:\n"
    "Start with a limited service area and one or two partner pharmacies/cli\n"
    "Measure operational performance first, then scale routes, pickup points\n"
)
draw_box(
    ax, 0.48, 0.07, 0.49, 0.20,
    "Implementation note",
    wrap(impact_text, 70),
    face="#f7f7f7", edge="#888888", body_fs=11
)

plt.tight_layout()
plt.show()

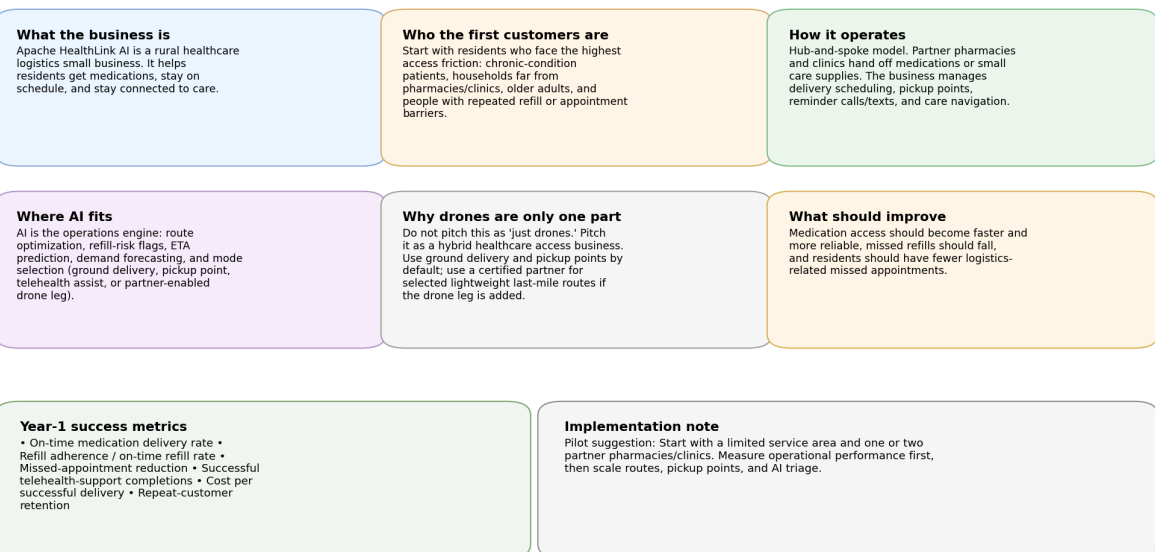
kpis.to_csv(OUT_DIR / "apache_business_kpis.csv", index=False)
fig.savefig(OUT_DIR / "slide_04_implementation_and_metrics.png", bbox_inches=

print("Saved:")
print("-", OUT_DIR / "apache_business_kpis.csv")
print("-", OUT_DIR / "slide_04_implementation_and_metrics.png")

```

Slide 4 — Implementation + Impact Metrics

Business recommendation: Apache HealthLink AI



Saved:

- data\processed\presentation_ready\apache_business_kpis.csv
- data\processed\presentation_ready\slide_04_implementation_and_metrics.png

```

In [72]: # =====
# Show the exact tracts in the final Apache cluster
# =====

import pandas as pd
from pathlib import Path

PROC_CANDIDATES = [
    Path("src/data/processed"),
    Path("data/processed"),
    Path.cwd() / "src/data/processed",
    Path.cwd() / "data/processed",
]

PROCESSED_DIR = next((p for p in PROC_CANDIDATES if p.exists()), None)
if PROCESSED_DIR is None:
    raise FileNotFoundError("Could not find src/data/processed or data/processed")

tract_cluster_map = pd.read_parquet(PROCESSED_DIR / "tract_cluster_map.parquet")

FINAL_CLUSTER_ID = "Arizona | Apache County | cluster_3"

apache_tracts = (
    tract_cluster_map.loc[tract_cluster_map["cluster_id"] == FINAL_CLUSTER_ID]
    .copy()
)

apache_tracts["geoid"] = apache_tracts["geoid"].astype(str).str.zfill(11)
apache_tracts["state_fips"] = apache_tracts["geoid"].str[:2]
apache_tracts["county_fips"] = apache_tracts["geoid"].str[:5]
apache_tracts["tract_code"] = apache_tracts["geoid"].str[5:]

apache_tracts = apache_tracts.sort_values("geoid").reset_index(drop=True)

print("Final cluster:", FINAL_CLUSTER_ID)
print("Number of tracts:", apache_tracts["geoid"].nunique())
print()
print(apache_tracts[["geoid", "tract_code", "display_state", "display_county"]])

# optional save
apache_tracts[["geoid", "tract_code", "display_state", "display_county"]].to_csv(
    PROCESSED_DIR / "apache_final_cluster_tracts.csv",
    index=False
)
print("\nSaved:", PROCESSED_DIR / "apache_final_cluster_tracts.csv")

```

Final cluster: Arizona | Apache County | cluster_3
Number of tracts: 18

	geoid	tract_code	display_state	display_county
0	04001942600	942600	Arizona	Apache County
1	04001942700	942700	Arizona	Apache County
2	04001944000	944000	Arizona	Apache County
3	04001944100	944100	Arizona	Apache County
4	04001944201	944201	Arizona	Apache County
5	04001944202	944202	Arizona	Apache County
6	04001944301	944301	Arizona	Apache County
7	04001944302	944302	Arizona	Apache County
8	04001944901	944901	Arizona	Apache County
9	04001944902	944902	Arizona	Apache County
10	04001945001	945001	Arizona	Apache County
11	04001945002	945002	Arizona	Apache County
12	04001945100	945100	Arizona	Apache County
13	04001970201	970201	Arizona	Apache County
14	04001970202	970202	Arizona	Apache County
15	04001970300	970300	Arizona	Apache County
16	04001970501	970501	Arizona	Apache County
17	04001970502	970502	Arizona	Apache County

Saved: data\processed\apache_final_cluster_tracts.csv

```
In [73]: import pandas as pd
from pathlib import Path

PROC_CANDIDATES = [
    Path("src/data/processed"),
    Path("data/processed"),
    Path.cwd() / "src/data/processed",
    Path.cwd() / "data/processed",
]

PROCESSED_DIR = next((p for p in PROC_CANDIDATES if p.exists()), None)
if PROCESSED_DIR is None:
    raise FileNotFoundError("Could not find src/data/processed or data/proce

# This file should contain the finalist community clusters
enrichment_clusters = pd.read_parquet(PROCESSED_DIR / "enrichment_clusters.p

# Keep only Wayne + Shelby finalist communities
cand = enrichment_clusters.loc[
    (
        (enrichment_clusters["display_state"] == "Michigan") &
        (enrichment_clusters["display_county"] == "Wayne County")
    )
    |
    (
        (enrichment_clusters["display_state"] == "Tennessee") &
        (enrichment_clusters["display_county"] == "Shelby County")
    )
].copy()

# Show useful columns if they exist
```

```

preferred_cols = [
    "cluster_id",
    "display_state",
    "display_county",
    "enrichment_rank",
    "rank",
    "cluster_shortlist_score",
    "combined_score",
    "n_cluster_tracts",
    "latest_cluster_pop_total",
]
cols = [c for c in preferred_cols if c in cand.columns]

# Sort by best available ranking field
sort_cols = [c for c in ["enrichment_rank", "rank"] if c in cand.columns]
if sort_cols:
    cand = cand.sort_values(sort_cols)
elif "cluster_shortlist_score" in cand.columns:
    cand = cand.sort_values("cluster_shortlist_score", ascending=False)
elif "combined_score" in cand.columns:
    cand = cand.sort_values("combined_score", ascending=False)

print(cand[cols].to_string(index=False))

```

	cluster_id	display_state	display_county	enrichment_rank	rank	cluster_shortlist_score	n_cluster_tracts	latest_cluster_pop_total
5	Michigan Wayne County cluster_38	Michigan	Wayne County	5	0.772703	152	319984.0	
9	Michigan Wayne County cluster_39	Michigan	Wayne County	9	0.733314	9	19947.0	
11	Tennessee Shelby County cluster_77	Tennessee	Shelby County	11	0.712122	96	268298.0	

```

In [74]: import pandas as pd
         from pathlib import Path

PROC_CANDIDATES = [
    Path("src/data/processed"),
    Path("data/processed"),
    Path.cwd() / "src/data/processed",
    Path.cwd() / "data/processed",
]

PROCESSED_DIR = next((p for p in PROC_CANDIDATES if p.exists()), None)
if PROCESSED_DIR is None:
    raise FileNotFoundError("Could not find src/data/processed or data/processed")

enrichment_cluster_tracts = pd.read_parquet(PROCESSED_DIR / "enrichment_cluster_tracts.parquet")

# Replace this with the exact cluster_id you want from Step 1
TARGET_CLUSTER_ID = "Michigan | Wayne County | cluster_38"

tracts = enrichment_cluster_tracts.loc[
    enrichment_cluster_tracts["cluster_id"] == TARGET_CLUSTER_ID
]

```

```
].copy()

tracts["geoid"] = tracts["geoid"].astype(str).str.zfill(11)
tracts["tract_code"] = tracts["geoid"].str[5:]

tracts = tracts.sort_values("geoid").reset_index(drop=True)

print("Cluster:", TARGET_CLUSTER_ID)
print("Number of tracts:", tracts["geoid"].nunique())
print()
print(tracts[["geoid", "tract_code"]].to_string(index=False))

# Optional save
out_file = PROCESSED_DIR / f"{TARGET_CLUSTER_ID.replace('|', '__').replace(' ', '_')}.csv"
tracts[["geoid", "tract_code"]].to_csv(out_file, index=False)
print("\nSaved:", out_file)
```


Cluster: Michigan | Wayne County | cluster_38

Number of tracts: 152

geoid	tract_code
26163500300	500300
26163500400	500400
26163500500	500500
26163500600	500600
26163501000	501000
26163501100	501100
26163501200	501200
26163501300	501300
26163501400	501400
26163502600	502600
26163502700	502700
26163503400	503400
26163503500	503500
26163503600	503600
26163503900	503900
26163504000	504000
26163504100	504100
26163504300	504300
26163504400	504400
26163505200	505200
26163505400	505400
26163505700	505700
26163505800	505800
26163506100	506100
26163506200	506200
26163506300	506300
26163506400	506400
26163506500	506500
26163506600	506600
26163506700	506700
26163506800	506800
26163506900	506900
26163507000	507000
26163507100	507100
26163507200	507200
26163507400	507400
26163507500	507500
26163508000	508000
26163508100	508100
26163509000	509000
26163509100	509100
26163511300	511300
26163511400	511400
26163511900	511900
26163513800	513800
26163513900	513900
26163514100	514100
26163514200	514200
26163514300	514300
26163514500	514500
26163515200	515200
26163515300	515300

26163515400	515400
26163516000	516000
26163516400	516400
26163519200	519200
26163519300	519300
26163521500	521500
26163521900	521900
26163522000	522000
26163522300	522300
26163522800	522800
26163525800	525800
26163526000	526000
26163526100	526100
26163526200	526200
26163526300	526300
26163526500	526500
26163527900	527900
26163530100	530100
26163530300	530300
26163530400	530400
26163530500	530500
26163530800	530800
26163530900	530900
26163531100	531100
26163531200	531200
26163531300	531300
26163531400	531400
26163531501	531501
26163531502	531502
26163531600	531600
26163531700	531700
26163531800	531800
26163531900	531900
26163532100	532100
26163532400	532400
26163532600	532600
26163532700	532700
26163533200	533200
26163533400	533400
26163533600	533600
26163533800	533800
26163533900	533900
26163534100	534100
26163534200	534200
26163534300	534300
26163534400	534400
26163534700	534700
26163534800	534800
26163535000	535000
26163535100	535100
26163535200	535200
26163535300	535300
26163535600	535600
26163535700	535700
26163535800	535800
26163536500	536500

26163536600	536600
26163536900	536900
26163537000	537000
26163537100	537100
26163537200	537200
26163537300	537300
26163537500	537500
26163537600	537600
26163537700	537700
26163537800	537800
26163539500	539500
26163540400	540400
26163542100	542100
26163542200	542200
26163542300	542300
26163542400	542400
26163542900	542900
26163543200	543200
26163543300	543300
26163543400	543400
26163543500	543500
26163543700	543700
26163543900	543900
26163544200	544200
26163544800	544800
26163545100	545100
26163545200	545200
26163546000	546000
26163546100	546100
26163546600	546600
26163546700	546700
26163547000	547000
26163547100	547100
26163547200	547200
26163553000	553000
26163553100	553100
26163553200	553200
26163553601	553601
26163553800	553800
26163559800	559800
26163573600	573600
26163985000	985000
26163985200	985200
26163985800	985800

Saved: data\processed\Michigan__Wayne_County__cluster_38_tracts.csv

```
In [75]: import pandas as pd
from pathlib import Path

PROC_CANDIDATES = [
    Path("src/data/processed"),
    Path("data/processed"),
    Path.cwd() / "src/data/processed",
    Path.cwd() / "data/processed",
]
```

```

PROCESSED_DIR = next((p for p in PROC_CANDIDATES if p.exists()), None)
if PROCESSED_DIR is None:
    raise FileNotFoundError("Could not find src/data/processed or data/proce

enrichment_clusters = pd.read_parquet(PROCESSED_DIR / "enrichment_clusters.p
enrichment_cluster_tracts = pd.read_parquet(PROCESSED_DIR / "enrichment_clus

cand = enrichment_clusters.loc[
    (
        (enrichment_clusters["display_state"] == "Michigan") &
        (enrichment_clusters["display_county"] == "Wayne County")
    )
    |
    (
        (enrichment_clusters["display_state"] == "Tennessee") &
        (enrichment_clusters["display_county"] == "Shelby County")
    )
].copy()

# Pick best rank field available
if "enrichment_rank" in cand.columns:
    best = (
        cand.sort_values("enrichment_rank")
        .groupby(["display_state", "display_county"], as_index=False)
        .first()
    )
elif "rank" in cand.columns:
    best = (
        cand.sort_values("rank")
        .groupby(["display_state", "display_county"], as_index=False)
        .first()
    )
elif "cluster_shortlist_score" in cand.columns:
    best = (
        cand.sort_values("cluster_shortlist_score", ascending=False)
        .groupby(["display_state", "display_county"], as_index=False)
        .first()
    )
else:
    raise ValueError("No recognizable ranking column found.")

for _, row in best.iterrows():
    cid = row["cluster_id"]
    sub = enrichment_cluster_tracts.loc[
        enrichment_cluster_tracts["cluster_id"] == cid
    ].copy()

    sub["geoid"] = sub["geoid"].astype(str).str.zfill(11)
    sub["tract_code"] = sub["geoid"].str[5:]
    sub = sub.sort_values("geoid")

    print("\n" + "=" * 80)
    print(f"{row['display_county']}, {row['display_state']}")
    print("Top cluster:", cid)

```

```
print("Number of tracts:", sub["geoid"].nunique())  
print(sub[["geoid", "tract_code"]].to_string(index=False))
```

=====
=====

Wayne County, Michigan

Top cluster: Michigan | Wayne County | cluster_38

Number of tracts: 152

geoid	tract_code
26163500300	500300
26163500400	500400
26163500500	500500
26163500600	500600
26163501000	501000
26163501100	501100
26163501200	501200
26163501300	501300
26163501400	501400
26163502600	502600
26163502700	502700
26163503400	503400
26163503500	503500
26163503600	503600
26163503900	503900
26163504000	504000
26163504100	504100
26163504300	504300
26163504400	504400
26163505200	505200
26163505400	505400
26163505700	505700
26163505800	505800
26163506100	506100
26163506200	506200
26163506300	506300
26163506400	506400
26163506500	506500
26163506600	506600
26163506700	506700
26163506800	506800
26163506900	506900
26163507000	507000
26163507100	507100
26163507200	507200
26163507400	507400
26163507500	507500
26163508000	508000
26163508100	508100
26163509000	509000
26163509100	509100
26163511300	511300
26163511400	511400
26163511900	511900
26163513800	513800
26163513900	513900
26163514100	514100
26163514200	514200
26163514300	514300
26163514500	514500

26163515200	515200
26163515300	515300
26163515400	515400
26163516000	516000
26163516400	516400
26163519200	519200
26163519300	519300
26163521500	521500
26163521900	521900
26163522000	522000
26163522300	522300
26163522800	522800
26163525800	525800
26163526000	526000
26163526100	526100
26163526200	526200
26163526300	526300
26163526500	526500
26163527900	527900
26163530100	530100
26163530300	530300
26163530400	530400
26163530500	530500
26163530800	530800
26163530900	530900
26163531100	531100
26163531200	531200
26163531300	531300
26163531400	531400
26163531501	531501
26163531502	531502
26163531600	531600
26163531700	531700
26163531800	531800
26163531900	531900
26163532100	532100
26163532400	532400
26163532600	532600
26163532700	532700
26163533200	533200
26163533400	533400
26163533600	533600
26163533800	533800
26163533900	533900
26163534100	534100
26163534200	534200
26163534300	534300
26163534400	534400
26163534700	534700
26163534800	534800
26163535000	535000
26163535100	535100
26163535200	535200
26163535300	535300
26163535600	535600
26163535700	535700

26163535800	535800
26163536500	536500
26163536600	536600
26163536900	536900
26163537000	537000
26163537100	537100
26163537200	537200
26163537300	537300
26163537500	537500
26163537600	537600
26163537700	537700
26163537800	537800
26163539500	539500
26163540400	540400
26163542100	542100
26163542200	542200
26163542300	542300
26163542400	542400
26163542900	542900
26163543200	543200
26163543300	543300
26163543400	543400
26163543500	543500
26163543700	543700
26163543900	543900
26163544200	544200
26163544800	544800
26163545100	545100
26163545200	545200
26163546000	546000
26163546100	546100
26163546600	546600
26163546700	546700
26163547000	547000
26163547100	547100
26163547200	547200
26163553000	553000
26163553100	553100
26163553200	553200
26163553601	553601
26163553800	553800
26163559800	559800
26163573600	573600
26163985000	985000
26163985200	985200
26163985800	985800

=====

=====

Shelby County, Tennessee

Top cluster: Tennessee | Shelby County | cluster_77

Number of tracts: 96

geoid	tract_code
47157000100	000100
47157000200	000200
47157000300	000300

47157000400	000400
47157000600	000600
47157000700	000700
47157000800	000800
47157000900	000900
47157001100	001100
47157001300	001300
47157001400	001400
47157001500	001500
47157001600	001600
47157001700	001700
47157001900	001900
47157002000	002000
47157002100	002100
47157002400	002400
47157002500	002500
47157003300	003300
47157003500	003500
47157003600	003600
47157003700	003700
47157003800	003800
47157004300	004300
47157004500	004500
47157004600	004600
47157005000	005000
47157005300	005300
47157005500	005500
47157005600	005600
47157005700	005700
47157005800	005800
47157005900	005900
47157006000	006000
47157006200	006200
47157006300	006300
47157006400	006400
47157006500	006500
47157006700	006700
47157006800	006800
47157006900	006900
47157007000	007000
47157007100	007100
47157007200	007200
47157007300	007300
47157007400	007400
47157007500	007500
47157007810	007810
47157007821	007821
47157007822	007822
47157007900	007900
47157008000	008000
47157008110	008110
47157008120	008120
47157008900	008900
47157009901	009901
47157009902	009902
47157010001	010001

47157010002	010002
47157010120	010120
47157010121	010121
47157010122	010122
47157010210	010210
47157010220	010220
47157010300	010300
47157011100	011100
47157011200	011200
47157011401	011401
47157011402	011402
47157011500	011500
47157011600	011600
47157011700	011700
47157020221	020221
47157020511	020511
47157020521	020521
47157020523	020523
47157020524	020524
47157020531	020531
47157020532	020532
47157020543	020543
47157022111	022111
47157022121	022121
47157022130	022130
47157022210	022210
47157022220	022220
47157022310	022310
47157022321	022321
47157022322	022322
47157022330	022330
47157022410	022410
47157022500	022500
47157022700	022700
47157980100	980100
47157980200	980200
47157980300	980300

```
In [76]: # =====
# Map the Shelby County finalist community (cluster_77)
# =====

import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

# -----
# 1. Final Shelby cluster GEOIDs
# -----
TARGET_GEIDS = [
    "47157000100", "47157000200", "47157000300", "47157000400", "47157000600", "4
    "47157000800", "47157000900", "47157001100", "47157001300", "47157001400", "4
    "47157001600", "47157001700", "47157001900", "47157002000", "47157002100", "4
    "47157002500", "47157003300", "47157003500", "47157003600", "47157003700", "4
    "47157004300", "47157004500", "47157004600", "47157005000", "47157005300", "4
```

```

"47157005600", "47157005700", "47157005800", "47157005900", "47157006000", "4
"47157006300", "47157006400", "47157006500", "47157006700", "47157006800", "4
"47157007000", "47157007100", "47157007200", "47157007300", "47157007400", "4
"47157007810", "47157007821", "47157007822", "47157007900", "47157008000", "4
"47157008120", "47157008900", "47157009901", "47157009902", "47157010001", "4
"47157010120", "47157010121", "47157010122", "47157010210", "47157010220", "4
"47157011100", "47157011200", "47157011401", "47157011402", "47157011500", "4
"47157011700", "47157020221", "47157020511", "47157020521", "47157020523", "4
"47157020531", "47157020532", "47157020543", "47157022111", "47157022121", "4
"47157022210", "47157022220", "47157022310", "47157022321", "47157022322", "4
"47157022410", "47157022500", "47157022700", "47157980100", "47157980200", "4
]

# -----
# 2. Find repo folders
# -----
ROOT_CANDIDATES = [
    Path.cwd(),
    Path.cwd() / "src",
    Path.cwd().parent,
]

RAW_DIR = None
PROCESSED_DIR = None

for root in ROOT_CANDIDATES:
    if (root / "src/data/raw").exists():
        RAW_DIR = root / "src/data/raw"
        PROCESSED_DIR = root / "src/data/processed"
        break
    if (root / "data/raw").exists():
        RAW_DIR = root / "data/raw"
        PROCESSED_DIR = root / "data/processed"
        break

if RAW_DIR is None:
    raise FileNotFoundError("Could not find data/raw or src/data/raw.")

OUT_DIR = PROCESSED_DIR / "presentation_ready"
OUT_DIR.mkdir(parents=True, exist_ok=True)

# -----
# 3. Find Tennessee tract shapefile
# -----
shp_candidates = list(RAW_DIR.rglob("*47*tract*.shp")) + list(RAW_DIR.rglob(

tract_shp = None
for p in shp_candidates:
    name = p.name.lower()
    parent = str(p.parent).lower()
    if "tract" in name and ("47" in name or "tennessee" in name or "state_tr

        tract_shp = p
        break

if tract_shp is None:
    raise FileNotFoundError("Could not find a Tennessee tract shapefile in c

```

```

print("Using tract shapefile:", tract_shp)

# -----
# 4. Load tract geometries
# -----
tracts = gpd.read_file(tract_shp).copy()

# Standardize GEOID field
possible_geoid_cols = ["GEOID", "geoid", "GEOIDFQ", "TRACTCE"]
geoid_col = None
for c in possible_geoid_cols:
    if c in tracts.columns:
        geoid_col = c
        break

if geoid_col is None:
    raise ValueError(f"Could not find a GEOID-like column. Columns: {list(tr

# If only TRACTCE exists, rebuild full GEOID from STATEFP/COUNTYFP/TRACTCE
if geoid_col == "TRACTCE":
    if not all(c in tracts.columns for c in ["STATEFP", "COUNTYFP", "TRACTCE"]):
        raise ValueError("TRACTCE found, but STATEFP/COUNTYFP/TRACTCE not al
    tracts["GEOID"] = (
        tracts["STATEFP"].astype(str).str.zfill(2)
        + tracts["COUNTYFP"].astype(str).str.zfill(3)
        + tracts["TRACTCE"].astype(str).str.zfill(6)
    )
else:
    tracts["GEOID"] = tracts[geoid_col].astype(str).str.extract(r"(\d{11})",
    if tracts["GEOID"].isna().all():
        tracts["GEOID"] = tracts[geoid_col].astype(str).str.zfill(11)

# Keep Tennessee + Shelby County only
if "STATEFP" in tracts.columns and "COUNTYFP" in tracts.columns:
    shelby_all = tracts.loc[
        (tracts["STATEFP"].astype(str).str.zfill(2) == "47") &
        (tracts["COUNTYFP"].astype(str).str.zfill(3) == "157")
    ].copy()
else:
    shelby_all = tracts.loc[tracts["GEOID"].str[:5] == "47157"].copy()

if shelby_all.empty:
    raise ValueError("No Shelby County tracts found in the shapefile.")

cluster_gdf = shelby_all.loc[shelby_all["GEOID"].isin(TARGET_GEOIDS)].copy()

missing = sorted(set(TARGET_GEOIDS) - set(cluster_gdf["GEOID"]))
print("Shelby County tract rows found:", len(shelby_all))
print("Cluster tract rows found:", len(cluster_gdf))
print("Missing GEOIDs:", len(missing))
if missing:
    print(missing[:20])

# Reproject for cleaner plotting if needed
if shelby_all.crs is None:

```

```

    print("Warning: shapefile has no CRS metadata.")
else:
    # Web Mercator is fine for simple plotting
    shelby_all = shelby_all.to_crs(3857)
    cluster_gdf = cluster_gdf.to_crs(3857)

# -----
# 5. Build county boundary from all tracts
# -----
county_outline = shelby_all.dissolve()

# -----
# 6. Plot
# -----
fig, axes = plt.subplots(1, 2, figsize=(16, 8))

# Left panel: county context
shelby_all.plot(ax=axes[0], color="#e6e6e6", edgecolor="#bdbdbd", linewidth=0.5)
cluster_gdf.plot(ax=axes[0], color="#1f78b4", edgecolor="white", linewidth=0.5)
county_outline.boundary.plot(ax=axes[0], color="black", linewidth=1.0)

axes[0].set_title("Shelby County, TN\nFinalist community cluster_77 in blue")
axes[0].set_axis_off()

# Right panel: zoom on selected community
cluster_gdf.plot(ax=axes[1], color="#1f78b4", edgecolor="white", linewidth=0.5)
county_outline.boundary.plot(ax=axes[1], color="black", linewidth=0.8, alpha=0.5)

minx, miny, maxx, maxy = cluster_gdf.total_bounds
pad_x = (maxx - minx) * 0.08
pad_y = (maxy - miny) * 0.08
axes[1].set_xlim(minx - pad_x, maxx + pad_x)
axes[1].set_ylim(miny - pad_y, maxy + pad_y)

axes[1].set_title(
    f"Zoomed finalist community\n{len(cluster_gdf):,} tracts highlighted",
    fontsize=13
)
axes[1].set_axis_off()

fig.suptitle(
    "Geographic footprint of the Shelby County finalist community",
    fontsize=17,
    y=0.98
)

plt.tight_layout()
plt.show()

# -----
# 7. Save outputs
# -----
map_png = OUT_DIR / "shelby_cluster_77_geoid_map.png"
fig.savefig(map_png, bbox_inches="tight", dpi=220)

# Save selected tracts as GeoJSON for reuse

```

```
cluster_out = cluster_gdf.copy()
cluster_out["tract_code"] = cluster_out["GE0ID"].str[5:]
geojson_path = OUT_DIR / "shelby_cluster_77_tracts.geojson"
cluster_out[["GE0ID", "tract_code", "geometry"]].to_file(geojson_path, driver="GeoJSON")

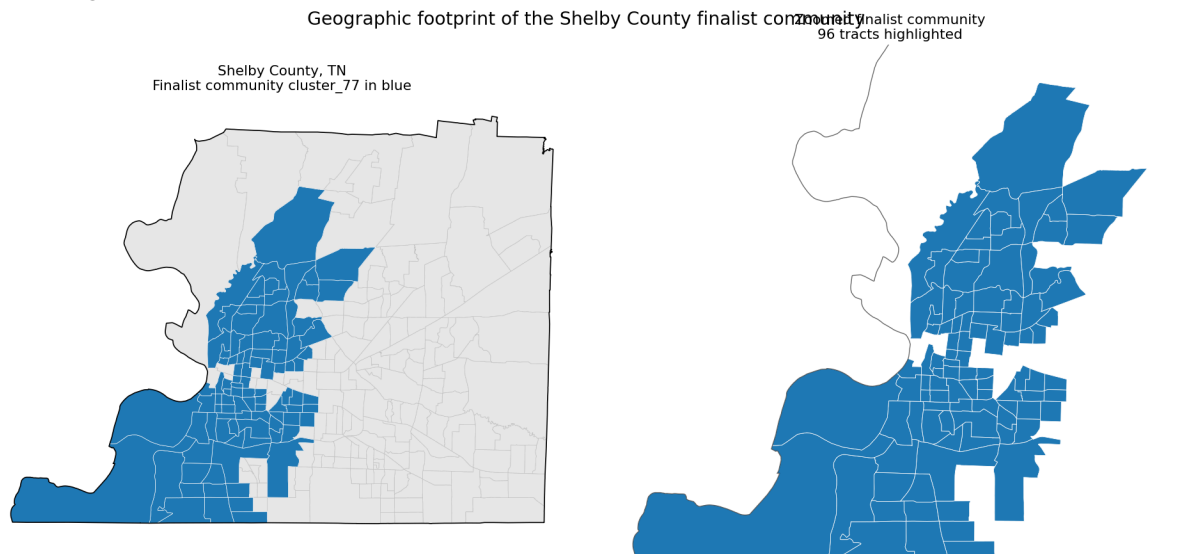
print("\nSaved:")
print("-", map_png)
print("-", geojson_path)
```

Using tract shapefile: C:\Users\jabba\Desktop\Code\machine_learning\AUC_mast
ercard_challenge\src\data\raw\tl_2024_state_tracts\tl_2024_47_tract\tl_2024_
47_tract.shp

Shelby County tract rows found: 249

Cluster tract rows found: 96

Missing GEOIDs: 0



Saved:

```
- C:\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_challenge\src
\data\processed\presentation_ready\shelby_cluster_77_geoid_map.png
- C:\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_challenge\src
\data\processed\presentation_ready\shelby_cluster_77_tracts.geojson
```

```
In [77]: # =====
# Shelby cluster context map (ROBUST VERSION)
# Filters directly from your known GEOIDs
# =====

import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

# -----
# 1. Final Shelby cluster GEOIDs
# -----
TARGET_GEOIDS = [
    "47157000100", "47157000200", "47157000300", "47157000400", "47157000600", "47157000800", "47157000900", "47157001100", "47157001300", "47157001400", "47157001600", "47157001700", "47157001900", "47157002000", "47157002100", "47157002500", "47157003300", "47157003500", "47157003600", "47157003700", "47157003800", "47157003900", "47157004000", "47157004100", "47157004200", "47157004300", "47157004400", "47157004500", "47157004600", "47157004700", "47157004800", "47157004900", "47157005000", "47157005100", "47157005200", "47157005300", "47157005400", "47157005500", "47157005600", "47157005700", "47157005800", "47157005900", "47157006000", "47157006100", "47157006200", "47157006300", "47157006400", "47157006500", "47157006600", "47157006700", "47157006800", "47157006900", "47157007000", "47157007100", "47157007200", "47157007300", "47157007400", "47157007500", "47157007600", "47157007700", "47157007800", "47157007900", "47157008000", "47157008100", "47157008200", "47157008300", "47157008400", "47157008500", "47157008600", "47157008700", "47157008800", "47157008900", "47157009000", "47157009100", "47157009200", "47157009300", "47157009400", "47157009500", "47157009600", "47157009700", "47157009800", "47157009900", "47157010000", "47157010100", "47157010200", "47157010300", "47157010400", "47157010500", "47157010600", "47157010700", "47157010800", "47157010900", "47157011000", "47157011100", "47157011200", "47157011300", "47157011400", "47157011500", "47157011600", "47157011700", "47157011800", "47157011900", "47157012000", "47157012100", "47157012200", "47157012300", "47157012400", "47157012500", "47157012600", "47157012700", "47157012800", "47157012900", "47157013000", "47157013100", "47157013200", "47157013300", "47157013400", "47157013500", "47157013600", "47157013700", "47157013800", "47157013900", "47157014000", "47157014100", "47157014200", "47157014300", "47157014400", "47157014500", "47157014600", "47157014700", "47157014800", "47157014900", "47157015000", "47157015100", "47157015200", "47157015300", "47157015400", "47157015500", "47157015600", "47157015700", "47157015800", "47157015900", "47157016000", "47157016100", "47157016200", "47157016300", "47157016400", "47157016500", "47157016600", "47157016700", "47157016800", "47157016900", "47157017000", "47157017100", "47157017200", "47157017300", "47157017400", "47157017500", "47157017600", "47157017700", "47157017800", "47157017900", "47157018000", "47157018100", "47157018200", "47157018300", "47157018400", "47157018500", "47157018600", "47157018700", "47157018800", "47157018900", "47157019000", "47157019100", "47157019200", "47157019300", "47157019400", "47157019500", "47157019600", "47157019700", "47157019800", "47157019900", "47157020000", "47157020100", "47157020200", "47157020300", "47157020400", "47157020500", "47157020600", "47157020700", "47157020800", "47157020900", "47157021000", "47157021100", "47157021200", "47157021300", "47157021400", "47157021500", "47157021600", "47157021700", "47157021800", "47157021900", "47157022000", "47157022100", "47157022200", "47157022300", "47157022400", "47157022500", "47157022600", "47157022700", "47157022800", "47157022900", "47157023000", "47157023100", "47157023200", "47157023300", "47157023400", "47157023500", "47157023600", "47157023700", "47157023800", "47157023900", "47157024000", "47157024100", "47157024200", "47157024300", "47157024400", "47157024500", "47157024600", "47157024700", "47157024800", "47157024900", "47157025000", "47157025100", "47157025200", "47157025300", "47157025400", "47157025500", "47157025600", "47157025700", "47157025800", "47157025900", "47157026000", "47157026100", "47157026200", "47157026300", "47157026400", "47157026500", "47157026600", "47157026700", "47157026800", "47157026900", "47157027000", "47157027100", "47157027200", "47157027300", "47157027400", "47157027500", "47157027600", "47157027700", "47157027800", "47157027900", "47157028000", "47157028100", "47157028200", "47157028300", "47157028400", "47157028500", "47157028600", "47157028700", "47157028800", "47157028900", "47157029000", "47157029100", "47157029200", "47157029300", "47157029400", "47157029500", "47157029600", "47157029700", "47157029800", "47157029900", "47157030000", "47157030100", "47157030200", "47157030300", "47157030400", "47157030500", "47157030600", "47157030700", "47157030800", "47157030900", "47157031000", "47157031100", "47157031200", "47157031300", "47157031400", "47157031500", "47157031600", "47157031700", "47157031800", "47157031900", "47157032000", "47157032100", "47157032200", "47157032300", "47157032400", "47157032500", "47157032600
```

```

"47157004300", "47157004500", "47157004600", "47157005000", "47157005300", "4
"47157005600", "47157005700", "47157005800", "47157005900", "47157006000", "4
"47157006300", "47157006400", "47157006500", "47157006700", "47157006800", "4
"47157007000", "47157007100", "47157007200", "47157007300", "47157007400", "4
"47157007810", "47157007821", "47157007822", "47157007900", "47157008000", "4
"47157008120", "47157008900", "47157009901", "47157009902", "47157010001", "4
"47157010120", "47157010121", "47157010122", "47157010210", "47157010220", "4
"47157011100", "47157011200", "47157011401", "47157011402", "47157011500", "4
"47157011700", "47157020221", "47157020511", "47157020521", "47157020523", "4
"47157020531", "47157020532", "47157020543", "47157022111", "47157022121", "4
"47157022210", "47157022220", "47157022310", "47157022321", "47157022322", "4
"47157022410", "47157022500", "47157022700", "47157980100", "47157980200", "4
]

TARGET_PREFIX = "47157"    # Shelby County, TN

# -----
# 2. Find folders
# -----
ROOT_CANDIDATES = [Path.cwd(), Path.cwd() / "src", Path.cwd().parent]

RAW_DIR = None
PROCESSED_DIR = None

for root in ROOT_CANDIDATES:
    if (root / "src/data/raw").exists():
        RAW_DIR = root / "src/data/raw"
        PROCESSED_DIR = root / "src/data/processed"
        break
    if (root / "data/raw").exists():
        RAW_DIR = root / "data/raw"
        PROCESSED_DIR = root / "data/processed"
        break

if RAW_DIR is None:
    raise FileNotFoundError("Could not find data/raw or src/data/raw.")

OUT_DIR = PROCESSED_DIR / "presentation_ready"
OUT_DIR.mkdir(parents=True, exist_ok=True)

# -----
# 3. Find shapefiles
# -----
def find_first_match(patterns):
    for pat in patterns:
        hits = list(RAW_DIR.rglob(pat))
        if hits:
            return hits[0]
    return None

state_shp = find_first_match([
    "tl_2024_us_state/*.shp",
    "*us_state*.shp",
])

# prefer a tract shapefile inside tl_2024_state_tracts

```

```

tract_candidates = list(RAW_DIR.rglob("tl_2024_state_tracts/*.shp")) + list(
    tract_shp = tract_candidates[0] if tract_candidates else None

place_shp = find_first_match([
    "tl_2024_47_place/*.shp",
    "*47_place*.shp",
    "*place*.shp",
])

if state_shp is None:
    raise FileNotFoundError("Could not find state shapefile.")
if tract_shp is None:
    raise FileNotFoundError("Could not find tract shapefile.")

print("Using state shapefile:", state_shp)
print("Using tract shapefile:", tract_shp)
print("Using place shapefile:", place_shp if place_shp else "None found")

# -----
# 4. Load states
# -----
states = gpd.read_file(state_shp).copy()
states = states.to_crs(3857)

# standardize abbreviation column if needed
if "STUSPS" not in states.columns and "STATEFP" in states.columns:
    fp_to_abbr = {
        "01": "AL", "05": "AR", "21": "KY", "28": "MS", "47": "TN", "22": "LA", "13": "GA"
    }
    states["STUSPS"] = states["STATEFP"].astype(str).str.zfill(2).map(fp_to_

STATE_ABBRS = ["TN", "AR", "MS", "AL", "KY", "MO", "GA"]
states_ctx = states.loc[states["STUSPS"].isin(STATE_ABBRS)].copy()
tn_state = states_ctx.loc[states_ctx["STUSPS"] == "TN"].copy()

# -----
# 5. Load tracts and build GEOID robustly
# -----
tracts = gpd.read_file(tract_shp).copy()
print("\nTRACT COLUMNS:\n", list(tracts.columns))

# Build a normalized 11-digit tract GEOID
if all(c in tracts.columns for c in ["STATEFP", "COUNTYFP", "TRACTCE"]):
    tracts["GEOID_11"] = (
        tracts["STATEFP"].astype(str).str.zfill(2)
        + tracts["COUNTYFP"].astype(str).str.zfill(3)
        + tracts["TRACTCE"].astype(str).str.zfill(6)
    )
elif "GEOID" in tracts.columns:
    tracts["GEOID_11"] = tracts["GEOID"].astype(str).str.extract(r"(\d{11})")
elif "geoid" in tracts.columns:
    tracts["GEOID_11"] = tracts["geoid"].astype(str).str.extract(r"(\d{11})")
elif "GEOIDFQ" in tracts.columns:
    tracts["GEOID_11"] = tracts["GEOIDFQ"].astype(str).str.extract(r"(\d{11})")
else:
    raise ValueError("Could not construct tract GEOID. Inspect the printed t

```



```

tracts["GE0ID_11"] = tracts["GE0ID_11"].astype(str).str.zfill(11)

# directly isolate Shelby from GE0ID prefix
shelby_all = tracts.loc[tracts["GE0ID_11"].str.startswith(TARGET_PREFIX)].copy()
cluster_gdf = shelby_all.loc[shelby_all["GE0ID_11"].isin(TARGET_GE0IDS)].copy()

print("\nShelby tract rows found from prefix:", len(shelby_all))
print("Selected cluster tract rows found:", len(cluster_gdf))

missing = sorted(set(TARGET_GE0IDS) - set(cluster_gdf["GE0ID_11"]))
print("Missing GE0IDs:", len(missing))
if missing:
    print("First missing examples:", missing[:10])

if cluster_gdf.empty:
    raise ValueError(
        "Still found zero selected Shelby tracts. The tract shapefile is likely empty."
    )

# reproject
shelby_all = shelby_all.to_crs(3857)
cluster_gdf = cluster_gdf.to_crs(3857)

shelby_outline = shelby_all.dissolve()
cluster_outline = cluster_gdf.dissolve()

# -----
# 6. Load place labels if available
# -----
place_points = None
if place_shp is not None:
    places = gpd.read_file(place_shp).copy()
    places = places.to_crs(3857)

    name_col = None
    for c in ["NAME", "NAMELSAD", "name"]:
        if c in places.columns:
            name_col = c
            break

    if name_col is not None:
        places["label_name"] = places[name_col].astype(str).str.replace(" city", "")
        wanted = ["Memphis", "Nashville-Davidson", "Nashville", "Jackson", "Birmingham"]
        place_points = places.loc[places["label_name"].isin(wanted)].copy()

# fallback city points
fallback_places = pd.DataFrame({
    "label_name": ["Memphis", "Nashville", "Little Rock", "Jackson, MS", "Birmingham"],
    "lon": [-90.0490, -86.7816, -92.2896, -90.1848, -86.8025],
    "lat": [35.1495, 36.1627, 34.7465, 32.2988, 33.5186],
})
fallback_places = gpd.GeoDataFrame(
    fallback_places,
    geometry=gpd.points_from_xy(fallback_places["lon"], fallback_places["lat"]),
    crs=4326
)

```

```

).to_crs(3857)

# label points
state_labels = states_ctx.copy()
state_labels["label_point"] = state_labels.representative_point()
cluster_label_pt = cluster_outline.representative_point().iloc[0]

# -----
# 7. Plot
# -----

fig, axes = plt.subplots(1, 2, figsize=(18, 9))

# LEFT: regional context
states_ctx.plot(ax=axes[0], color="#f2f2f2", edgecolor="#999999", linewidth=1)
tn_state.plot(ax=axes[0], color="#d9e8fb", edgecolor="#2b6cb0", linewidth=1)

# plot Shelby cluster footprint on regional map
cluster_outline.plot(ax=axes[0], color="#1f78b4", edgecolor="white", linewidth=1)

for _, row in state_labels.iterrows():
    pt = row["label_point"]
    axes[0].text(pt.x, pt.y, row["NAME"], fontsize=10, ha="center", va="center",
                 color="dimgray", weight="bold")

if place_points is not None and not place_points.empty:
    for _, row in place_points.iterrows():
        pt = row.geometry.representative_point() if row.geometry.geom_type != "Point" else row.geometry
        axes[0].plot(pt.x, pt.y, "o", markersize=3, color="black")
        axes[0].text(pt.x + 15000, pt.y + 15000, row["label_name"], fontsize=10, ha="center", va="center",
                     color="dimgray", weight="bold")
else:
    for _, row in fallback_places.iterrows():
        pt = row.geometry
        axes[0].plot(pt.x, pt.y, "o", markersize=3, color="black")
        axes[0].text(pt.x + 15000, pt.y + 15000, row["label_name"], fontsize=10, ha="center", va="center",
                     color="dimgray", weight="bold")

minx, miny, maxx, maxy = states_ctx.total_bounds
pad_x = (maxx - minx) * 0.03
pad_y = (maxy - miny) * 0.03
axes[0].set_xlim(minx - pad_x, maxx + pad_x)
axes[0].set_ylim(miny - pad_y, maxy + pad_y)
axes[0].set_title("Regional context: Tennessee and nearby states", fontsize=12, color="dimgray")
axes[0].set_axis_off()

# RIGHT: Shelby county zoom
shelby_all.plot(ax=axes[1], color="#ededed", edgecolor="#c0c0c0", linewidth=1)
cluster_gdf.plot(ax=axes[1], color="#1f78b4", edgecolor="white", linewidth=1)
shelby_outline.boundary.plot(ax=axes[1], color="black", linewidth=1.0)

axes[1].text(
    cluster_label_pt.x, cluster_label_pt.y,
    "Selected Shelby community",
    fontsize=10, ha="center", va="center",
    bbox=dict(boxstyle="round,pad=0.2", fc="white", ec="gray", alpha=0.9)
)

# Memphis label

```

```

mem_pt = fallback_places.loc[fallback_places["label_name"] == "Memphis"].iloc[0]
axes[1].plot(mem_pt.x, mem_pt.y, "o", markersize=4, color="black")
axes[1].text(mem_pt.x + 6000, mem_pt.y + 6000, "Memphis", fontsize=10, color="black")

minx, miny, maxx, maxy = shelly_all.total_bounds
pad_x = (maxx - minx) * 0.05
pad_y = (maxy - miny) * 0.05
axes[1].set_xlim(minx - pad_x, maxx + pad_x)
axes[1].set_ylim(miny - pad_y, maxy + pad_y)
axes[1].set_title(f"Shelby County zoom: finalist community ({len(cluster_gdf)})")
axes[1].set_axis_off()

fig.suptitle("Shelby County finalist community with state and city context",
plt.tight_layout()
plt.show()

# save
out_png = OUT_DIR / "shelby_cluster_77_context_map.png"
fig.savefig(out_png, bbox_inches="tight", dpi=220)

print("\nSaved:")
print("-", out_png)

```

Using state shapefile: C:\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_challenge\src\data\raw\tl_2024_us_state\tl_2024_us_state\tl_2024_us_state.shp

Using tract shapefile: C:\Users\jabba\Desktop\Code\machine_learning\AUC_mastercard_challenge\src\data\raw\tl_2024_us_ttract\tl_2024_us_ttract.shp

Using place shapefile: None found

TRACT COLUMNS:

```
['AIANNHCE', 'TTRACTCE', 'GEOID', 'GEOIDFQ', 'NAME', 'NAMELSAD', 'MTFCC', 'ALAND', 'AWATER', 'INTPTLAT', 'INTPTLON', 'geometry']
```

Shelby tract rows found from prefix: 0

Selected cluster tract rows found: 0

Missing GEOIDs: 96

First missing examples: ['47157000100', '47157000200', '47157000300', '47157000400', '47157000600', '47157000700', '47157000800', '47157000900', '47157001100', '47157001300']

```

-----
ValueError                                Traceback (most recent call last)
Cell In[77], line 147
    144     print("First missing examples:", missing[:10])
    146     if cluster_gdf.empty:
--> 147         raise ValueError(
    148             "Still found zero selected Shelby tracts. The tract shapefile is likely not the correct Census tract file."
    149         )
    151     # reproject
    152     shelly_all = shelly_all.to_crs(3857)

```

ValueError: Still found zero selected Shelby tracts. The tract shapefile is likely not the correct Census tract file.

In []:

In []:

In []:

In []:

In []: